Prof. Dr.-Ing. A. Siggelkow

Specification - **RWU-RV64I**

Hochschule Ravensburg-Weingarten
University of Applied Sciences

B

Chapter 1

Requirements

Table 1.1: Functional Requirements

Requirement	ID	Importance	Verifiable	Description	Remarks
General					
Gen.: #persons	G01	High	VHDL-TB	The number of persons in a room must be counted.	
Gen.: max	G02	High	VHDL-TB	The number of persons in a room must not exceed a given limit.	
Gen.: only one pers.	G03	High	?	Only one person can either enter or leave the room at a time.	Check test
Gen.: three light sensors	G04	Medium	VHDL-TB	Along the doorway, there are three light-curtains to allow direction-tracking of possible visitors.	Why not only two?

Chapter 2

Document Overview

2.1 Validity of this document

This document is valid for **RWU-RV64I** version:

- V1.0: First samples
- V1.x: Revision

2.2 Target Specification Status

This **RWU-RV64I** target specification is initially derived from the **RV64I** specification. So readers being familiar with the **RV64I** architecture concepts and hardware building blocks will find most of them unchanged within this document.

This **RWU-RV64I** document represents and describes all features of the **RWU-RV64I** and is adequate as a standalone reference.

This release of the specification has the status of an target specification.

2.3 Major Changes since last Revision

Table 2.1: Spec Changes as compared to the previous **RWU-RV64I** Spec Revision

History		
Target Spec.	Current version : 1.0, 2022-11-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)

Contents

List of Figures

List of Tables

2.4 Glossary

Table 2.2: Glossary Type Descriptions

Type	Description
UART	Universal Asynchronous Receiver Transmitter
Bus	Binary Unit System
RGB	Red-Green-Blue. A color mixing system.
VGA	Video Graphics Array. A computer graphics standard.

2.5 Naming Conventions

Table 2.3: Alias Names for Cores and other Objects in the Spec

Object	Names also used in the spec
ARM926EJ-S ™	ARM, Controller, Micro controller unit, MCU, Processor, CPU
Bus	Binary Unit System

2.6 Document List

Table 2.4: History

History		
Target Spec.	Current version : 1.0, 2022-11-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2022-11-22		
	2022-11-22	First draft (AS)

The following documents are referred to in this document. They are either available in a separate attachment directory or are general standards.

Table 2.5: Document List

Nr	Doc Type	Version	Date	Filename	Attached
1	ARM engineering specification	A09	15.10.2024	ARM926EJ-S_EngSpec_A09	No
2	ARM926 technical reference manual	-	16.10.2024	DDI0198B_926	Yes

Chapter 3

Product Overview

3.1 Top Level View

Table 3.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.1.1 Introduction

The **RWU-RV64I** is a RISC-V processor with the integer 64 bit instruction set.

Figure 3.1 shows the top level of the chip.

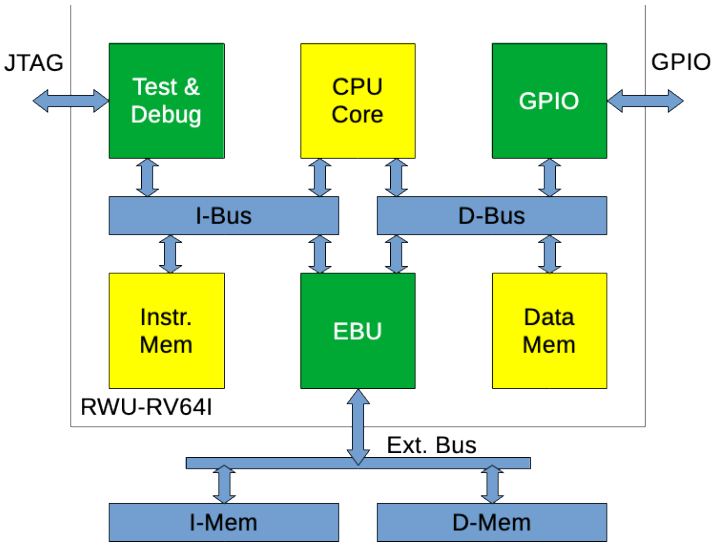

Figure 3.1: **RWU-RV64I** Overview

3.1.2 Key Features

First version, single cycle, on-chip memory only:

- RISC-V RV64I core
- Instruction memory: 65 kB
- Data memory: 65 kB
- JTAG, for loading the I-Mem

3.1.3 Functional Block Diagram

This block diagram shows only the most important architectural features. To maintain legibility some aspects have been simplified. More details of architecture, concepts and block integration are contained in other chapters of this specification.

Figure 3.2 shows a simplified block diagram of the chip.

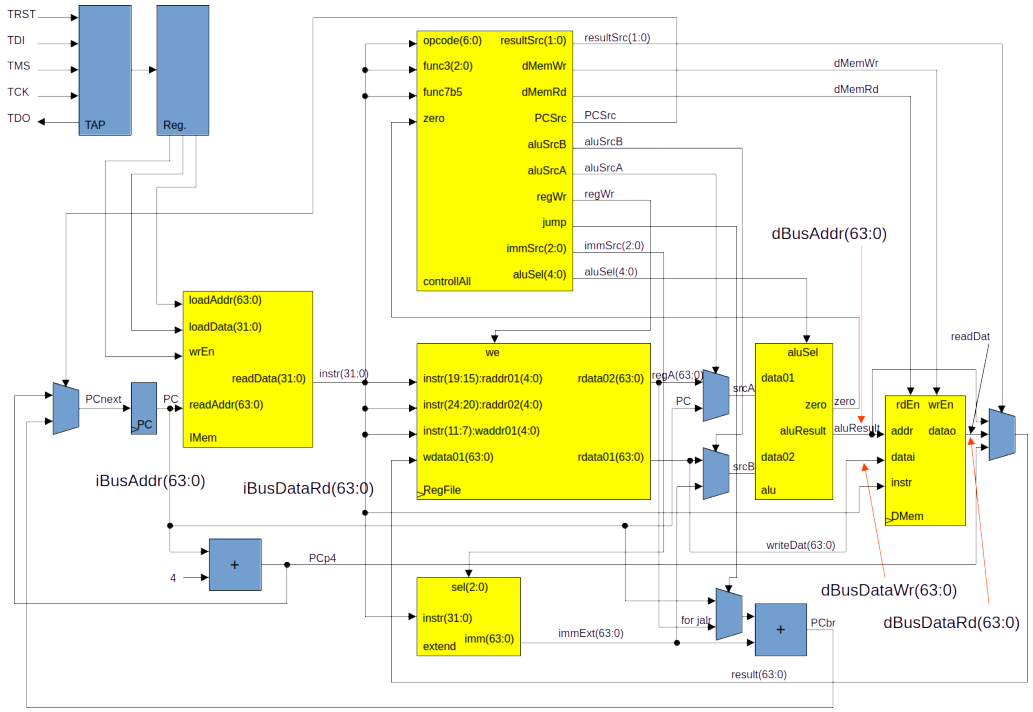

Figure 3.2: Simplified Block diagram

3.1.4 Package

The package is not defined yet.

3.2 System View

Table 3.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The system is not defined yet.

3.3 Architecture Concepts Overview

Table 3.3: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.3.1 Technology

Europractice 90 nm technology.

Infineon's 90 nm (70 nm physical gate length) CMOS technology is designed for digital and analog circuit design. It offers advanced possibilities for low power design (triple well, low leakage transistor types). The technology has 4-9 layers of copper metallization. It offers an increase in peak performance of more than 50% and a reduction of active power consumption of more than a factor of 2. Compared to the predecessor technology twice the cell density for standard cells can be achieved in L90.

The nominal core voltage is 1.2 V, IO devices have a nominal supply voltage of 2.5 V or 1.8 V.

3.3.2 System Memory Concept

The first version has an on-chip instruction memory of ?? MB and an on-chip data memory of ?? MB. The technology is ??.

3.3.3 Software Architecture

The RISC-V compiler ??? is supported.

3.3.4 Interprocessor Communication Concept

Planned: UART and (Q)SPI.

3.3.5 Bus Concept

The main bus system is compliant to the Wishbone specification.

There is on WB bus for the instructions and one for the data.

3.3.6 Clock Concept

Not defined yet.

Until now, there is only one clock input. For a next generation, with external NOR flash memory for instruction and NAND flash for data, a QSPI interface will be needed. This requires a faster clock than the CPU clock.

3.3.7 System Control Concept

System control will be needed for:

- Reset and boot
- Interrupt

3.3.8 Interrupt Concept

Must be implemented.

3.3.9 Debug Concept

Must be implemented.

3.3.10 Power Management

Must be implemented.

3.3.11 Protection and Security Concept

Must be implemented.

3.4 Functional Block Overview

Table 3.4: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

3.4.1 RISC-V Core

The core is a standard compliant RISC-V with 64 bit and integer implementation only. It is single cycle, without cache and memory hierarchy.

3.4.2 Instruction Memory

Instruction memory:

3.4.3 Data Memory

Data memory:

3.4.4 Wishbone Peripherals

GPIO

There are 8 GPIO pins implemented. Together with the data, an address will be given to the pins. With this, 16 times 8 GPIOs can be driven with just one external decoder.

3.4.5 Debug

The test and debug will be done via a JTAG interface. Until now, the loading of the instruction memory is possible.

Chapter 4

Architecture Concepts

The logic style used here is as often as possible a "one pulse" logic.

Figure 4.1 shows the "pulsed" logic.

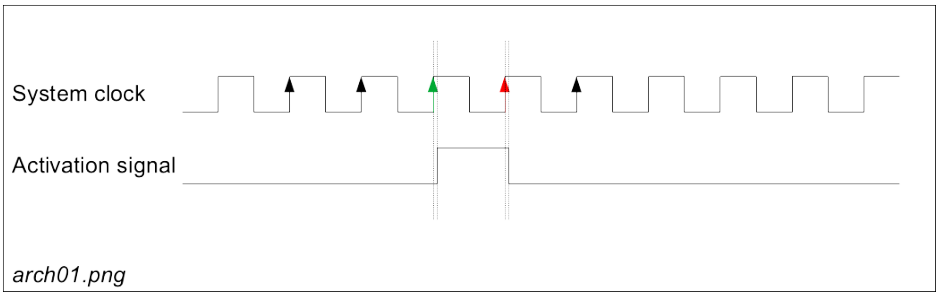

Figure 4.1: Enable with a Pulse

An "activation signal" will be generated synchronously with the "system clock" by its rising edge (green). This results in a delayed generation of the rising edge of the "activation signal". With the next rising edge of the "system clock" (red), the "activation signal" will be removed. Also here, the falling edge of the "activation signal" will have a small delay to the clock. So, the "activation signal" is exactly on "system clock" period high. This pair of signals can now be taken to start a following device (e.g. a counter), the counter will count only when the activation signal is high and the system clock will show a rising edge (red).

4.1 Technology

Table 4.1: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.2 CPU Concept

Table 4.2: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

The **RWU-RV64I** is a single cycle implementation without pipeline, memory hierarchy, MMU, etc. It is a students test chip.

Table 4.3 shows the notations which are used in the explanations for the instructions in tables 4.4 and 4.5.

Table 4.3: Notations for Instructions

Notation	Description
pc	program counter
rd	integer register destination
rsN	integer register source N
imm	immediate operand value
offs	immediate program counter relative offset
ux(reg)	unsigned XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
sx(reg)	signed XLEN-bit integer (32-bit on RV32, 64-bit on RV64)
uN(reg)	zero extended N-bit integer register value
sN(reg)	sign extended N-bit integer register value
uN[reg + imm]	unsigned N-bit memory reference
sN[reg + imm]	signed N-bit memory reference

Table 4.4 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 4.4: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Example	Meaning
Load Upper Immediate	*lui rd, imm*	$rd \leftarrow imm$
Add Upper Immediate to PC	*auipc rd, offs*	$rd \leftarrow pc + offs$
Jump and Link	*jal rd, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow pc + offs$
Jump and Link Register	*jalr rd, rs1, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	*beq rs1, rs2, offs*	$if\ rs1 = rs2\ then\ pc \leftarrow pc + offs$
Branch Not Equal	*bne rs1, rs2, offs*	$if\ rs1 \neq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than	*blt rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal	*bge rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$

... next page

Instruction	Example	Meaning
Branch Less Than Unsigned	*bltu rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	*bgeu rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Load Byte	*lb rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half	*lh rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Load Word	*lw rd, offs(rs1)*	$rd \leftarrow s32[rs1 + offs]$
Load Byte Unsigned	*lbu rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	*lhu rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Store Byte	*sb rs2, offs(rs1)*	$u8[rs1 + offs] \leftarrow rs2$
Store Half	*sh rs2, offs(rs1)*	$u16[rs1 + offs] \leftarrow rs2$
Store Word	*sw rs2, offs(rs1)*	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	*addi rd, rs1, imm*	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	*slti rd, rs1, imm*	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	*sltiu rd, rs1, imm*	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	*xori rd, rs1, imm*	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	*ori rd, rs1, imm*	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	*andi rd, rs1, imm*	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	*add rd, rs1, rs2*	$rd \leftarrow sx(rs1) + sx(rs2)$
Subtract	*sub rd, rs1, rs2*	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	*sll rd, rs1, rs2*	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	*slt rd, rs1, rs2*	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	*sltu rd, rs1, rs2*	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	*xor rd, rs1, rs2*	$rd \leftarrow ux(rs1) \oplus ux(rs2)$
Shift Right Logical	*srl rd, rs1, rs2*	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	*sra rd, rs1, rs2*	$rd \leftarrow sx(rs1) >> rs2$
Or	*or rd, rs1, rs2*	$rd \leftarrow ux(rs1) \vee ux(rs2)$

... next page

Instruction	Example	Meaning
And	*and rd, rs1, rs2*	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	*fence pred, succ*	
Fence Instruction	*fence.i*	

End of table.

Table 4.5 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 4.5: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	*lwu rd, offs(rs1)*	$rd \leftarrow u32[rs1 + offs]$
Load Double	*ld rd, offs(rs1)*	$rd \leftarrow u64[rs1 + offs]$
Store Double	*sd rs2, offs(rs1)*	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << sx(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> sx(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> sx(imm)$
Add Immediate Word	*addiw rd, rs1, imm*	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	*slliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) << imm)$
Shift Right Logical Immediate Word	*srliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) >> imm)$
Shift Right Arithmetic Immediate Word	*sraiw rd, rs1, imm*	$rd \leftarrow s32(rs1) >> imm$
Add Word	*addw rd, rs1, rs2*	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	*subw rd, rs1, rs2*	$rd \leftarrow s32(rs1) - s32(rs2)$
Shift Left Logical Word	*sllw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) << rs2)$
Shift Right Logical Word	*srlw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) >> rs2)$

... next page

Instruction	Example	Meaning
Shift Right Arithmetic Word	*sraw rd, rs1, rs2*	$rd \leftarrow s32(rs1) >> rs2$

End of table.

4.3 Bus Concept

Table 4.6: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22 2026-02-19	First draft (AS) First fill in cooperation with Claude-AI (AS)

4.3.1 Overview

The RWU-RV64I chip uses a **dual-bus architecture** with two independent on-chip buses serving different purposes:

- **Wishbone B4 Classic** – the peripheral bus. All configuration registers, GPIO, UART, and QSPI control registers are connected to the CPU via this bus. It was chosen for its simplicity, open-source IP availability, and compatibility with the existing peripheral IP blocks.
- **AMBA AXI4** (read-only subset) – the cache memory bus. The I-Cache and D-Cache controllers use this bus to fetch 32-byte cache lines from NOR Flash via the Memory Arbiter and the QSPI controller data port. AXI4 was chosen for its burst capability: a 4-beat INCR burst fills one cache line in a single transaction instead of eight independent Wishbone single-cycle transfers.

The two buses are completely independent. There is no bridge between them. The Wishbone bus carries only single-word register accesses; the AXI4 bus carries

only 32-byte cache-line bursts. Table 4.7 summarises the key parameters of each bus.

Table 4.7: On-Chip Bus Summary

Property	Wishbone B4 Classic	AXI4 (cache subset)	Comment
Purpose	Peripheral access	Cache line fill	
Standard	Wishbone B4 Classic	AMBA AXI4	
Data width	64 bit	64 bit	
Address width	64 bit	32 bit	Physical address only
Transfer type	Single-word	4-beat INCR burst	ARLEN=3, ARSIZE=3
Burst support	No	Yes (read only)	Write channels eliminated
Masters	CPU core	I-Cache, D-Cache	
Slaves	GPIO, UART, QSPI cfg regs	QSPI controller (data)	
Arbitration	Single master (no arbiter)	Fixed priority (D > I)	Memory Arbiter
Error signalling	None (ERR_O not driven)	RRESP≠OKAY	Maps to RISC-V exceptions

4.3.2 Bus Topology

Figure 4.2 shows the complete on-chip bus topology. The two buses are rendered in different colours: Wishbone in blue, AXI4 in red. The QSPI controller is the only block that participates in both buses (configuration registers on Wishbone, data port on AXI4).

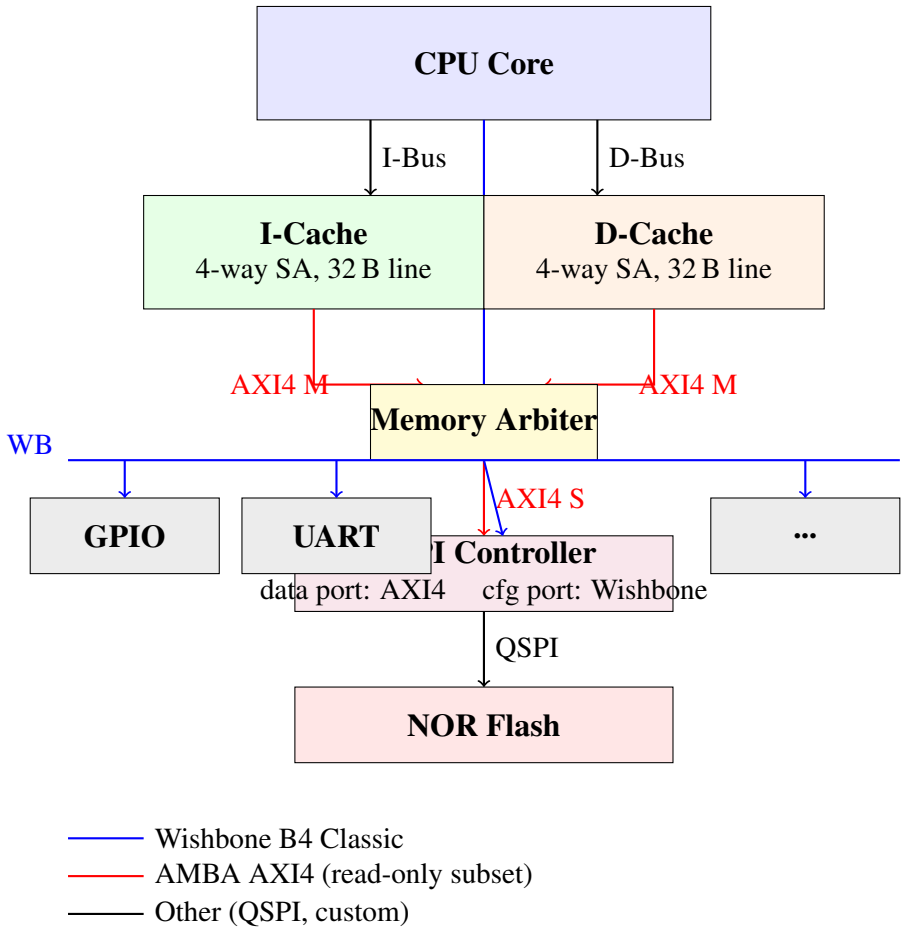

Figure 4.2: On-Chip Bus Topology – Dual-Bus Architecture

4.3.3 Bus Selection Rationale

Table 4.8 documents the decision to use a dual-bus architecture rather than a single unified bus for all on-chip traffic.

Table 4.8: Bus Selection Rationale

Criterion	Wishbone (peripherals)	AXI4 (cache)
Transfer pattern	Single register read/write; one word per transaction	32-byte cache line fill; 4 beats per transaction
Burst support	Not needed; register accesses are always single-word	Essential; without bursts, 8 independent transactions per cache miss instead of 1
Latency	Zero wait states (combinatorial ACK); adequate for register accesses	Many cycles per miss; decoupled AR and R channels allow the QSPI controller to accept the address and then stream data without stalling the bus
Complexity	Very simple; no state machines required in the BPI	Higher; separate address/-data channels, handshaking, ID tracking – justified by the burst benefit
IP compatibility	All existing peripheral IP blocks use Wishbone; no changes required	Cache controllers are new IP; designed for AXI4 from the start
Write support	Full read/write (register access)	Read-only (write channels eliminated by MEM-01 resolution; Flash is read-only)
Decision	**Keep Wishbone for all peripherals.** Adding AXI4 to GPIO/UART etc. would add complexity with no benefit.	**Use AXI4 for cache–memory path.** The 8× reduction in bus transactions per cache miss justifies the added complexity.

4.3.4 Wishbone B4 Classic – Peripheral Bus

Signal Naming Convention

Wishbone signal names in this design follow the convention shown in Table 4.9. The suffix `_i` indicates an input, `_o` an output, and `_s` an internal wire. The prefix `wb_s_` is used for slave-side signals and `wb_m_` for master-side signals.

Table 4.9: Wishbone Signal Naming

Wishbone Name	Slave Port	Master Port	Width	Description
ADR	wb_s_addr_i	wb_m_addr_o	64	Address bus
DAT_I	wb_s_dat_i	wb_m_dat_i	64	Data from master to slave (write data)
DAT_O	wb_s_dat_o	wb_m_dat_o	64	Data from slave to master (read data)
WE	wb_s_we_i	wb_m_we_o	1	Write enable: 1 = write, 0 = read
SEL	wb_s_sel_i	wb_m_sel_o	8	Byte select (all bytes must be set)
STB	wb_s_stb_i	wb_m_stb_o	1	Strobe: valid cycle indicator from master
ACK	wb_s_ack_o	wb_m_ack_i	1	Acknowledge: normal end of transfer from slave
CYC	wb_s_cyc_i	wb_m_cyc_o	1	Cycle: bus cycle active (held for complete transfer)

Transfer Protocol

A Wishbone Classic single transfer proceeds as follows:

1. The master asserts CYC and STB, presents a valid address on ADR, drives WE (1 for write, 0 for read), sets SEL to all-ones, and – for a write – provides data on DAT_O.

2. In this implementation the slave responds combinatorially in the same clock cycle: it drives read data on DAT_O and asserts ACK.

3. The transaction is complete when both STB and ACK are seen high on the rising clock edge. The master may then begin a new transfer in the very next cycle.

A transfer is considered valid only when both CYC and STB are asserted simultaneously and all byte-select bits are set. If any byte-select bit is clear the BPI treats the access as invalid and suppresses both the ACK and the write enable to the peripheral.

The timing diagram below illustrates a write followed by a read.

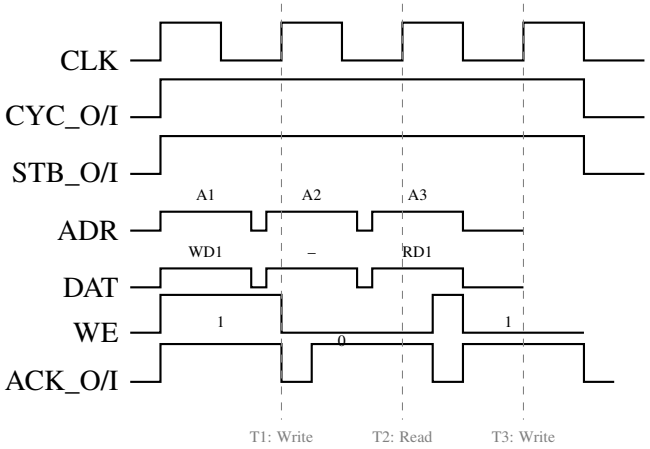

Figure 4.3: Wishbone Classic Single Transfers: Write – Read – Write

Byte Select

The 8-bit SEL signal indicates which byte lanes carry valid data. Bit n of SEL corresponds to byte lane n (bits $8n + 7 \ldots 8n$) of the 64-bit data bus. In the current implementation, the master BPI always drives all eight SEL bits to 1 (full-word transfers only). The slave BPI checks that all SEL bits are set before asserting ACK or issuing a write enable to the peripheral kernel. Sub-word accesses are therefore not supported.

Reset Behaviour

All Wishbone outputs are driven to their inactive state while the synchronous active-high reset rst_i is asserted. For the master BPI this means STB_O and CYC_O are held low; for the slave BPI ACK_O is held low. Note that in the current

purely combinatorial implementation the reset is not registered inside the BPI modules; the reset is applied by the surrounding peripheral logic.

4.3.5 AMBA AXI4 – Cache Memory Bus

Overview

The AXI4 bus connects the two cache controllers (I-Cache and D-Cache) through the Memory Arbiter to the QSPI controller data port. It carries exclusively 32-byte cache-line read bursts. Write channels (AW, W, B) are not instantiated: the AXI4 write path was eliminated when the scratchpad SRAM replaced write-back-to-Flash (MEM-01 resolution).

Participants

Table 4.10: AXI4 Bus Participants

Block	Role	Notes
I-Cache controller	AXI4 Master (ID=0x1)	Issues read burst on cache miss; read-only
D-Cache controller	AXI4 Master (ID=0x2)	Issues read burst on cache miss; read-only
Memory Arbiter	AXI4 Slave + Master	Accepts two masters; forwards one at a time to QSPI; fixed priority D-Cache > I-Cache
QSPI controller (data port)	AXI4 Slave	Translates AXI4 read burst to Quad Fast Read SPI command; slow slave (many cycles per burst)

Bus Parameters

The AXI4 bus uses a reduced parameter set: all transfers are fixed-length 4-beat INCR read bursts.

Table 4.11: AXI4 Cache Bus Parameters

Parameter	Value	Comment
Protocol	AXI4 (not AXI4-Lite)	Burst support required
Data width	64 bit	One RV64 doubleword per beat
Address width	32 bit	Physical address (`PA_WIDTH=32`)
ID width	4 bit	I-Cache: `4'h1`; D-Cache: `4'h2`
Burst type	INCR only	Sequential address; no WRAP, no FIXED
Burst length	ARLEN = 3	4 beats × 8 B = 32 B per cache line
Beat size	ARSIZE = 3	8 bytes per beat
Channels used	AR, R only	AW, W, B channels not instantiated
Outstanding tx	1 per master	Each cache has at most 1 transaction in flight

The full AXI4 channel signal definitions, the SystemVerilog interface `as_axi4_if`, and the AXI4 timing diagrams are documented in the Memory Concept section (Section “Memory Concept”, subsection “AXI4 Memory Bus”).

QSPI Controller – Dual Bus Participation

The QSPI controller is the only block that participates in both buses:

- **AXI4 slave port (data):** Receives 4-beat INCR read bursts from the Memory Arbiter. Translates each burst into a Quad Fast Read (`0x6B`) command sequence on the SPI bus and returns 32 bytes in 4 AXI4 R-channel beats. This port is part of the AXI4 cache memory bus.

- **Wishbone slave port (configuration):** Receives single-word register accesses from the CPU for QSPI mode configuration, status readback, and manual SPI command injection. This port is part of the Wishbone peripheral bus and uses the standard `as_slave_bpi` wrapper.

The two ports are independent and may be active simultaneously. The QSPI controller arbitrates internally between the AXI4 data path and the Wishbone configuration path, with the AXI4 burst taking priority over a concurrent Wishbone access.

4.3.6 Wishbone Slave BPI (`as_slave_bpi`)

History

Table 4.12: History of as_slave_bpi

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, zero wait states

Functional Configuration of as_slave_bpi

The `as_slave_bpi` module implements the Wishbone B4 Classic slave interface. It converts Wishbone bus signals into simple register-file read and write enables for the peripheral kernel. Table 4.13 lists the configurable parameters.

Table 4.13: Functional Configuration of as_slave_bpi

Parameter	Type	Default	Description
`addr_width`	integer	64	Width of the address bus in bits
`data_width`	integer	64	Width of the data bus in bits; also determines SEL width as `data_width/8`

HW Interfaces

Table 4.14 lists all input and output ports of the `as_slave_bpi` module.

Table 4.14: I/O of as_slave_bpi

Port	Direction	Width	Description
rst_i	in	1	Synchronous reset, active high
clk_i	in	1	Bus clock
Peripheral kernel side:			
addr_o	out	64	Decoded register address to peripheral
dat_from_core_i	in	64	Read data returned by peripheral kernel
dat_to_core_o	out	64	Write data forwarded to peripheral kernel
wr_o	out	1	Write enable to peripheral: asserted for one cycle on a valid write
rd_o	out	1	Read enable to peripheral: asserted for one cycle on a valid read
Wishbone bus side:			
wb_s_addr_i	in	64	Address from Wishbone master
wb_s_dat_i	in	64	Write data from Wishbone master
wb_s_dat_o	out	64	Read data to Wishbone master
wb_s_we_i	in	1	Write enable from Wishbone master
wb_s_sel_i	in	8	Byte select from Wishbone master
wb_s_stb_i	in	1	Strobe from Wishbone master
wb_s_ack_o	out	1	Acknowledge to Wishbone master
wb_s_cyc_i	in	1	Cycle from Wishbone master

Bus Interface

Wishbone Classic Slave.

Registers

The as_slave_bpi module contains no registers of its own. All logic is purely combinatorial. Registers are located in the peripheral kernel that is connected via the kernel-side interface of this module.

Related Sections of this Specification

- Wishbone Master BPI Section 4.3.7
- QSPI Peripheral Section 6.3, which uses this BPI
- GPIO Peripheral Section 6.1, which uses this BPI

Functional Overview

The slave BPI converts two Wishbone control signals plus the byte-select into two conditions used throughout the peripheral:

$$\textit{valid\_transfer} = \texttt{CYC} \wedge \texttt{STB}$$

$$\textit{all\_sel} = \& \ \texttt{SEL}_{7:0}$$

From these two conditions the write and read enables to the peripheral kernel are derived:

$$\texttt{wr\_o} = \textit{valid\_transfer} \wedge \texttt{WE} \wedge \textit{all\_sel}$$

$$\texttt{rd\_o} = \textit{valid\_transfer} \wedge \neg\texttt{WE} \wedge \textit{all\_sel}$$

The Wishbone acknowledge is generated combinatorially:

$$\texttt{ACK\_O} = \textit{valid\_transfer} \wedge \textit{all\_sel}$$

Because ACK is driven combinatorially from STB and CYC, the slave introduces zero wait states. The peripheral kernel must therefore provide read data (`dat_from_core_i`) in the same clock cycle that `rd_o` is asserted.

Structural Overview

Figure 4.4 shows the internal signal flow of the slave BPI.

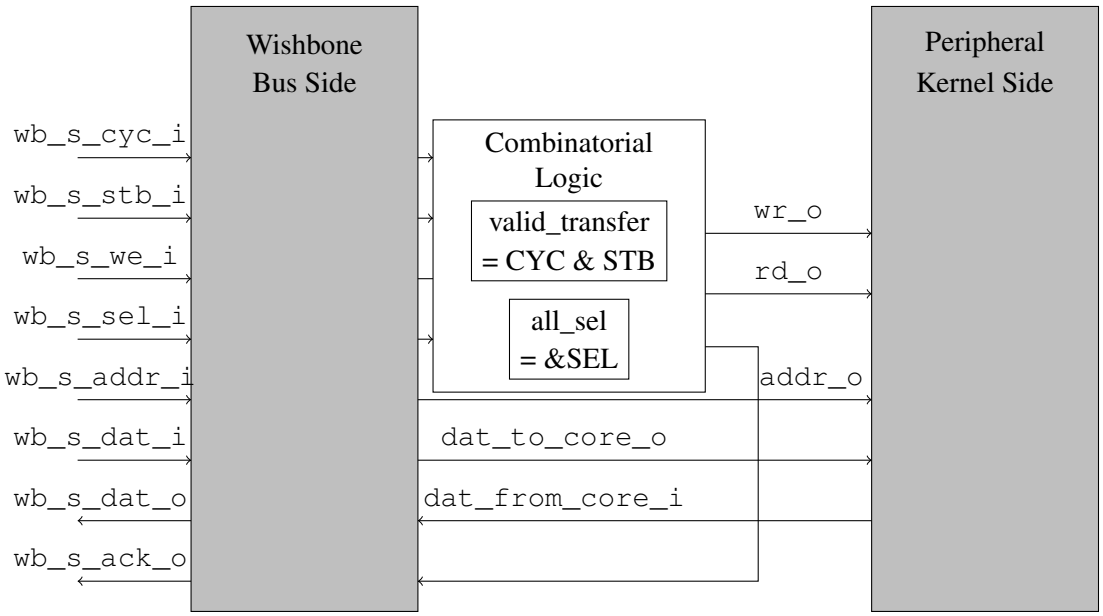

Figure 4.4: as_slave_bpi – Internal Signal Flow

Table 4.15 lists the internal signals.

Table 4.15: as_slave_bpi Internal Signals

Signal	Type	Width	Description
valid_transfer_s	logic	1	High when both CYC and STB are asserted. Indicates an active Wishbone bus cycle.
all_sel_s	logic	1	High when all byte-select bits are set. Only full 64-bit word accesses are accepted.

Instantiation

The module is parameterised and instantiated as follows:

```
as_slave_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_qspi_bpi (
    .rst_i              ( rst_i              ),
    .clk_i              ( clk_i              ),
    // kernel side
```

```
    .addr_o             ( addr_s             ),
    .dat_from_core_i    ( dat_from_core_s    ),
    .dat_to_core_o      ( dat_to_core_s      ),
    .wr_o               ( wr_s               ),
    .rd_o               ( rd_s               ),
    // wishbone side
    .wb_s_addr_i        ( wb_s_addr_i        ),
    .wb_s_dat_i         ( wb_s_dat_i         ),
    .wb_s_dat_o         ( wb_s_dat_o         ),
    .wb_s_we_i          ( wb_s_we_i          ),
    .wb_s_sel_i         ( wb_s_sel_i         ),
    .wb_s_stb_i         ( wb_s_stb_i         ),
    .wb_s_ack_o         ( wb_s_ack_o         ),
    .wb_s_cyc_i         ( wb_s_cyc_i         )
);
```

Limitations and Design Rules

- **Zero wait states only.** The peripheral kernel must provide read data combinatorially. If the kernel uses registered outputs, a wait-state mechanism (delayed ACK) must be added externally.

- **Full-word accesses only.** Sub-word byte or half-word accesses are not supported. The `SEL` check ensures that partial accesses are rejected silently.

- **No ERR_O.** Bus errors are not signalled. Invalid accesses are simply not acknowledged (ACK stays low).

- **No burst support.** The module supports only Wishbone Classic single-cycle transfers. Pipeline or burst modes are not implemented.

- **Combinatorial ACK.** Connecting multiple slave BPIs in a shared-bus topology requires that ACK is only driven by the selected slave. Address decoding and ACK gating must be handled by the bus interconnect logic, not by this module.

4.3.7 Wishbone Master BPI (`as_master_bpi`)

History

Table 4.16: History of as_master_bpi

Version	Date	Author	Change
0.1	2026-02-19	Claude-AI	Initial version, Wishbone B4 Classic, continuous CYC/STB

Functional Configuration of as_master_bpi

The `as_master_bpi` module implements the Wishbone B4 Classic master interface. It is a simple combinatorial wrapper that translates address, data and direction signals from the CPU or DMA core into Wishbone master signals. Table 4.17 lists the configurable parameters.

Table 4.17: Functional Configuration of as_master_bpi

Parameter	Type	Default	Description
`addr_width`	integer	64	Width of the address bus in bits
`data_width`	integer	64	Width of the data bus in bits; also determines SEL width as `data_width/8`

HW Interfaces

Table 4.18 lists all input and output ports of the `as_master_bpi` module.

Table 4.18: I/O of as_master_bpi

Port	Direction	Width	Description
rst_i	in	1	Synchronous reset, active high
clk_i	in	1	Bus clock
CPU / master core side:			
addr_i	in	64	Address from CPU core
dat_from_core_i	in	64	Write data from CPU core
dat_to_core_o	out	64	Read data returned to CPU core
wr_i	in	1	Write enable from CPU core: 1 = write, 0 = read
Wishbone bus side:			
wb_m_addr_o	out	64	Address to Wishbone bus
wb_m_dat_i	in	64	Read data from Wishbone bus (from selected slave)
wb_m_dat_o	out	64	Write data to Wishbone bus
wb_m_we_o	out	1	Write enable to Wishbone bus
wb_m_sel_o	out	8	Byte select to Wishbone bus (always all-ones)
wb_m_stb_o	out	1	Strobe to Wishbone bus (tied high)
wb_m_ack_i	in	1	Acknowledge from Wishbone bus (from selected slave)
wb_m_cyc_o	out	1	Cycle to Wishbone bus (tied high)

Bus Interface

Wishbone Classic Master.

Registers

The as_master_bpi module contains no registers of its own. All logic is purely combinatorial.

Related Sections of this Specification

- Wishbone Slave BPI Section 4.3.6
- RWU-RV64I Core Chapter, Bus Interface subsection

Functional Overview

The master BPI is a thin combinatorial bridge. All Wishbone master outputs are driven directly from the CPU core signals without any registered state:

$$
\begin{aligned}
\texttt{wb\_m\_addr\_o} &= \texttt{addr\_i} \\
\texttt{wb\_m\_dat\_o} &= \texttt{dat\_from\_core\_i} \\
\texttt{wb\_m\_we\_o} &= \texttt{wr\_i} \\
\texttt{wb\_m\_sel\_o} &= \texttt{8'hFF} \quad \text{(all bytes always valid)} \\
\texttt{wb\_m\_stb\_o} &= 1 \\
\texttt{wb\_m\_cyc\_o} &= 1 \\
\texttt{dat\_to\_core\_o} &= \texttt{wb\_m\_dat\_i}
\end{aligned}
$$

Because `STB` and `CYC` are continuously asserted, a new Wishbone transfer is initiated on every clock cycle as long as a slave is present on the bus. The transfer completes when the slave returns `ACK`. In the zero-wait-state case (slave BPI also combinatorial), ACK arrives in the same cycle as STB, so every clock cycle is a completed transfer.

The CPU core must handle the `ack_i` signal to determine when a read result is valid or when the next write can proceed.

Structural Overview

Figure 4.5 shows the internal signal flow of the master BPI.

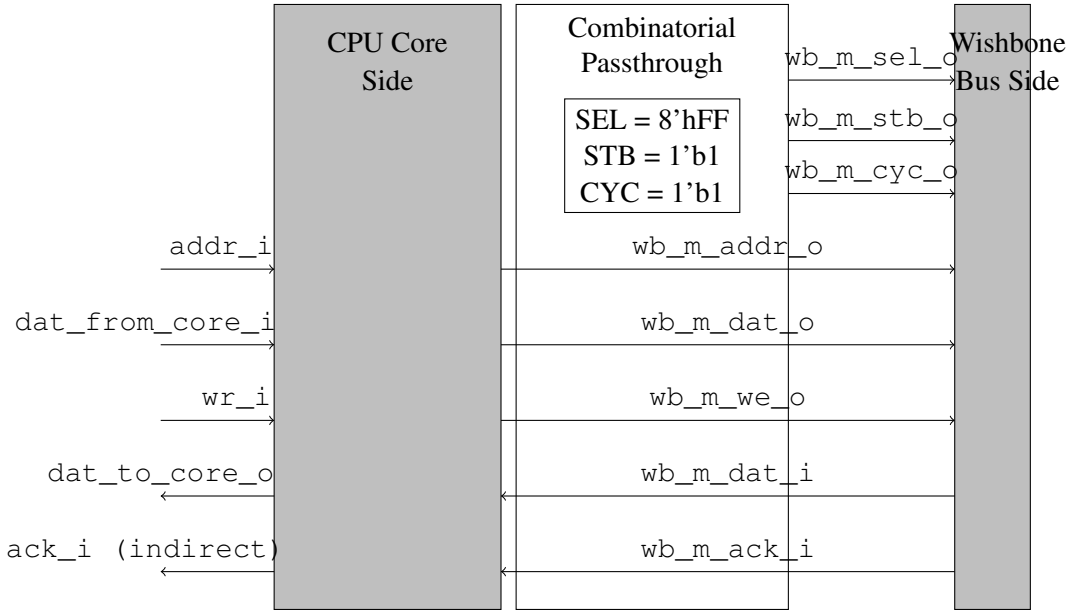

Figure 4.5: as_master_bpi – Internal Signal Flow

Table 4.19 lists the internal signals (none in this implementation, as the module is purely combinatorial):

Table 4.19: as_master_bpi Internal Signals

Signal	Type	Width	Description
(none)	–	–	All logic is combinatorial; no internal signals

Instantiation

```
as_master_bpi #(
    .addr_width ( 64 ),
    .data_width ( 64 )
) u_cpu_bpi (
    .rst_i              ( rst_i              ),
    .clk_i              ( clk_i              ),
    // CPU core side
    .addr_i             ( cpu_addr_s         ),
    .dat_from_core_i    ( cpu_wdat_s         ),
    .dat_to_core_o      ( cpu_rdat_s         ),
    .wr_i               ( cpu_wr_s           ),
    // wishbone side
```

```
    .wb_m_addr_o        ( wb_m_addr_s       ),
    .wb_m_dat_i         ( wb_m_dat_i_s      ),
    .wb_m_dat_o         ( wb_m_dat_o_s      ),
    .wb_m_we_o          ( wb_m_we_s         ),
    .wb_m_sel_o         ( wb_m_sel_s        ),
    .wb_m_stb_o         ( wb_m_stb_s        ),
    .wb_m_ack_i         ( wb_m_ack_s        ),
    .wb_m_cyc_o         ( wb_m_cyc_s        )
);
```

Limitations and Design Rules

- **Continuous bus occupation.** Because `STB` and `CYC` are permanently asserted, the master occupies the bus at all times. Multi-master arbitration requires that these signals are gated by the arbiter before being placed on the shared bus.

- **No idle state.** The master BPI does not support idle cycles between transfers. The surrounding CPU core logic must insert idle cycles if needed by controlling `addr_i` and `wr_i` appropriately and ignoring any ACK responses during idle periods.

- **ACK not evaluated internally.** The `wb_m_ack_i` signal is passed through directly to the CPU core via the bus-side port; the BPI itself does not act on it. The CPU must sample ACK and stall its pipeline if required.

- **No error handling.** There is no support for Wishbone `ERR_I` or `RTY_I`.

4.3.8 Bus Interconnect

Two Independent Buses

The Wishbone and AXI4 buses are completely separate. There is no bridge, no shared wires, and no arbitration between them. Traffic routing is determined at design time by which port of each block is connected to which bus:

- All CPU accesses to peripheral registers use the Wishbone bus.

- All cache-line fills use the AXI4 bus.

- The QSPI controller has one port on each bus; they operate independently.

Wishbone Address Map

Each Wishbone slave is mapped to a contiguous, non-overlapping region of the address space. Address decoding is performed by the bus interconnect logic, which forwards `STB` and `CYC` only to the addressed slave. Slaves whose address range is not selected must drive `ACK_O` low and `DAT_O` to zero. The actual address boundaries are defined in the chip-level memory map (Chapter "Memory Map and Registers").

Wishbone Topology

The current Wishbone implementation uses a **shared bus** topology with a single master (the CPU core). A crossbar switch is not implemented because there is currently only one Wishbone master and the zero-wait-state slave design means bus bandwidth is limited only by the clock frequency. If a DMA controller is added, its data buffers should be placed in the Scratchpad SRAM (which is not a Wishbone slave), not on the Wishbone bus, so the Wishbone topology does not need to change.

AXI4 Topology

The AXI4 bus uses a simple **Memory Arbiter** (two masters, one slave): I-Cache and D-Cache are both AXI4 masters; the QSPI controller data port is the single AXI4 slave. The arbiter implements fixed priority (D-Cache $>$ I-Cache) and never interrupts an in-progress burst. Details are documented in the Memory Concept section.

Peripheral Register Offset Alignment

All peripheral registers are 64 bit wide and aligned to 8-byte boundaries. The address offset within a peripheral starts at `0x00` for the ID register and increases in steps of `0x08`. This is consistent with the Wishbone bus data width and the 64-bit alignment of the RISC-V load and store instructions used to access the peripheral registers.

Read-Modify-Write

The Wishbone protocol as implemented does not support atomic read-modify-write operations. Software must therefore use disable-interrupt / critical-section patterns when accessing shared registers from multiple contexts.

4.3.9 Comparison: Wishbone vs. AXI4

Table 4.20 summarises the key differences between the two on-chip buses and the rationale for using each.

Table 4.20: Wishbone B4 Classic vs. AXI4 – Key Differences

Property	Wishbone B4 Classic	AXI4 (cache subset)
Transfer unit	1 word (64 bit)	4 beats × 64 bit = 32 B
Channels	Shared address/data	Decoupled AR and R
Handshake	STB + ACK	VALID + READY per channel
Outstanding tx	0 (blocking)	1 per master
Burst	Not supported	INCR, 4 beats fixed
Write support	Full	None (write channels removed)
Error signalling	None	RRESP; maps to RISC-V exceptions
Latency model	0 wait states typical	Variable; QSPI controller stalls until Flash data arrives
Complexity	Minimal	Moderate (5 channels, ID tracking, burst counter)
Used for	Peripheral registers	Cache-line fills from NOR Flash

4.4 Interrupt Concept

Table 4.21: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.5 Clock Concept

Table 4.22: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.6 System Control Concept

Table 4.23: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.7 Power Concept

Table 4.24: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.8 Memory Concept

Table 4.25: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.8.1 Overview

The RWU-RV64I chip uses a Harvard-style split-cache architecture. A dedicated **Instruction Cache (I-Cache)** serves the CPU fetch path, and a dedicated **Data Cache (D-Cache)** serves the load/store path. Both caches are 4-way set-associative with a fixed cache-line size of 32 bytes.

The backing store for both caches is an off-chip **NOR Flash** memory accessed through the on-chip QSPI controller. NOR Flash is read-only from the cache's perspective; accordingly the D-Cache operates as a **read-allocate, no-write-back** cache for the Flash address region. Cache data is held in two on-chip **SRAM macros** (X-Fab process), one per cache. A **Memory Arbiter** serialises simultaneous miss requests from the two cache controllers onto the single AXI4 bus that leads to the QSPI controller.

Read-write data (stack, heap, `.data`, `.bss`) resides in a dedicated on-chip **Scratchpad SRAM**. The D-Cache controller decodes the physical address of every load/store: if the address falls in the Flash region the request is served through the cache; if it falls in the Scratchpad region the cache is bypassed and the Scratchpad SRAM is accessed directly via a simple synchronous interface. Because the scratchpad is uncacheable, dirty evictions from the D-Cache can never occur; the dirty bit and the write-back FSM state are therefore eliminated.

The on-chip bus between cache controllers and the Memory Arbiter uses a subset of the **AMBA AXI4** protocol with burst support, replacing the Wishbone B4 Classic interface used elsewhere in the design. This allows a full 32-byte cache-line fill to complete in a single four-beat burst transaction instead of eight independent single transfers.

Table 4.26: Memory Hierarchy Summary

Level	Block	Capacity	Technology	Latency (typical)
L1-I	I-Cache	4 KB (param.)	On-chip SRAM macro	1 cycle (hit)
L1-D	D-Cache	4 KB (param.)	On-chip SRAM macro	1 cycle (hit)
–	Scratchpad	TBD (param.)	On-chip SRAM macro	1–2 cycles (direct)
Main	NOR Flash	device-specific	Off-chip via QSPI	$\gg$10 cycles (miss)

4.8.2 System Memory Architecture

Figure 4.6 shows the system-level block diagram of the memory subsystem.

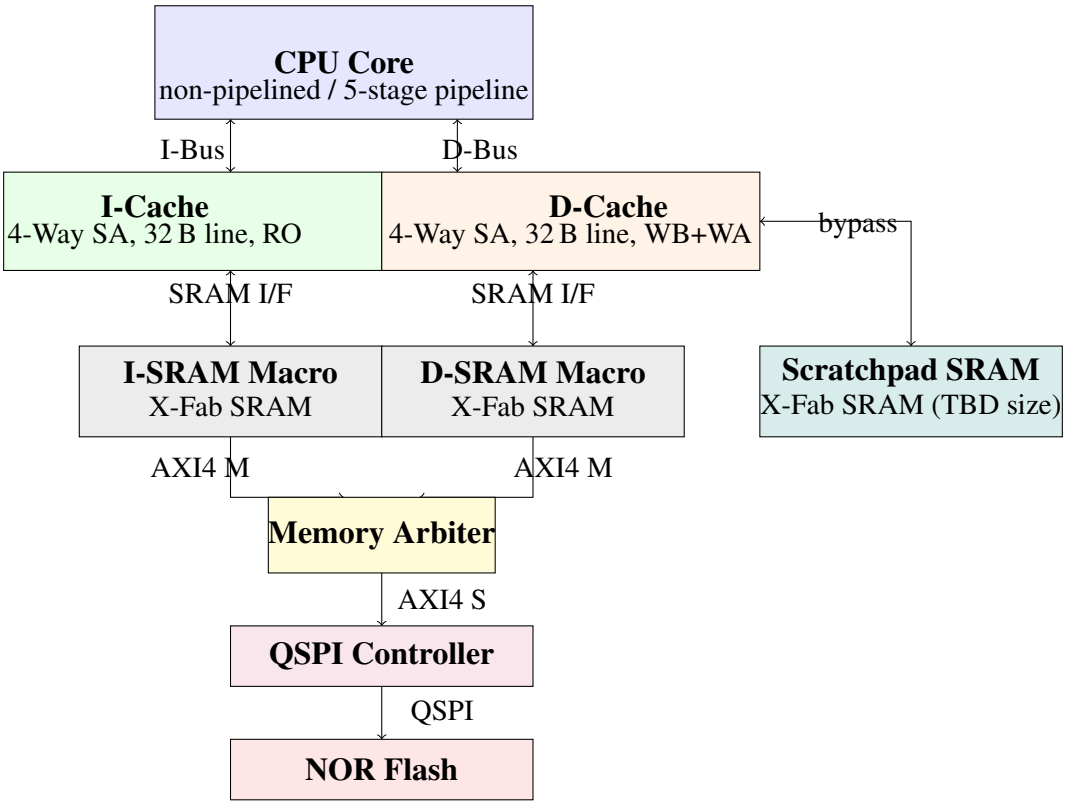

Figure 4.6: System Memory Architecture Block Diagram

4.8.3 Cache Configuration Parameters

Both caches share the same structural parameters, listed in Table 4.27. Parameters marked fixed are part of the cache-line protocol and cannot be changed without redesigning the AXI4 burst transactions and the SRAM macros.

Table 4.27: Cache Configuration Parameters

Parameter	Symbol	Default	Fixed?	Description
Cache size	`CACHE_SIZE_B`	4096	no	Total capacity in bytes per cache
Associativity	`WAYS`	4	yes	Number of ways per set
Cache-line size	`LINE_BYTES`	32	yes	Bytes per cache line
Number of sets	`SETS`	32	no	Derived: `CACHE_SIZE_B / (WAYS * LINE_BYTES)`
Physical addr width	`PA_WIDTH`	32	no	Physical address bits
AXI4 data width	`AXI_DW`	64	yes	AXI4 memory bus data width

4.8.4 Physical Address Decomposition

With the default parameters (4 KB cache, 4-way, 32 B line, 32-bit physical address), a physical address is divided as shown in Figure 4.7 and Table 4.28.

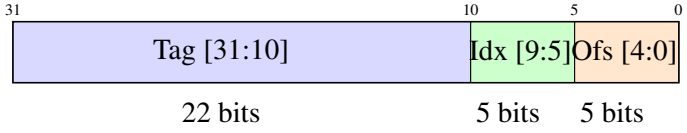

Figure 4.7: Physical Address Decomposition (32-bit PA, 4 KB cache, 32 B line)

Table 4.28: Physical Address Fields

Field	Bits	Width	Description
`tag`	[31:10]	22	Compared against stored tag to determine hit/miss
`index`	[9:5]	5	Selects one of 32 sets
`offset`	[4:0]	5	Byte offset within the 32-byte cache line
`word`	[4:3]	2	Selects one of four 64-bit words within the line
`instr`	[4:2]	3	Selects one of eight 32-bit instructions (I-Cache only)
`byte`	[2:0]	3	Byte offset within a 64-bit word (D-Cache sub-word)

The general formula for derived address parameters is:

$$\text{OFFSET\_BITS} = \log_2(\texttt{LINE\_BYTES}) = 5$$

$$\text{INDEX\_BITS} = \log_2(\texttt{SETS}) = \log_2\left(\frac{\texttt{CACHE\_SIZE\_B}}{\texttt{WAYS} \times \texttt{LINE\_BYTES}}\right) = 5$$

$$\text{TAG\_BITS} = \texttt{PA\_WIDTH} - \text{INDEX\_BITS} - \text{OFFSET\_BITS} = 32 - 5 - 5 = 22$$

4.8.5 Cache Tag and Metadata Arrays

Each set contains per-way metadata (valid, tag, and for D-Cache: dirty bit) plus shared set-level pseudo-LRU bits.

I-Cache Metadata Layout

The I-Cache is read-only; no dirty bit is required.

Table 4.29: I-Cache – Metadata Bits per Set

Field	Bits	Width	Description
Per way (repeated 4×):			
`valid[w]`	1 per way	1	Line present in cache; cleared on reset or flush
`tag[w]`	22 per way	22	Upper address bits stored at fill time
Per set (shared):			
`plru[2:0]`	3 per set	3	Pseudo-LRU replacement state (see Section 4.8.5)
Total per set:			
(sum)	–	95	$4 \times (1 + 22) + 3 = 95$ bits; rounded to 96 for SRAM alignment

D-Cache Metadata Layout

The D-Cache is read-allocate with no write-back; no dirty bit is required (identical structure to the I-Cache metadata).

Table 4.30: D-Cache – Metadata Bits per Set

Field	Bits	Width	Description
Per way (repeated 4×):			
`valid[w]`	1 per way	1	Line present in cache; cleared on reset or flush
`tag[w]`	22 per way	22	Tag stored at fill time
Per set (shared):			
`plru[2:0]`	3 per set	3	Pseudo-LRU replacement state (see Section 4.8.5)
Total per set:			
(sum)	–	95	$4 \times (1 + 22) + 3 = 95$ bits; rounded to 96 for SRAM alignment

Pseudo-LRU Replacement Policy

A 3-bit pseudo-LRU (PLRU) binary tree approximates LRU replacement for 4 ways without the overhead of true LRU. The tree is shown in Figure 4.8 and the bit semantics are defined in Table 4.31.

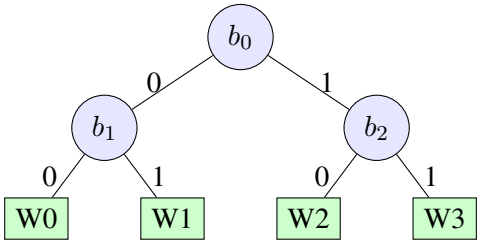

Figure 4.8: Pseudo-LRU Binary Tree for 4 Ways

Table 4.31: PLRU Bit Semantics

Bit	Meaning (bit=0)	Meaning (bit=1)
b_0	Left pair (W0, W1) is LRU; victim from left half	Right pair (W2, W3) is LRU; victim from right half
b_1	W1 is LRU within left pair; evict W1	W0 is LRU within left pair; evict W0
b_2	W3 is LRU within right pair; evict W3	W2 is LRU within right pair; evict W2

Victim selection and PLRU update rules are defined in Table 4.32.

Table 4.32: PLRU Victim Selection and Update on Cache Access

Accessed Way	Victim (for miss/eviction)	PLRU bits updated
W0	b_0=0: (b_1=0 $\Rightarrow$ W1), (b_1=1 $\Rightarrow$ W0)	$b_0 \leftarrow 1,\ b_1 \leftarrow 1$
W1	same left half	$b_0 \leftarrow 1,\ b_1 \leftarrow 0$
W2	b_0=1: (b_2=0 $\Rightarrow$ W3), (b_2=1 $\Rightarrow$ W2)	$b_0 \leftarrow 0,\ b_2 \leftarrow 1$
W3	same right half	$b_0 \leftarrow 0,\ b_2 \leftarrow 0$

The PLRU state is stored in a small register array (32 sets $\times$ 3 bits = 96 bits). It is reset to all-zero on power-on reset.

4.8.6 SRAM Macro Organisation

The memory subsystem uses five X-Fab XO035 SPRAM macros in total (see Section 4.8.7 for the selected instances). All macros operate as single-port synchronous SRAMs at 3.3 V.

Table 4.33: SRAM Macro Logical Requirements

Macro	Width	Depth	Size	Notes
Tag SRAM (I)	96 bit	32	384 B	valid + tag for 4 ways + 3 PLRU bits; rounded from 95
Tag SRAM (D)	96 bit	32	384 B	same structure as I-Cache (no dirty bit); rounded from 95
Data SRAM (I)	256 bit	128	4 KB	32 sets × 4 ways × 32 B; addressed as `{index,way}`
Data SRAM (D)	256 bit	128	4 KB	same organisation as I-Cache data SRAM
Scratchpad	64 bit	`SP_DEPTH`	`SP_SIZE_B`	Default: `SP_DEPTH=1024` ⇒ 8 KB; addressed by `dc_addr_i[12:3]`

The data SRAM is addressed by the concatenation `{index[4:0], way[1:0]}` giving a 7-bit address (128 entries). All 256 bits of one cache line are read or written in a single SRAM cycle.

During a cache-line fill, the cache controller writes the four 64-bit AXI4 beat words into the data SRAM on four consecutive cycles using a word-level write enable, while the full line address (`index` ‖ `fill_way`) is held constant.

The Scratchpad SRAM is accessed with a 64-bit data bus and an 8-bit byte enable (`dc_wstrb_i`), matching the D-Cache data bus width. Its address is derived directly from `dc_addr_i[12:3]` (for the default 8 KB instance).

4.8.7 SRAM Macro Selection – XO035 SPRAM

Library and Process

All SRAM macros are generated with the X-Fab **XO035** (350 nm Modular Sensor Technology Platform) **SPRAM** (Single-Port RAM) compiler at the nominal supply of 3.3 V (2.2 V low-voltage option not used). Key library properties relevant to this design:

- Zero static DC power consumption (only intrinsic leakage).

- Separate input and output data buses (no bus contention during read-modify-write).
- Byte-level write enable supported (required for sub-word stores to the Scratchpad SRAM).
- Wide range of instance sizes; the compiler generates macros for the exact width×depth parameters listed in Table 4.34.
- Optional BIST block: **not instantiated** in the first tapeout; test coverage via scan chain is sufficient.

Selected Instances

Table 4.34 lists the five SPRAM instances to be generated from the XO035 compiler. Macro names follow the compiler naming convention and are to be confirmed against the current catalogue release.

Table 4.34: XO035 SPRAM Instances

Instance	Width	Depth	Size	Macro name	Usage
I-Tag	96 bit	32	384 B	`SPRAM_96x32`	I-Cache tag + valid + PLRU
D-Tag	96 bit	32	384 B	`SPRAM_96x32`	D-Cache tag + valid + PLRU
I-Data	256 bit	128	4 KB	`SPRAM_256x128`	I-Cache data array
D-Data	256 bit	128	4 KB	`SPRAM_256x128`	D-Cache data array
SP	64 bit	1024	8 KB	`SPRAM_64x1024`	Scratchpad (stack-/heap/.data/.bss)

Notes on macro names: The names `SPRAM_{W}x{D}` are placeholders using the pattern ⟨width⟩x⟨depth⟩. The actual names generated by the XO035 SPRAM compiler must be verified against the compiler catalogue and substituted in the netlist and floorplan before tapeout. I-Tag and D-Tag use the same compiled macro; two instances are placed. I-Data and D-Data likewise share one compiled macro with two instances.

SRAM Port Mapping

Table 4.35 maps the logical cache-controller signals to the standard XO035 SPRAM port names.

Table 4.35: XO035 SPRAM Port Mapping

SPRAM port	Direction	Connected to
`CLK`	in	System clock `clk_i`
`CEN`	in	Chip enable, active-low; asserted when cache controller drives a valid address
`WEN`	in	Write enable, active-low; low for write, high for read
`BWEN`	in	Bit-write enable, active-low; all-zero = write all bits (tag/data SRAMs); `~dc_wstrb_i` expanded to bit-level (Scratchpad only)
`A`	in	Address bus; see Table 4.33 for width
`D`	in	Write data
`Q`	out	Read data; valid one cycle after `CEN` low on a read

Timing Parameters

Table 4.36 lists the timing parameters to be extracted from the XO035 SPRAM compiler output for the worst-case process corner (SS – slow NMOS, slow PMOS), maximum temperature (125 °C), and minimum supply voltage (3.0 V for 3.3 V nominal).

Table 4.36: XO035 SPRAM Timing Parameters (worst-case SS/125 °C/3.0 V)

Parameter	Symbol	Value	Description
Clock period (min.)	t_{cyc}	TBD ns	Minimum clock period for correct operation
Access time	t_{aa}	TBD ns	Address to `Q` valid; determines read latency
Setup time (data)	t_{ds}	TBD ns	`D` / `BWEN` setup before rising `CLK`
Hold time (data)	t_{dh}	TBD ns	`D` / `BWEN` hold after rising `CLK`
Setup time (addr.)	t_{as}	TBD ns	`A` / `CEN` / `WEN` setup before rising `CLK`
Hold time (addr.)	t_{ah}	TBD ns	`A` / `CEN` / `WEN` hold after rising `CLK`

The system clock frequency must satisfy:

$$f_{\text{clk}} \leq \frac{1}{t_{\text{cyc, worst}}}$$

All cache hit paths (tag compare + data SRAM read) must close timing within a single clock cycle at the selected frequency. Static Timing Analysis (STA) with the compiler-supplied Liberty (`.lib`) file is required before tapeout.

Action (MEM-02): Run the XO035 SPRAM compiler for all five instances listed in Table 4.34 and fill in the timing values in Table 4.36. Verify that the maximum target frequency is achievable at the SS/125 °C/3.0 V corner.

4.8.8 Instruction Cache (I-Cache)

Functional Description

The I-Cache is a read-only, 4-way set-associative cache that serves the CPU instruction fetch stage. It has no write path to the backing NOR Flash, and no dirty bit.

On every fetch request the cache controller:

1. Reads the tag SRAM at address `index`.
2. Compares the four stored tags with the request tag in parallel.
3. Checks the corresponding valid bits.
4. On a hit: reads the data SRAM at `{index, hit_way}`, selects the 32-bit instruction word using `offset[4:2]`, and returns it to the CPU in the same cycle.
5. On a miss: stalls the CPU, issues an AXI4 read burst to the Memory Arbiter, fills the selected SRAM line, updates the tag SRAM and PLRU state, then un-stalls the CPU.

Cache Hit Path

The hit path is combinatorial: tag comparison, data SRAM read, and instruction word selection complete within one clock cycle. The CPU pipeline sees a zero-cycle miss overhead when the cache hits.

Cache Miss Handling

On a miss the I-Cache controller:

- Uses the PLRU state to select the victim way.
- Asserts the `stall` signal to the CPU.
- Issues an AXI4 INCR burst of 4 beats (4 $\times$ 64 bit = 32 B) aligned to the cache-line boundary (`addr[63:5] & 5'b0`).
- Writes the four beats sequentially into the data SRAM.

- Updates the tag and valid bit for the filled way.
- Updates the PLRU bits.
- De-asserts `stall` on the cycle the last beat arrives.

I-Cache State Machine

Table 4.37: I-Cache Controller States

State	Description
`IDLE`	Waiting for a fetch request or flush signal from the CPU
`LOOKUP`	Tag comparison active; result available next cycle
`MISS`	AXI4 AR channel: sending read address burst to arbiter
`FILL`	Receiving 4 AXI4 R-channel beats; writing data SRAM
`RESPOND`	First word of filled line presented to CPU; stall de-asserted
`ERR`	AXI4 RRESP $\neq$ OKAY detected during FILL: valid bit not set; `ic_err_o` pulsed high for one cycle; return to IDLE
`FLUSH`	Invalidation: iterates set counter 0...31; writes `valid=0`, `plru=0` to tag SRAM each cycle; asserts `ic_flush_done_o` after 32 cycles

4.8.9 Data Cache (D-Cache)

Functional Description

The D-Cache is a **read-allocate** 4-way set-associative cache that serves the CPU load/store stage. Sub-word accesses (byte, halfword, word, doubleword) are supported.

The D-Cache controller decodes the physical address of every CPU load/store and dispatches to one of two paths:

- **Flash region (cacheable, read-only):** the request goes through the tag-lookup / fill pipeline described below. Stores to the Flash region are not supported; the cache controller asserts `dc_err_o` and raises a Load Access Fault (see Section 4.8.16).
- **Scratchpad region (uncacheable, read-write):** the cache is bypassed entirely. The controller drives the Scratchpad SRAM interface directly with the physical address, write data, byte enables, and read/write strobe, and returns the read data to the CPU after one SRAM cycle.

On a load from the Flash region the cache controller:

1. Reads the tag SRAM at address `index`.

2. Compares the four stored tags with the request tag in parallel.

3. Checks the corresponding valid bits.

4. On a read hit: reads the data SRAM, extracts the addressed word or sub-word, and returns it to the CPU in the same cycle.

5. On a miss: stalls the CPU, issues an AXI4 read burst to the Memory Arbiter, fills the selected SRAM line, updates tag and valid bit, updates the PLRU state, then un-stalls the CPU.

Address Region Policy and Absence of Write-Back

Because the Flash backing store is read-only and the Scratchpad is uncacheable, **the dirty bit is never set** and no cache line is ever marked for write-back. As a direct consequence:

- The D-Cache metadata does **not** contain a dirty bit (contrast with Table 4.30, which is superseded: the dirty field is removed; per-way metadata reduces from $(1 + 1 + 22) = 24$ to $(1 + 22) = 23$ bits).

- The `WRITEBACK` FSM state and the full AXI4 write path (AW, W, B channels) are **eliminated** from the D-Cache controller. The D-Cache requires only the AXI4 read channels (AR, R).

- Write-allocate is **not applicable**: stores go to the Scratchpad directly; there is no "write miss" into the cache.

This simplification resolves open item MEM-01 and reduces D-Cache controller complexity significantly.

D-Cache State Machine

Table 4.38: D-Cache Controller States

State	Description
IDLE	Waiting for a load/store request or flush signal from the CPU
DECODE	Physical address decoded; dispatch to CACHE path (Flash region) or BYPASS path (Scratchpad region)
LOOKUP	Flash region: tag comparison active; result available next cycle
FILL	Flash region: issuing AXI4 AR burst; receiving and writing 4 beats to data SRAM
RESPOND	Flash region: read data presented to CPU; stall de-asserted
BYPASS	Scratchpad region: driving Scratchpad SRAM interface directly; read data or write acknowledgement returned to CPU after 1 cycle
ERR	AXI4 RRESP $\neq$ OKAY detected during FILL: valid bit not set; dc_err_o pulsed high for one cycle; return to IDLE
FLUSH	Invalidation: iterates set counter 0...31; writes valid=0, plru=0 to tag SRAM each cycle; asserts dc_flush_done_o after 32 cycles

Note: The WRITEBACK and WRITE_HIT states present in earlier revisions of this document are removed. Neither state can be reached: no dirty lines exist (Flash is read-only) and stores bypass the cache via the BYPASS state.

4.8.10 CPU-to-Cache Interface

The interface between the CPU core and each cache uses a simple valid/ready + stall handshake. It is not based on Wishbone or AXI4: those protocols are reserved for the off-chip memory path.

Instruction Bus (CPU – I-Cache)

The Instruction Bus carries instruction fetch requests from the IF stage to the I-Cache.

Table 4.39: Instruction Bus Interface (as_icache_if)

Signal	Dir (CPU)	Width	Description
ic_addr_i	out	64	Instruction fetch address; must be 4-byte aligned
ic_req_i	out	1	Fetch request; held high while the CPU needs an instruction
ic_flush_i	out	1	Pulse high for one cycle to trigger full cache invalidation (FENCE.I)
ic_stall_o	in	1	High while a cache miss, flush, or error recovery is in progress
ic_flush_done_o	in	1	Pulsed high for one cycle when flush is complete
ic_err_o	in	1	Pulsed high for one cycle on AXI4 read error; CPU raises Instruction Access Fault (mcause=1, mtval=ic_addr_i)
ic_rdata_o	in	32	Instruction word returned by the I-Cache
ic_rvalid_o	in	1	ic_rdata_o is valid; asserted one cycle after a hit

```
interface as_icache_if (input logic clk_i,
                        input logic rst_i);
  logic [63:0] ic_addr;
  logic        ic_req;
  logic        ic_flush;
  logic        ic_stall;
  logic        ic_flush_done;
  logic        ic_err;
  logic [31:0] ic_rdata;
  logic        ic_rvalid;
  modport cpu  (output ic_addr, ic_req, ic_flush,
                input  ic_stall, ic_flush_done, ic_err, ic_rdata,
  modport cache(input  ic_addr, ic_req, ic_flush,
                output ic_stall, ic_flush_done, ic_err, ic_rdata,
endinterface
```

Data Bus (CPU – D-Cache)

The Data Bus carries load and store requests from the MEM stage to the D-Cache.

Table 4.40: Data Bus Interface (`as_dcache_if`)

Signal	Dir (CPU)	Width	Description
`dc_addr_i`	out	64	Data address
`dc_req_i`	out	1	Request valid (load or store)
`dc_wr_i`	out	1	1 = store, 0 = load
`dc_size_i`	out	3	Transfer size: 000=byte, 001=half, 010=word, 011=double
`dc_wdata_i`	out	64	Write data from CPU (store)
`dc_wstrb_i`	out	8	Byte enables for store; derived from `dc_size_i` and `dc_addr_i[2:0]`
`dc_flush_i`	out	1	Pulse high for one cycle to trigger full D-Cache invalidation (FENCE.I)
`dc_stall_o`	in	1	High while cache miss, scratchpad access, flush, or error recovery is in progress
`dc_flush_done_o`	in	1	Pulsed high for one cycle when flush is complete
`dc_err_o`	in	1	Pulsed high for one cycle on AXI4 read error; CPU raises Load Access Fault (`mcause`=5, `mtval=dc_addr_i`)
`dc_rdata_o`	in	64	Read data returned to CPU (load)
`dc_rvalid_o`	in	1	`dc_rdata_o` is valid

```
interface as_dcache_if (input logic clk_i,
                        input logic rst_i);
  logic [63:0] dc_addr;
  logic        dc_req;
  logic        dc_wr;
  logic [2:0]  dc_size;
  logic [63:0] dc_wdata;
  logic [7:0]  dc_wstrb;
```

```
  logic        dc_flush;
  logic        dc_stall;
  logic        dc_flush_done;
  logic        dc_err;
  logic [63:0] dc_rdata;
  logic        dc_rvalid;
  modport cpu  (output dc_addr, dc_req, dc_wr, dc_size,
                       dc_wdata, dc_wstrb, dc_flush,
                input  dc_stall, dc_flush_done, dc_err, dc_rdata,
  modport cache(input  dc_addr, dc_req, dc_wr, dc_size,
                       dc_wdata, dc_wstrb, dc_flush,
                output dc_stall, dc_flush_done, dc_err, dc_rdata,
endinterface
```

4.8.11 AXI4 Memory Bus (Cache Controllers to Memory Arbiter)

AXI4 Subset Used

Both cache controllers use the AMBA AXI4 protocol as masters on the memory bus. The Memory Arbiter presents a single AXI4 slave port toward the cache controllers and a single AXI4 master port toward the QSPI controller. Table 4.41 lists the AXI4 bus parameters.

Table 4.41: AXI4 Memory Bus Parameters

Parameter	Value	Comment
Protocol	AXI4 (not AXI4-Lite)	Burst support required for cache-line fills
Data width	64 bit	`AXI_DW=64`; 4 beats fill a 32 B cache line
Address width	32 bit	Matches physical address space
ID width	4 bit	Allows up to 16 outstanding transactions; I-Cache uses ID=0x1, D-Cache uses ID=0x2
Burst type	INCR (01)	Sequential address increment; no WRAP or FIXED
Burst length	ARLEN=3 / AWLEN=3	4 beats $\times$ 8 B = 32 B per cache line
Beat size	ARSIZE=3 (8 B)	One 64-bit word per beat
Outstanding	1 per master	Each cache has at most 1 transaction in flight
Protection	ARPROT=3'b000	Normal non-secure unprivileged
Cache hints	ARCACHE=4'b0010	Normal non-cacheable (device is the QSPI controller)
Write response	BRESP checked	Error on BRESP $\neq$ OKAY is flagged; TBD behaviour

AXI4 Channel Signals

Table 4.42 through Table 4.46 list the signals of each AXI4 channel. Signals marked † are tied to a fixed value; they do not require routing logic.

Table 4.42: AXI4 AR Channel (Read Address) – I/D Cache Master

Signal	Dir	Width	Description
`arid`	M→S	4	Transaction ID; I-Cache: 4'h1, D-Cache: 4'h2
`araddr`	M→S	32	Start address of the burst; must be aligned to 32 B (`addr[4:0]=0`)
`arlen`	M→S	8	Burst length minus one; always 8'h03 (4 beats)
`arsize`	M→S	3	Bytes per beat; always 3'b011 (8 bytes)
`arburst`	M→S	2	Burst type; always 2'b01 (INCR)
`arvalid`	M→S	1	Address and control information valid
`arready`	S→M	1	Slave ready to accept address
`arprot`†	M→S	3	Always 3'b000
`arcache`†	M→S	4	Always 4'b0010

Table 4.43: AXI4 R Channel (Read Data) – I/D Cache Master

Signal	Dir	Width	Description
`rid`	S→M	4	Echo of `arid`; cache controller checks this matches its pending ID
`rdata`	S→M	64	Read data beat; received in natural address order (beat 0 = lowest address)
`rresp`	S→M	2	2'b00 = OKAY; other values indicate an error
`rlast`	S→M	1	Asserted on the last beat of the burst (beat 3)
`rvalid`	S→M	1	Read data and strobes valid
`rready`	M→S	1	Cache controller ready to receive; held high once a miss is in progress

Table 4.44: AXI4 AW Channel (Write Address) – D-Cache Master only

Signal	Dir	Width	Description
`awid`	M→S	4	Always 4'h2 (D-Cache)
`awaddr`	M→S	32	Start address of dirty line write-back; 32-byte aligned
`awlen`	M→S	8	Always 8'h03 (4 beats)
`awsize`	M→S	3	Always 3'b011 (8 bytes/beat)
`awburst`	M→S	2	Always 2'b01 (INCR)
`awvalid`	M→S	1	Write address and control valid
`awready`	S→M	1	Slave ready to accept write address
`awprot`†	M→S	3	Always 3'b000
`awcache`†	M→S	4	Always 4'b0010

Table 4.45: AXI4 W Channel (Write Data) – D-Cache Master only

Signal	Dir	Width	Description
`wdata`	M→S	64	One 64-bit beat of the dirty cache line
`wstrb`	M→S	8	Always 8'hFF (full 64-bit write per beat)
`wlast`	M→S	1	Asserted on beat 3 (last beat)
`wvalid`	M→S	1	Write data valid
`wready`	S→M	1	Slave ready to accept write data

Table 4.46: AXI4 B Channel (Write Response) – D-Cache Master only

Signal	Dir	Width	Description
bid	S→M	4	Echo of awid
bresp	S→M	2	2'b00 = OKAY; error handling TBD
bvalid	S→M	1	Write response valid
bready	M→S	1	D-Cache ready to accept response

AXI4 SystemVerilog Interface Definition

```
interface as_axi4_if #(
  parameter int ADDR_W = 32,
  parameter int DATA_W = 64,
  parameter int ID_W   = 4,
  parameter int STRB_W = DATA_W / 8
)(
  input logic clk_i,
  input logic rst_i
);
  // AR channel
  logic [ID_W-1:0]   arid;
  logic [ADDR_W-1:0] araddr;
  logic [7:0]        arlen;
  logic [2:0]        arsize;
  logic [1:0]        arburst;
  logic              arvalid;
  logic aready;
  // R channel
  logic [ID_W-1:0]   rid;
  logic [DATA_W-1:0] rdata;
  logic [1:0]        rresp;
  logic              rlast;
  logic              rvalid;
  logic              rready;
  // AW channel
  logic [ID_W-1:0]   awid;
  logic [ADDR_W-1:0] awaddr;
  logic [7:0]        awlen;
  logic [2:0]        awsize;
  logic [1:0]        awburst;
  logic              awvalid;
```

```
  logic awready;
  // W channel
  logic [DATA_W-1:0] wdata;
  logic [STRB_W-1:0] wstrb;
  logic              wlast;
  logic              wvalid;
  logic wready;
  // B channel
  logic [ID_W-1:0]   bid;
  logic [1:0]        bresp;
  logic              bvalid;
  logic              bready;

  modport master (
    output arid, araddr, arlen, arsize, arburst, arvalid,
           rready, awid, awaddr, awlen, awsize, awburst,
           awvalid, wdata, wstrb, wlast, wvalid, bready,
    input  arready, rid, rdata, rresp, rlast, rvalid,
           awready, wready, bid, bresp, bvalid
  );
  modport slave (
    input  arid, araddr, arlen, arsize, arburst, arvalid, rready,
           awid, awaddr, awlen, awsize, awburst, awvalid,
           wdata, wstrb, wlast, wvalid, bready,
    output arready, rid, rdata, rresp, rlast, rvalid,
           awready, wready, bid, bresp, bvalid
  );
endinterface
```

Cache-Line Fill Transaction (Read Burst)

Figure 4.9 shows the timing of a 4-beat INCR read burst used for a cache-line fill. `ARLEN=3` encodes 4 beats (AXI4 convention: beats = ARLEN+1).

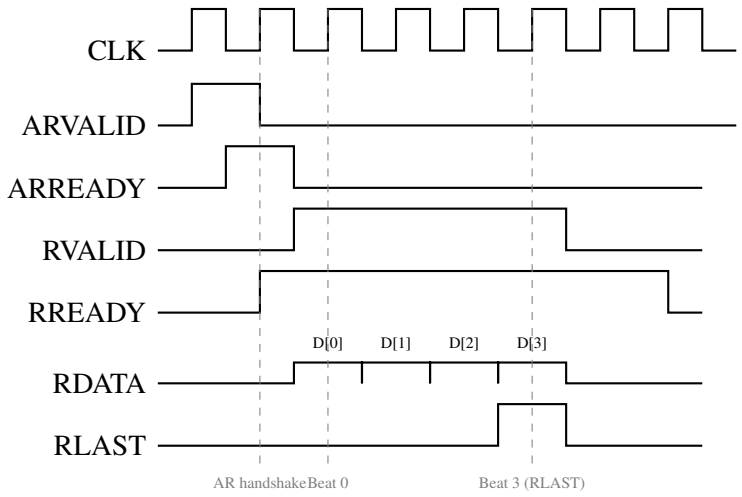

Figure 4.9: AXI4 Cache-Line Fill: 4-Beat INCR Read Burst (ARLEN=3)

Cache-Line Write-Back Transaction (Write Burst)

Figure 4.10 shows the timing of a 4-beat INCR write burst for a D-Cache dirty-line write-back. The AW and W channels are issued simultaneously.

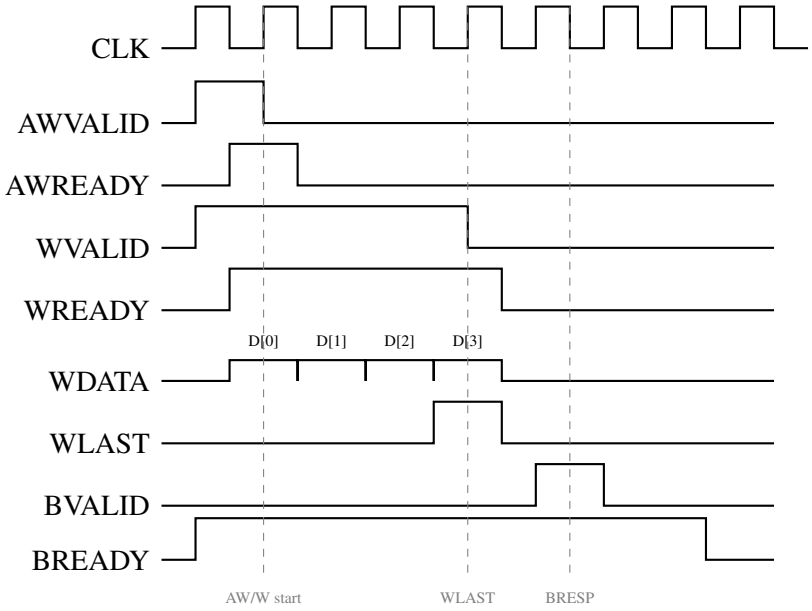

Figure 4.10: AXI4 Cache Write-Back: 4-Beat INCR Write Burst (AWLEN=3)

4.8.12 Memory Arbiter

Arbitration Policy

The Memory Arbiter serialises concurrent AXI4 requests from the I-Cache and D-Cache controllers onto the single AXI4 port that drives the QSPI controller. It implements a **fixed-priority** scheme with D-Cache having higher priority than I-Cache.

The rationale is that D-Cache misses in the MEM stage of the pipeline stall the entire pipeline, while I-Cache misses only stall the IF stage and can be partially hidden by a deeper instruction buffer in future pipeline implementations.

Table 4.47: Memory Arbiter Priority

Priority	Master	Notes
1 (highest)	D-Cache	Data miss / write-back; stalls the entire pipeline
2	I-Cache	Instruction miss; stalls IF only

An active transaction is never interrupted: the arbiter grants the bus to one master for the full burst (ARLEN+1 beats or AWLEN+1 beats) before switching to the other.

Note: A round-robin policy with equal priority or dynamic priority based on stall urgency is a future option.

Memory Arbiter Interface

Table 4.48: Memory Arbiter Ports

Port	Type	Description
`icache_axi4`	AXI4 slave	AXI4 master port of I-Cache controller
`dcache_axi4`	AXI4 slave	AXI4 master port of D-Cache controller
`qspi_axi4`	AXI4 master	AXI4 slave port of QSPI controller

The arbiter is purely combinatorial (pass-through mux) for AR, R, AW, W, and B channels; it adds no pipeline registers. ID bits from the selected master are forwarded without modification; the QSPI controller sees only one active ID at a time.

4.8.13 QSPI Controller Interface

The QSPI controller presents an AXI4 slave port to the Memory Arbiter. From the arbiter's perspective the QSPI controller is a slow memory: it may de-assert

ARREADY / WREADY for many cycles while the QSPI flash transaction completes.

The cache controllers handle this latency transparently because they hold RREADY asserted throughout the fill and wait for RLAST before returning data to the CPU.

Note: The QSPI controller itself is specified in a separate section of this document. Its AXI4 slave interface must be compatible with the parameter set in Table 4.41. In particular it must support INCR bursts of length 4 (ARLEN=3) at ARSIZE=3.

4.8.14 Cache Flush and Invalidation – FENCE.I

Overview

A cache flush invalidates all lines in a cache so that subsequent accesses reload data from the backing store. In the RWU-RV64I memory architecture, both I-Cache and D-Cache are **read-allocate with no write-back**; a flush therefore consists solely of clearing all valid bits and all PLRU bits in the tag SRAM. No dirty eviction is required (MEM-01 resolved).

Trigger Mechanism

The flush is triggered by the **FENCE.I** instruction (RISC-V Zifencei extension, encoding 0x0000100F). When the instruction decoder recognises FENCE.I it:

1. Stalls the CPU pipeline.
2. Pulses ic_flush_i and dc_flush_i high for one clock cycle.
3. Waits until both ic_flush_done_o and dc_flush_done_o are asserted.
4. Releases the stall and advances the PC to PC+4.

Both caches execute their FLUSH state machines in parallel; the total stall duration is therefore **32 clock cycles** (one write per set, 32 sets). In a non-pipelined CPU both caches are idle when FENCE.I is decoded, so no in-flight transaction needs to be drained.

No CSR is provided for cache control; FENCE.I is the sole software-visible mechanism. Scratchpad SRAM is not a cache and has no valid bits; it is unaffected by FENCE.I.

Flush State Machine

Both the I-Cache and D-Cache controllers implement an identical `FLUSH` state entered when `*_flush_i` is sampled high in `IDLE`:

Table 4.49: FLUSH State – Operation per Clock Cycle

Cycle	Action
0…30	Write `valid[3:0]=4'b0000, plru[2:0]=3'b000` to tag SRAM at address `flush_cnt[4:0]`; increment `flush_cnt`
31	Same write for set 31; assert `*_flush_done_o` for one cycle; return to `IDLE`

The 5-bit counter `flush_cnt` is implemented inside each cache controller and is reset to zero on entry to `FLUSH`. The `*_stall_o` signal remains asserted throughout the 32 cycles and is de-asserted together with `*_flush_done_o`.

Software Usage Guidelines

- **Self-modifying code:** not supported. All executable code must reside in NOR Flash; the scratchpad is data-only. FENCE.I is therefore not required for normal program flow.
- **DMA transfers:** if a future DMA engine writes to the Flash address region (e.g. via a shadow RAM or XIP staging area), the CPU must issue FENCE.I after the DMA completes and before executing any updated code. See MEM-04 for the coherency protocol.
- **Power-on reset:** both caches initialise with all valid bits cleared (reset value of tag SRAM flip-flops); no software flush is required at boot.
- **Flush latency:** 32 cycles. At a hypothetical 10 MHz clock (100 ns cycle) this is 3.2 µs – negligible for all expected use cases.

4.8.15 DMA Coherency Protocol

Overview

A DMA engine transfers data between memory regions without CPU intervention. Coherency problems arise when the CPU has cached a memory location and the DMA engine reads or writes that same location, causing one party to see stale data.

The RWU-RV64I cache architecture minimises DMA coherency complexity through two structural properties:

1. **Scratchpad is uncacheable.** All CPU stores bypass the D-Cache and go directly to the Scratchpad SRAM. A DMA engine accessing the Scratchpad always sees current data without any cache action.

2. **D-Cache is read-allocate with no dirty lines.** The D-Cache only caches Flash reads. Flash is read-only hardware; a DMA engine cannot write to it. Therefore the D-Cache can never hold data that is newer than the backing store.

Recommendation: place all DMA source and destination buffers in the Scratchpad address region. This eliminates every coherency concern at zero hardware cost.

DMA Access Classification

Table 4.50 enumerates all source/destination combinations and the required CPU action.

Table 4.50: DMA Coherency Requirements by Memory Region

DMA source	DMA destination	Coherency issue?	Required CPU action
Scratchpad	Peripheral	None	None. Scratchpad is uncacheable; DMA reads current data.
Peripheral	Scratchpad	None	None. CPU reads Scratchpad directly; no cache involved.
Scratchpad	Scratchpad	None	None. Both ends uncacheable.
Flash	Scratchpad	None	None. Flash data in D-Cache always matches Flash (read-only).
Flash	Peripheral	None	None. As above.
Scratchpad	Flash	N/A	Flash is read-only hardware; this transfer is not possible.
Flash	Flash	N/A	Flash is read-only hardware as destination; not possible.
Peripheral	Flash-mapped cached region†	**Yes**	CPU must issue `FENCE.I`

†Applicable only if a future memory map revision introduces a writable RAM region within the cacheable address space (e.g. an on-chip SRAM shadow of the Flash area). In the current architecture this case does not arise.

Software Protocol

For the one coherency-relevant scenario (DMA writes to a cacheable region):

1. CPU configures the DMA transfer (source, destination, length, direction) by writing to DMA control registers (peripheral region, uncacheable).
2. CPU starts the DMA engine and waits for the DMA-done event (interrupt or polling of a status register).
3. CPU issues `FENCE.I` to invalidate both I-Cache and D-Cache (32 cycles).
4. CPU may now fetch instructions from, or load data out of, the DMA-written region; it will see the updated content.

No hardware handshake between the DMA engine and the cache controllers is required. The software ordering (wait for DMA done, then FENCE.I) is sufficient because:

- The DMA engine writes directly to memory (not through any cache).
- FENCE.I invalidates the cache after the DMA write is visible in memory.
- The non-pipelined CPU executes FENCE.I atomically with respect to subsequent loads.

Future DMA Engine Interface Requirements

When the DMA engine is designed, it must expose at minimum:

Table 4.51: Minimum DMA Engine Signals for Cache Coherency

Signal	Type	Purpose
`dma_done_irq`	out (to CPU)	Interrupt asserted when DMA transfer completes; CPU uses this as the trigger to issue `FENCE.I` before accessing DMA-written data
`dma_active_o`	out (to system)	High while a transfer is in progress; optional, used to prevent conflicting CPU accesses to the same address range

No cache-flush or cache-snoop signal is required from the DMA engine; the software protocol (Section above) is sufficient.

4.8.16 AXI4 Error Handling

Overview

The AXI4 read response channel (`RRESP[1:0]`) carries a completion status from the QSPI controller back to the cache controllers. A non-OKAY response indicates that the underlying NOR Flash transaction failed (e.g. timeout, command error, or address out of Flash range).

In the RWU-RV64I design the AXI4 write channels (AW, W, B) are eliminated from the cache controllers (MEM-01); `BRESP` is therefore not observed and requires no handling.

AXI4 errors are surfaced to software as standard **RISC-V Machine-Mode Exceptions** following the Privileged Architecture specification. This is the correct approach: the CPU already has exception infrastructure (for illegal instructions, etc.) and the trap handler can log, retry, or halt in a well-defined way.

AXI4 Response Codes

Table 4.52: AXI4 RRESP Codes Handled by Cache Controllers

RRESP	Name	Used?	Action
2'b00	OKAY	Yes	Normal completion; data written to cache SRAM
2'b01	EXOKAY	No	Exclusive access OK; not used (no LR/SC in this design)
2'b10	SLVERR	Yes	Slave error (QSPI / Flash fault); triggers exception
2'b11	DECERR	Yes	Decode error (address unmapped); triggers exception

Exception Mapping

Table 4.53 maps AXI4 error events to RISC-V exception causes.

Table 4.53: AXI4 Error to RISC-V Exception Mapping

Cache	AXI4 signal	mcause	Exception name	mtval
I-Cache	RRESP $\neq$ OKAY	1	Instruction Access Fault	Faulting fetch address (ic_addr_i at time of miss)
D-Cache	RRESP $\neq$ OKAY	5	Load Access Fault	Faulting load address (dc_addr_i at time of miss)

Error Detection in the Cache Controllers

Both cache controllers monitor RRESP on every beat of the R channel during the FILL state. If any beat carries RRESP $\neq$ 2'b00:

1. The cache controller immediately stops writing to the data SRAM.
2. The valid bit for the partially filled line is **not** set (the line remains invalid).
3. The controller transitions to the new ERR state.
4. In ERR, the controller pulses the error output (ic_err_o or dc_err_o) for one cycle and asserts the corresponding stall signal for that cycle.
5. The controller returns to IDLE.

The CPU samples the error output on the same cycle as the stall de-assertion, latches the faulting address, and raises the appropriate exception.

Updated Cache Controller FSMs

The ERR state is added to both I-Cache and D-Cache FSMs.

Table 4.54: ERR State – I-Cache and D-Cache (identical behaviour)

Condition	Action
Entry condition	Any AXI4 R-channel beat with `RRESP` $\neq$ `2'b00` during `FILL`
In `ERR` (1 cycle)	Do not update tag SRAM valid bit. Assert `*_err_o` for one cycle. Assert `*_stall_o` for the same cycle.
Exit condition	Unconditional; return to `IDLE` after one cycle

Updated Cache Interface Signals

One error output signal is added to each cache interface.

Table 4.55: Error Signals Added to Cache Interfaces

Signal	Interface	Dir (CPU)	Width	Description
`ic_err_o`	`as_icache_if`	in	1	Pulsed high for one cycle on an I-Cache AXI4 read error; CPU raises Instruction Access Fault (`mcause`=1)
`dc_err_o`	`as_dcache_if`	in	1	Pulsed high for one cycle on a D-Cache AXI4 read error; CPU raises Load Access Fault (`mcause`=5)

CPU Exception Handling Sequence

When the CPU samples `ic_err_o` or `dc_err_o`:

1. Save the current PC to `mepc`.
2. Write the exception cause to `mcause` (1 for I-Cache, 5 for D-Cache).
3. Write the faulting address to `mtval` (last value driven on `ic_addr_i` or `dc_addr_i`).
4. Clear `mstatus.MIE` (disable interrupts).
5. Jump to `mtvec` (trap vector).

The trap handler at `mtvec` may:

- **Halt:** for a non-recoverable Flash fault this is the correct default action (e.g. output error code via UART/GPIO, then `wfi`).
- **Retry:** re-execute the faulting instruction (`mret` after optionally re-initialising the QSPI controller). Appropriate for transient QSPI timeouts.
- **Substitute:** fill a scratchpad buffer with alternative data, remap the faulting address (if an MPU is present), and `mret`. Advanced; not required for the first tapeout.

Updated SystemVerilog Interfaces

```
// as_icache_if additions:
logic ic_err;       // new: AXI4 read error → Instruction Access Fault
modport cpu  (..., input ic_err);
modport cache(..., output ic_err);

// as_dcache_if additions:
logic dc_err;       // new: AXI4 read error → Load Access Fault
modport cpu  (..., input dc_err);
modport cache(..., output dc_err);
```

Full updated interface definitions are in Sections 4.8.10 and 4.8.10.

4.8.17 QSPI Operating Mode – Cache-Miss-Driven Read

XIP Terminology Clarification

The term Execute-In-Place (XIP) is used with two distinct meanings in the literature:

- **True XIP (Continuous Read mode):** the QSPI controller keeps the Flash chip-select asserted and the device in a special Continuous Read state; subsequent reads require only the address (no repeated opcode), reducing per-transaction overhead. Requires the Flash to support a Performance Enhance or XIP mode bit.
- **Indirect XIP (cache-miss-driven):** the CPU executes code that resides in Flash, but each cache miss triggers a complete SPI command sequence (opcode + address + dummy + data). The chip-select is asserted only for the duration of each burst. No special Flash mode is required.

Decision: the RWU-RV64I uses **indirect XIP** (cache-miss-driven burst mode). True XIP mode is neither necessary nor advantageous for this architecture, as explained below.

Rationale

True XIP mode is not adopted for the following reasons:

1. **The I-Cache already provides the XIP benefit.** Once a cache line is loaded, all subsequent accesses to it cost 1 clock cycle. The amortised QSPI overhead per instruction is negligible at realistic hit rates (>90 %).
2. **Sequential access is not guaranteed.** A 4-way set-associative cache evicts and reloads lines at non-sequential addresses. True XIP's main advantage (no repeated opcode/address) applies only to sequential reads; for random-address cache refills its benefit is minimal.
3. **Simpler QSPI controller.** Indirect XIP requires no Flash-mode-bit management, no continuous-read entry/exit sequence, and no special handling for the transition back from XIP mode (required before write/erase operations in True XIP).
4. **Lower power between cache misses.** With CS# de-asserted between bursts the Flash enters standby mode automatically, reducing leakage current.

QSPI Command Sequence per Cache Miss

On each cache miss the QSPI controller executes the following sequence on the SPI bus:

1. Assert CS# (chip select, active-low).
2. Send read opcode on IO0 (single-bit SPI).
3. Send 24-bit (3-byte) Flash address on IO0; address is taken directly from `ARADDR[24:0]` (for Flash devices up to 16 MB) or `ARADDR[31:0]` for larger devices. The lower 5 bits are always zero (cache-line aligned burst).
4. Send dummy cycles (device-specific; typically 8 clock cycles for Fast Read, 6–8 for Quad Fast Read).
5. Receive 32 bytes of read data, output on IO0 (Fast Read) or IO0–IO3 simultaneously (Quad Fast Read). Data is returned to the AXI4 R channel as 4 beats of 64 bits, LSB-first within each beat.

6. De-assert CS#.

Table 4.56: Supported QSPI Read Commands

Command	Opcode	Mode	Throughput	Notes
Fast Read	`0x0B`	1-bit SPI	Low	Fallback; maximum QSPI clock limited by Flash spec
Quad Fast Read	`0x6B`	4-bit QSPI	High	**Recommended**; data on IO0–IO3 in parallel; requires QUAD enable bit set in Flash status register

The Quad Fast Read command (`0x6B`) is recommended because it reduces the data phase from 256 QSPI clock cycles (single-bit) to 64 QSPI clock cycles (four-bit), significantly lowering cache-miss latency.

Address Mapping

The AXI4 `ARADDR` is used directly as the Flash physical address:

$$\texttt{Flash\_Addr}[24:5] = \texttt{ARADDR}[24:5], \quad \texttt{Flash\_Addr}[4:0] = 5'b00000$$

The five least-significant address bits are always zero because the cache controller aligns all burst requests to the 32-byte cache-line boundary before issuing the AXI4 transaction.

Latency Estimate

Table 4.57 estimates the QSPI bus cycles and system clock cycles consumed by a single cache-line fill using Quad Fast Read. Actual values depend on the Flash device datasheet and the QSPI clock divider setting.

Table 4.57: Cache-Line Fill Latency Estimate (Quad Fast Read, `0x6B`)

Phase	QSPI clocks	Notes
CS# assert + opcode	8	8-bit opcode on IO0
Address	24	24-bit address on IO0
Dummy cycles	8	Device-specific; 8 typical for `0x6B`
Data (32 bytes)	64	256 bits ÷ 4 (quad) = 64 QSPI clocks
CS# de-assert	2	Minimum CS# high time (device-specific)
Total	**106**	Per cache miss

At an example QSPI clock of 25 MHz and system clock of 10 MHz:

$$t_{\text{miss}} = \frac{106}{25\,\text{MHz}} = 4.24\,\mu\text{s} \mathrel{\widehat{=}} 42 \text{ system clock cycles.}$$

This is consistent with the "$\gg$10 cycles" estimate in Table 4.26. The exact QSPI clock frequency and divider ratio are to be determined once the target NOR Flash device is selected.

NOR Flash Device Requirements

The selected NOR Flash must support:

- **Quad Fast Read (0x6B)** with up to 8 dummy cycles.
- **3-byte address mode** (24-bit, supports Flash up to 16 MB); or 4-byte mode if a larger device is required.
- **AXI4 burst compatibility:** the device must sustain continuous 4-bit output for 64 QSPI clocks without inserting wait states.
- **3.3 V supply** (matching XO035 I/O voltage).

4.8.18 Open Items and Refinements

Table 4.58: Open Items – Memory Concept v0.1

ID	Description	Owner
MEM-01	**[RESOLVED]** Scratchpad SRAM chosen as read-write region. D-Cache operates as read-allocate only for the Flash address region; scratchpad is uncacheable (bypass path in D-Cache controller). Dirty bit and `WRITEBACK` state eliminated.	–
MEM-02	**[IN PROGRESS]** XO035 SPRAM library selected; five instances defined (I-Tag 96×32, D-Tag 96×32, I-Data 256×128, D-Data 256×128, Scratchpad 64×1024). Macro names and timing parameters (Table 4.36) to be filled after running the SPRAM compiler. See Section 4.8.7.	TBD
MEM-03	**[RESOLVED]** FENCE.I (Zifencei, opcode `0x0000100F`) triggers full invalidation of both I-Cache and D-Cache in 32 cycles. No CSR provided. Scratchpad unaffected. No dirty eviction (MEM-01). See Section 4.8.14.	–
MEM-04	**[RESOLVED]** Scratchpad is uncacheable; DMA buffers placed there require no cache action. For DMA writes to cacheable regions (future): CPU waits for `dma_done_irq`, then issues `FENCE.I`. No hardware handshake between DMA and cache controllers required. See Section 4.8.15.	–
MEM-05	Physical address width: 32 bits chosen (`PA_WIDTH=32`). With scratchpad and Flash regions both well within a 4 GB space this is confirmed sufficient; finalise when memory map is defined.	AS: 32 bits – ok
MEM-06	**[RESOLVED]** RRESP $\neq$ OKAY mapped to RISC-V exceptions: I-Cache $\rightarrow$ Instruction Access Fault (`mcause`=1), D-Cache $\rightarrow$ Load Access Fault (`mcause`=5). ERR state added to both FSMs; `ic_err_o`/`dc_err_o` signals added to cache interfaces. BRESP not used (no AXI4 writes). See Section 4.8.16.	–
MEM-07	**[RESOLVED]** Cache-miss-driven burst (indirect XIP) chosen. True XIP (Continuous Read mode) not adopted: I-Cache provides the XIP benefit, random-address refills gain little from True XIP, and the QSPI controller stays simpler. Quad Fast Read (`0x6B`) recommended; 106 QSPI clocks per 32-byte fill ($\approx$42 system clocks at 25 MHz QSPI / 10 MHz system). See Section 4.8.17.	–

4.9 System Protection and Security Concept

Table 4.59: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.10 Debug Concept

Table 4.60: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

4.11 Register Concept

Table 4.61: History

History		
Target Spec.	Current version : 1.0, 2025-02-22 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-22		
	2025-02-22	First draft (AS)

Chapter 5

RWU-RV64I Core

5.1 ALU

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of ALU0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a ALU peripheral are described in its functional description in Section 5.1.2. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for ALU0 in Table 5.3.

The **RWU-RV64I** has only one ALU.

Table 5.3: ALU0 Feature Set

	Generic GPIO Feature Set	Details
no	Seperate Kernel Clock	not needed
no	FIFO Data Buffering	not needed

HW Interfaces

The following table 5.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.4: HW Interfaces of Module ALU0

	Module internal name	Chip level name
Supply		
Clock	-	-
Bus	-	-
Interrupt	-	-
DMA	-	-
External Signals	-	-
Others	data01_i(63:0) data02_i(63:0) aluSel_i(4:0) aluZero_o aluNega_o aluCarr_o aluOver_o aluResult_o(63:0)	srcA_s(63:0) srcB_s(63:0) aluSel_s(4:0) zero_s nega_s carry_s overflow_s dBusAddr_s(63:0)

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.1.1 History of ALU

Table 5.5: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.1	5.1 / 2025-02-13	First draft (AS)

5.1.2 Functional Overview

The ALU operates on 64 bit data and generates 64 bit data. The ALU indicates a zero result, a negative result, the carry of a result, and the overflow.

General Features:

- 64 bit integer addition
- 64 bit integer subtraction
- 64 bit AND
- 64 bit OR
- 64 bit XOR
- 64 bit set less than
- 64 bit set less than, unsigned
- 32 bit shift right arithmetic word
- 32 bit shift right logical word
- 32 bit shift left logical word
- 32 bit subtract word
- 32 bit add word

- 64 bit shift right arithmetic
- 64 bit shift right logical
- 64 bit shift left logical
- branch if equal
- branch if not equal
- branch if less than
- branch if greater or equal
- branch if less than, unsigned
- branch if greater or equal, unsigned

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.1.3 Structural Overview

I/O of the ALU

Table 5.6 shows the input and outputs of the ALU0. This are all primary inputs and outputs. There is no bi-directional, tristate or open drain pin. All pins are high-active except the pins with a _n_ in its name.

Table 5.6: ALU0 I/O

Pin	Dir.	Wd.	Explanation
data01_i	in	64	Source data input. Comes either from the register file or the program conter (PC). Swiched by signal “aluSrcA”. Source one for all ALU operators.
data02_i	in	64	Source data input. Comes either from the register file or the immediate generator. Swiched by signal “aluSrcB”. Source two for all ALU operators.
aluSel_i	in	5	Selects the ALU operator.
aluZero_o	out	1	Zero flag of the ALU. Not buffered.
aluNega_o	out	1	Negative flag of the ALU. Not buffered.
aluCarr_o	out	1	Carry flag of the ALU. Not buffered.
aluOver_o	out	1	Overflow flag of the ALU. Not buffered.
aluResult_o	out	64	The result of each ALU operation.

Internals of the ALU

The following list (Table) shows the internal signals of the ALU. The table shows the signal name, the width, the driver, the target and a short explanation.

- Signal: aluSel_i[0]
  - Width: 1 bit
  - Source: aluSel_i
  - Destination: aluResult_o, overf_s
  - Explanation (special signal):
    - if aluSel_i[0] = 0, it is an ADD instruction
    - ... the second operand must be taken as it is
    - ... the carry-in of the addition is 0 (aluSel_i[0])
    - if aluSel_i[0] = 1, it is a SUB instruction
    - ... the second operand must be the 2-complement
    - ... the carry-in of the addition is 1 which acts as the +1 (aluSel_i[0])
- Signal: data01_s
  - Width: 64 bit, signed
  - Source: data01_i
  - Destination: aluResult_o, aluZero_o

  - Explanation: It is the signed interpretation of data01_i. It is needed for some branches (BLT, BGE) and SRA.

    logic signed [reg_width-1:0] data01_s;

- Signal: data02_s
  - Width: 64 bit, signed
  - Source: data02_i
  - Destination: aluResult_o, aluZero_o
  - Explanation: It is the signed interpretation of data02_i. It is needed for some branches (BLT, BGE) and SRA.

    logic signed [reg_width-1:0] data02_s;

- Signal: sum_s
  - Width: 64 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluResult_o
  - Explanation: It is the sum of the two data inputs data01_i and data02_i. The second data input could be negative and so 2-complemented. cinvb_s is the inversion of data02_e when $aluSel\_i[0] = 1$, aluSel_i[0] also acts as the plus 1.

    assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: cinvb_s
  - Width: 64 bit
  - Source: data02_i, aluSel_i[0]
  - Destination: sum_s, carry_s
  - Explanation: cinvb_s is the inversion of data02_i when $aluSel\_i[0] = 1$. Together with aluSel_i[0] ($cinvb\_s + aluSel\_i[0]$) it is the 2-complement of data02_i.

    assign cinvb_s = aluSel_i[0] ? data02_i : data02_i;

- Signal: carry_s

  - Width: 1 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluCarr_o (carry flag)
  - Explanation: It is the carry of sum_s.

    assign {carry_s, sum_s} = data01_i + cinvb_s + aluSel_i[0];

- Signal: sumw_s
  - Width: 32 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluResult_o
  - Explanation: For add word, etc. It is the sum of the two data inputs data01_i(31:0) and data02_i(31:0. The second data input could be negative and so 2-complemented. cinvbw_s is the inversion of data02_e(31:0) when $aluSel\_i[0] = 1$, aluSel_i[0] also acts as the plus 1.

    assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: cinvbw_s
  - Width: 32 bit
  - Source: data02_e, aluSel_i[0]
  - Destination: sum_s, carry_s
  - Explanation: For add word, etc. cinvbw_s is the inversion of data02_i(31:0) when $aluSel\_i[0] = 1$. Together with aluSel_i[0] ($cinvbw\_s+aluSel\_i[0]$) it is the 2-complement of data02_i(31:0).

    assign cinvbw_s = aluSel_i[0] ? data02_i[31:0] : data02_i[31:0];

- Signal: carryw_s
  - Width: 1 bit
  - Source: data01_i, cinvb_s (data02_i), aluSel_i[0]
  - Destination: aluCarr_o (carry flag)
  - Explanation: For add word, etc. It is the carry of sumw_s.

    assign {carryw_s, sumw_s} = data01_i[31:0] + cinvbw_s + aluSel_i[0];

- Signal: sllw_s
  - Width: 32 bit
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SLLW (shift left logical word)
    Implementation:
    $sllw\_s = data01\_i(31:0) << data02\_i(4:0)$
    aluResult_o is sllw_s, sign extended.
- srlw_s
  - Width: 32 bit
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SRLW (shift right logical word)
    Implementation:
    $srlw\_s = data01\_i(31:0) >> data02\_i(4:0)$
    aluResult_o is srlw_s, sign extended.
- sraw_s
  - Width: 32 bit, signed
  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SRAW (shift right algorithmic word)
    Implementation:
    $sraw\_s = data01w\_s >>> data02\_i(4:0)$
    aluResult_o is sraw_s, sign extended.
- data01w_s
  - Width: 32 bit, signed
  - Source: data01_i
  - Destination: sraw_s
  - Explanation: 32 bit signed representation of data01_i.
- sltiu_s
  - Width: 1 bit

  - Source: data01_i, data02_i
  - Destination: aluResult_o
  - Explanation: SLT, SLTU (set less than; signed, unsigned)
    Implementation:
    if(data01_i < data02_i)
    sltiu_s = 1;
    else
    sltiu_s = 0;
    aluResult_o is sltiu_s, zero extended.

- overf_s
  - Width: 1 bit
  - Source: data01_i, data02_i, sum_s, aluSel_i[0]
  - Destination: aluOver_o
  - Explanation: Overflow occurs when the result-value affects the sign:
    - overflow when adding two positives yields a negative
    - or, adding two negatives gives a positive
    - or, subtract a negative from a positive and get a negative
    - or, subtract a positive from a negative and get a positive

- aluResult_o
  - Width: 64 bit
  - Source: all signals
  - Destination: output
  - Explanation: The result of each ALU operation.
    If aluSel_i =

0	aluResult_o = sum_s;	ADD, ADDI, LB, LH, LW, LBU, LHU, LD, LWU, AUIPC, LUI, SB, SH, SW, SD, JALR, JAL
1	aluResult_o = sum_s;	SUB
2	aluResult_o = data01_i & data02_i;	AND, ANDI
3		OR
4		XOR
5		SLT
6		SLTU

5.1.4 Service Requests of the ALU

None.

5.1.5 The ALU Peripheral Kernel

The ALU calculates a result (aluResult_o) and the corresponding flags zero (aluZero_o), negative (aluNega_o), carry (asCarr_o) and overflow (asOver_o). Table 5.7 and table 5.8 are describing the function of the ALU for each instruction.

The zero flag is not only a zero flag but also the control for all branch instruction.

Table 5.7 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Table 5.7: RV32I Base Integer Instruction Set [2], [3], [1]

Instruction	Operation	Remarks
Load Upper Immediate	-	-
Add Upper Immediate to PC	-	-
Jump and Link	*jal rd, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow pc + offs$
Jump and Link Register	*jalr rd, rs1, offs*	$rd \leftarrow pc + len(instr)$, $pc \leftarrow (rs1 + offs) \wedge -2$
Branch Equal	*beq rs1, rs2, offs*	$if\ rs1 = rs2\ then\ pc \leftarrow pc + offs$
Branch Not Equal	*bne rs1, rs2, offs*	$if\ rs1 \neq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than	*blt rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal	*bge rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Branch Less Than Unsigned	*bltu rs1, rs2, offs*	$if\ rs1 < rs2\ then\ pc \leftarrow pc + offs$
Branch Greater than Equal Unsigned	*bgeu rs1, rs2, offs*	$if\ rs1 \geq rs2\ then\ pc \leftarrow pc + offs$
Load Byte	*lb rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half	*lh rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Load Word	*lw rd, offs(rs1)*	$rd \leftarrow s32[rs1 + offs]$

... next page

Instruction	Operation	Remarks
Load Byte Unsigned	*lbu rd, offs(rs1)*	$rd \leftarrow s8[rs1 + offs]$
Load Half Unsigned	*lhu rd, offs(rs1)*	$rd \leftarrow s16[rs1 + offs]$
Store Byte	*sb rs2, offs(rs1)*	$u8[rs1 + offs] \leftarrow rs2$
Store Half	*sh rs2, offs(rs1)*	$u16[rs1 + offs] \leftarrow rs2$
Store Word	*sw rs2, offs(rs1)*	$u32[rs1 + offs] \leftarrow rs2$
Add Immediate	*addi rd, rs1, imm*	$rd \leftarrow rs1 + sx(imm)$
Set Less Than Immediate	*slti rd, rs1, imm*	$rd \leftarrow sx(rs1) < sx(imm)$
Set Less Than Immediate Unsigned	*sltiu rd, rs1, imm*	$rd \leftarrow ux(rs1) < ux(imm)$
Xor Immediate	*xori rd, rs1, imm*	$rd \leftarrow ux(rs1) \oplus ux(imm)$
Or Immediate	*ori rd, rs1, imm*	$rd \leftarrow ux(rs1) \vee ux(imm)$
And Immediate	*andi rd, rs1, imm*	$rd \leftarrow ux(rs1) \wedge ux(imm)$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << ux(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> ux(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> ux(imm)$
Add	aluResult_o = sum_s = data01_i + cinvb_s + aluSel_i[0]	aluSel_i = 0; cinvb_s = aluSel_i[0] ? not data02_i : data02_i
Subtract	*sub rd, rs1, rs2*	$rd \leftarrow sx(rs1) - sx(rs2)$
Shift Left Logical	*sll rd, rs1, rs2*	$rd \leftarrow ux(rs1) << rs2$
Set Less Than	*slt rd, rs1, rs2*	$rd \leftarrow sx(rs1) < sx(rs2)$
Set Less Than Unsigned	*sltu rd, rs1, rs2*	$rd \leftarrow ux(rs1) < ux(rs2)$
Xor	*xor rd, rs1, rs2*	$rd \leftarrow ux(rs1) \oplus ux(rs2)$
Shift Right Logical	*srl rd, rs1, rs2*	$rd \leftarrow ux(rs1) >> rs2$
Shift Right Arithmetic	*sra rd, rs1, rs2*	$rd \leftarrow sx(rs1) >> rs2$
Or	*or rd, rs1, rs2*	$rd \leftarrow ux(rs1) \vee ux(rs2)$
And	*and rd, rs1, rs2*	$rd \leftarrow ux(rs1) \wedge ux(rs2)$
Fence	*fence pred, succ*	
Fence Instruction	*fence.i*	

End of table.

Table 5.8 lists all base integer instructions of a RISC-V 64 bit architecture, additional to the 32 bit integer instruction set, according the specification [1].

Table 5.8: RV64I Base Integer Instruction Set (in addition to RV32I) [2], [3], [1]

Instruction	Example	Meaning
Load Word Unsigned	*lwu rd, offs(rs1)*	$rd \leftarrow u32[rs1 + offs]$
Load Double	*ld rd, offs(rs1)*	$rd \leftarrow u64[rs1 + offs]$
Store Double	*sd rs2, offs(rs1)*	$u64[rs1 + offs] \leftarrow rs2$
Shift Left Logical Immediate	*slli rd, rs1, imm*	$rd \leftarrow ux(rs1) << sx(imm)$
Shift Right Logical Immediate	*srli rd, rs1, imm*	$rd \leftarrow ux(rs1) >> sx(imm)$
Shift Right Arithmetic Immediate	*srai rd, rs1, imm*	$rd \leftarrow sx(rs1) >> sx(imm)$
Add Immediate Word	*addiw rd, rs1, imm*	$rd \leftarrow s32(rs1) + imm$
Shift Left Logical Immediate Word	*slliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) << imm)$
Shift Right Logical Immediate Word	*srliw rd, rs1, imm*	$rd \leftarrow s32(u32(rs1) >> imm)$
Shift Right Arithmetic Immediate Word	*sraiw rd, rs1, imm*	$rd \leftarrow s32(rs1) >> imm$
Add Word	*addw rd, rs1, rs2*	$rd \leftarrow s32(rs1) + s32(rs2)$
Subtract Word	*subw rd, rs1, rs2*	$rd \leftarrow s32(rs1) - s32(rs2)$
Shift Left Logical Word	*sllw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) << rs2)$
Shift Right Logical Word	*srlw rd, rs1, rs2*	$rd \leftarrow s32(u32(rs1) >> rs2)$
Shift Right Arithmetic Word	*sraw rd, rs1, rs2*	$rd \leftarrow s32(rs1) >> rs2$

End of table.

5.1.6 Registers

None.

5.2 Instruction Decoder

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 5.9: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 5.10: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)

Functional Configuration of InstructionDecode0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a Instruction Decoder are described in its functional description in Section 5.2.2. Generic features, that can be decided for each specific

instantiation of the peripheral are listed and completed for InstructionDecode0 in Table 5.11.

The **RWU-RV64I** has only one Instruction Decoder.

Table 5.11: InstrDec0 Feature Set

	Generic GPIO Feature Set	Details
no	Seperate Kernel Clock	not needed
no	FIFO Data Buffering	not needed

HW Interfaces

The following table 5.12 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 5.12: HW Interfaces of Module InstrDec0

	Module internal name	Chip level name
Supply		
Clock	-	-
Bus	-	-
Interrupt	-	-
DMA	-	-
External Signals	-	-
Others	opcode_i(6:0) func3_i(2:0) func7b5_i zero_i resultSrc_o(1:0) dMemWr_o dMemRd_o PCSrc_o aluSrcB_o aluSrcA_o regWr_o jump_o immSrc_o(2:0) aluSel_o(4:0) branch_s_o jumo_s_o	iBusDataRd_s(6:0) iBusDataRd_s(14:12) iBusDataRd_s(30) zero_s resultSrc_s(1:0) dMemWr_s dMemRd_s PCSrc_s (not for pipeline) aluSrcB_s aluSrcA_s regWr_s jump_s immSrc_s(2:0) aluSel_s(4:0) only for pipeline only for pipeline

Bus Interface

None.

Registers

None.

Related Sections of this Spec

None.

5.2.1 History of Instruction Decoder

Table 5.13: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
5.2	5.2 / 2025-02-13	First draft (AS)

5.2.2 Functional Overview

The Instruction Decoder operates on 32 bit instructions and generates all control signals needed for the operation of the CPU.

General Features:

- Decodes all instructions of the RV64I:
  - opcode=3: I-type, loads
  - opcode=19: I-type, immediate arithmetics, logic and shift
  - opcode=23: U-type, auipc
  - opcode=27: I-type, immediate arithmetics, logic and shift (word)
  - opcode=35: S-type, stores
  - opcode=51: R-type, RV32I
  - opcode=55: U-type, lui
  - opcode=59: R-type, RV64I
  - opcode=99: B-type, all conditional branches
  - opcode=103: I-type, jalr
  - opcode=111: J-type, jal
- generates the control signal for the ALU,
- ... for the D-Mem MUX,
- ... for the D-Mem read- and write enable,

- ... for the PC MUX,
- ... for the register A and B output MUXes,
- ... for the register file write enable,
- ... for the branch control,
- ... for the immediate generation selection,
- ... for the jump and branch control in the pipelined version.

Synchronous Mode:

- -

Asynchronous Mode:

- -

5.2.3 Structural Overview

I/O of the Instruction Decoder

Table 5.14 shows the input and outputs of the ALU0. This are all primary inputs and outputs. There is no bi-directional, tristate or open drain pin. All pins are high-active except the pins with a _n_ in its name.

Table 5.14: InstrDec0 I/O

Pin	Dir.	Wd.	Explanation
opcode_i	in	7	The opcode field of the instruction. This are the least significant 7 bits of the instruction.
func3_i	in	3	The func3-field of the instruction. It specifies the opcode.
func7b5_i	in	1	Bit 5 of func7 field.
zero_i	in	1	Indicates a branch. Comes from the ALU.
resultSrc_o	out	2	Controls the Mux behind the DMem.
dMemWr_o	out	1	D-Mem write enable.
dMemRd_o	out	1	D-Mem read enable; almost not needed anymore.
PCSrc_o	out	1	Control of the Mux for the PC; normal operation or jump/branch.
aluSrcB_o	out	1	Data Mux in front of ALU.
aluSrcA_o	out	1	Data Mux in front of ALU.
regWr_o	out	1	Register file write enable.
jump_o	out	1	Src for branch target mux: register or PC as input for branch target adder.
immSrc_o	out	3	Selects, how the immediate value will be generated.
aluSel_o	out	5	ALU control.
branch_s_o	out	1	only for pipeline solution
jump_s_o	out	1	only for pipeline solution

Internals of the Instruction Decoder

The following list (Table) shows the internal signals of the Instruction Decoder. The table shows the signal name, the width, the driver, the target and a short explanation.

- Signal: opcode_i
  - Width: 7 bit unsigned
  - Source: Instruction from I-Mem.
  - Destination: Case statement generating controls_s and aluSel_o
  - Explanation:
    - opcode=3: loads
      - regWr_o=1; Write to register file: Yes
      - immSrc_o=3'b000; Instruction format: I-type
      - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=1; Read from D-Mem: Yes
- resultSrc_o=2'b01; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: D-Mem-data to RegFile
- branch_s=0; Branch: No
- jump_s=0; Only for jalr, jal
- aluSel_o=5'b00000 (ADD)

* opcode=19: arithmetic immediate
  - regWr_o=1; Write to register file: Yes
  - immSrc_o=3'b000; Instruction format: I-type
  - aluSrcB_o=1; Mux - Second register or immediate: immediate
  - aluSrcA_o=0; Mux - First register or PC: register
  - dMemWr_o=0; Write to D-Mem: No
  - dMemRd_o=0; Read from D-Mem: No
  - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
  - branch_s=0; Branch: No
  - jump_s=0; Only for jalr, jal
  - aluSel_o=various
* opcode=23: auipc
  - regWr_o=1; Write to register file: Yes
  - immSrc_o=3'b100; Instruction format: U-type
  - aluSrcB_o=1; Mux - Second register or immediate: immediate
  - aluSrcA_o=1; Mux - First register or PC: PC
  - dMemWr_o=0; Write to D-Mem: No
  - dMemRd_o=0; Read from D-Mem: No
  - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
  - branch_s=0; Branch: No
  - jump_s=0; Only for jalr, jal
  - aluSel_o=5'b00000 (ADD)
* opcode=27: arithmetic immediate (64 bit)
  - regWr_o=1; Write to register file: Yes
  - immSrc_o=3'b000; Instruction format: I-type
  - aluSrcB_o=1; Mux - Second register or immediate: immediate

        - aluSrcA_o=0; Mux - First register or PC: register
        - dMemWr_o=0; Write to D-Mem: No
        - dMemRd_o=0; Read from D-Mem: No
        - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
        - branch_s=0; Branch: No
        - jump_s=0; Only for jalr, jal
        - aluSel_o=various
    * opcode=35: stores
        - regWr_o=0; Write to register file: No
        - immSrc_o=3'b001; Instruction format: S-type
        - aluSrcB_o=1; Mux - Second register or immediate: immediate
        - aluSrcA_o=0; Mux - First register or PC: register
        - dMemWr_o=1; Write to D-Mem: Yes
        - dMemRd_o=0; Read from D-Mem: No
        - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
        - branch_s=0; Branch: No
        - jump_s=0; Only for jalr, jal
        - aluSel_o=5'b00000 (ADD)
    * opcode=51: arithmetic, logic and shifts
        - regWr_o=1; Write to register file: Yes
        - immSrc_o=3'bxxx; Instruction format: R-type (don't care)
        - aluSrcB_o=0; Mux - Second register or immediate: register
        - aluSrcA_o=0; Mux - First register or PC: register
        - dMemWr_o=0; Write to D-Mem: No
        - dMemRd_o=0; Read from D-Mem: No
        - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
        - branch_s=0; Branch: No
        - jump_s=0; Only for jalr, jal
        - aluSel_o=various
    * opcode=55: lui
        - regWr_o=1; Write to register file: Yes
        - immSrc_o=3'b100; Instruction format: U-type
        - aluSrcB_o=1; Mux - Second register or immediate: immediate

- aluSrcA_o=0; Mux - First register or PC: register
- dMemWr_o=0; Write to D-Mem: No
- dMemRd_o=0; Read from D-Mem: No
- resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
- branch_s=0; Branch: No
- jump_s=0; Only for jalr, jal
- aluSel_o=5'b00000 (ADD)

* opcode=59: arithmetic, logic and shifts (64 bit)
  - regWr_o=1; Write to register file: Yes
  - immSrc_o=3'bxxx; Instruction format: R-type (don't care)
  - aluSrcB_o=0; Mux - Second register or immediate: register
  - aluSrcA_o=0; Mux - First register or PC: register
  - dMemWr_o=0; Write to D-Mem: No
  - dMemRd_o=0; Read from D-Mem: No
  - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
  - branch_s=0; Branch: No
  - jump_s=0; Only for jalr, jal
  - aluSel_o=various
* opcode=99: branches
  - regWr_o=0; Write to register file: No
  - immSrc_o=3'bxxx; Instruction format: B-type
  - aluSrcB_o=0; Mux - Second register or immediate: register
  - aluSrcA_o=0; Mux - First register or PC: register
  - dMemWr_o=0; Write to D-Mem: No
  - dMemRd_o=0; Read from D-Mem: No
  - resultSrc_o=2'b00; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: ALU-result to RegFile
  - branch_s=1; Branch: Yes
  - jump_s=0; Only for jalr, jal
  - aluSel_o=various
* opcode=103: jalr
  - regWr_o=1; Write to register file: Yes
  - immSrc_o=3'b000; Instruction format: I-type
  - aluSrcB_o=1; Mux - Second register or immediate: immediate

            - aluSrcA_o=0; Mux - First register or PC: register
            - dMemWr_o=0; Write to D-Mem: No
            - dMemRd_o=0; Read from D-Mem: No
            - resultSrc_o=2'b10; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: PC+4 to RegFile
            - branch_s=0; Branch: No
            - jump_s=1; Only for jalr, jal
            - aluSel_o=5'b00000 (ADD)
        * opcode=111: jal
            - regWr_o=1; Write to register file: Yes
            - immSrc_o=3'b011; Instruction format: J-type
            - aluSrcB_o=0; Mux - Second register or immediate: register
            - aluSrcA_o=0; Mux - First register or PC: register
            - dMemWr_o=0; Write to D-Mem: No
            - dMemRd_o=0; Read from D-Mem: No
            - resultSrc_o=2'b10; Mux (behind D-Mem) - D-Mem-data or PC+4 or ALU-result: PC+4 to RegFile
            - branch_s=0; Branch: No
            - jump_s=1; Only for jalr, jal
            - aluSel_o=5'b00000 (ADD)

- Signal: func3_i
    - Width: 3 bit, unsigned
    - Source: Instruction from I-Mem.
    - Destination: Case statement generating controls_s and aluSel_o
    - Explanation:
        * opcode=19
            - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b00001 (SUBI) else aluSel_o=5'b00000 (ADDI)
            - func3_i=1: aluSel_o=5'b01111 (SLLI)
            - func3_i=2: aluSel_o=5'b00101 (SLTI)
            - func3_i=3: aluSel_o=5'b00110 (SLTIU)
            - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01110 (SRLI) else aluSel_o=5'b01101 (SRAI)
            - func3_i=4: aluSel_o=5'b00100 (XORI)

        - func3_i=6: aluSel_o=5'b00011 (ORI)
        - func3_i=7: aluSel_o=5'b00010 (ANDI)
      - opcode=27
        - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b01011 (SUBIW) else aluSel_o=5'b01100 (ADDIW)
        - func3_i=1: aluSel_o=5'b01010 (SLLIW)
        - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01001 (SRLIW) else aluSel_o=5'b01000 (SRAIW)
      - opcode=51
        - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b00001 (SUB) else aluSel_o=5'b00000 (ADD)
        - func3_i=1: aluSel_o=5'b01111 (SLL)
        - func3_i=2: aluSel_o=5'b00101 (SLT)
        - func3_i=3: aluSel_o=5'b00110 (SLTU)
        - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01110 (SRL) else aluSel_o=5'b01101 (SRA)
        - func3_i=4: aluSel_o=5'b00100 (XOR)
        - func3_i=6: aluSel_o=5'b00011 (OR)
        - func3_i=7: aluSel_o=5'b00010 (AND)
      - opcode=59
        - func3_i=0: if(func7b5_i & opcode_i[5]) aluSel_o=5'b01011 (SUBW) else aluSel_o=5'b01100 (ADDW)
        - func3_i=1: aluSel_o=5'b01010 (SLLW)
        - func3_i=5: if(func7b5_i == 0) aluSel_o=5'b01001 (SRLW) else aluSel_o=5'b01000 (SRAW)
      - opcode=99
        - func3_i=0: aluSel_o=5'b10001 (BEQ, SUB)
        - func3_i=1: aluSel_o=5'b10011 (BNE, SUB)
        - func3_i=4: aluSel_o=5'b10000 (BLT, <)
        - func3_i=5: aluSel_o=5'b10010 (BGE, >=)
        - func3_i=6: aluSel_o=5'b10100 (BLTU, <)
        - func3_i=7: aluSel_o=5'b10101 (BGEU, >=)

- Signal: func7b5_i
  - Width: 1 bit, unsigned
  - Source: Instruction from I-Mem.

  - Destination: Case statement generating controls_s and aluSel_o, rType-Subtract_s
  - Explanation: Differentiates between ADD and SUB, shift variants

- Signal: zero_i
  - Width: 1 bit, unsigned
  - Source: ALU
  - Destination: zero_s
  - Explanation: Indicates that a branch should be taken.

- Signal: zero_s
  - Width: 1 bit, unsigned
  - Source: zero_i
  - Destination: PCSrc_o
  - Explanation: Indicates that a branch should be taken.

$$PCSrc\_o = (branch\_s \& zero\_s) | jump\_s$$

- Signal: branch_s
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: PCSrc_o
  - Explanation: Indicates that a branch should be taken.

$$PCSrc\_o = (branch\_s \& zero\_s) | jump\_s$$

- Signal: jump_s
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: PCSrc_o
  - Explanation: Indicates that a branch should be taken.

$$PCSrc\_o = (branch\_s \& zero\_s) | jump\_s$$

- Signal: rTypeSubtract_s

  - Width: 1 bit, unsigned
  - Source:

$$rTypeSubtract\_s = func7b5\_i \& opcode\_i[5]$$

  - Destination: Case statement generating controls_s and aluSel_o
  - Explanation: Short form for distinguishing between ADD AND SUB.

- Signal: resultSrc_o
  - Width: 2 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: Mux behind D-Mem
  - Explanation:

    00 for ALU-result to RegFile
    01 for D-Mem-data to regFile
    10 for PC+4 to RegFile

- Signal: dMemWr_o
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: D-Mem
  - Explanation: Write-enable for D-Mem

- Signal: dMemRd_o
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: D-Mem
  - Explanation: Read-enable for D-Mem; check if realy needed

- Signal: PCSrc_o
  - Width: 1 bit, unsigned
  - Source:

$$PCSrc\_o = (branch\_s \& zero\_s) | jump\_s$$

  - Destination: PC

  - Explanation: Select input for the PC MUX.

    $0 \Rightarrow$ PC+4 (PCp4) to PC
    $1 \Rightarrow$ branch target address (PCbr) to PC

- Signal: aluSrcB_o
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: Select input for the MUX between RegFile and ALU
  - Explanation:

    $0 \Rightarrow$ 2nd RegFile output to 2nd source input of ALU
    $1 \Rightarrow$ immediate value to 2nd source input of ALU

- Signal: aluSrcA_o
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: Select input for the MUX between RegFile and ALU
  - Explanation:

    $0 \Rightarrow$ 1st RegFile output to 1st source input of ALU
    $1 \Rightarrow$ PC to 1st source input of ALU

- Signal: regWr_o
  - Width: 1 bit, unsigned
  - Source: generated from different instructions, see opcode_i
  - Destination: Write enable for the RegFile
  - Explanation: Write enable for the RegFile

- Signal: jump_o
  - Width: 1 bit, unsigned
  - Source:

    $$jump\_o = jump\_s \& (\sim immSrc\_o[0]) \& (\sim immSrc\_o[1]) \& (\sim immSrc\_o[2])$$

  - Destination: Select input for the MUX in front of the branch target adder

  - Explanation:

$$0 \Rightarrow \text{ PC to adder input A}$$
$$1 \Rightarrow \text{ regA to adder input A}$$

- Signal: immSrc_o

  - Width: 3 bit, unsigned

  - Source: generated from different instructions, see opcode_i

  - Destination: Immediate generator control

  - Explanation:

$$000 \Rightarrow \text{ I-type}$$
$$001 \Rightarrow \text{ S-type}$$
$$010 \Rightarrow \text{ B-type}$$
$$011 \Rightarrow \text{ J-type}$$
$$100 \Rightarrow \text{ U-type}$$

- Signal: aluSel_o

  - Width: 5 bit, unsigned

  - Source: generated from different instructions, see opcode_i

  - Destination: ALU control

  - Explanation:

    00000 ⇒ ADD, ADDI, (JAL, JALR, AUIPC, LUI, stores, loads)
    00001 ⇒ SUB, SUBI
    00010 ⇒ AND, ANDI
    00011 ⇒ OR, ORI
    00100 ⇒ XOR, XORI
    00101 ⇒ SLT, SLTI
    00110 ⇒ SLTU, SLTIU
    00111 ⇒ tbd
    01000 ⇒ SRAW, SRAIW
    01001 ⇒ SRLW, SRLIW
    01010 ⇒ SLLW, SLLIW
    01011 ⇒ SUBW, SUBIW
    01100 ⇒ ADDW, ADDIW
    01101 ⇒ SRA, SRAI
    01110 ⇒ SRL, SRLI
    01111 ⇒ SLL, SLLI
    10000 ⇒ BLT
    10001 ⇒ BEQ
    10010 ⇒ BGE
    10011 ⇒ BNE
    10100 ⇒ BLTU
    10101 ⇒ BGEU

- Signal: branch_s_o
  - Width: 1 bit, unsigned
  - Source: branch_s
  - Destination: Branch control for a pipelined solution
  - Explanation: pipeline solution only
- Signal: jump_s_o
  - Width: 1 bit, unsigned
  - Source: jump_s
  - Destination: Branch control for a pipelined solution
  - Explanation: pipeline solution only

5.2.4 Service Requests of the Instruction Decoder

None.

5.2.5 The InstrDec Peripheral Kernel

The Instruction Decoder decodes all RV64I instructions and delivers the control signals to all design elements in the CPU.

Table 5.15 lists all base integer instructions of a RISC-V 32 bit architecture according the specification [1]. A 64 bit architecture has the same instructions with an adapted length of some numbers and registers.

Legend:

- Ins: Instruction short
- op: opcode_i
- f3: func3_i
- 7: func7b5_i
- z: zero_i
- S: resultSrc_o
- W: dMemWr_o
- R: dMemRd_o
- P: PCSrc_o
- B: aluSrcB_o
- A: aluSrcA_o
- r: regWr_o
- j: jump_o
- im: immSrc_o
- aluSel: aluSel_o
- b: branch_s_o
- js: jump_s_o

Table 5.15: RV32I/RV64I Base Integer Instruction Set [2], [3], [1]

Ins	op	f3	7	z	S	W	R	P	B	A	r	j	im	aluSel	b	js
lb	0000011	000	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lh	0000011	001	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lw	0000011	010	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lbu	0000011	100	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lhu	0000011	101	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
ld	0000011	011	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
lwu	0000011	110	-	-	01	0	1	0	1	0	1	0	000	00000	0	0
addi	0010011	000	-	-	00	0	0	0	1	0	1	0	000	00000	0	0
slli	0010011	001	0	-	00	0	0	0	1	0	1	0	000	01111	0	0
slti	0010011	010	-	-	00	0	0	0	1	0	1	0	000	00101	0	0
sltiu	0010011	011	-	-	00	0	0	0	1	0	1	0	000	00110	0	0
xori	0010011	100	-	-	00	0	0	0	1	0	1	0	000	00100	0	0
srli	0010011	101	0	-	00	0	0	0	1	0	1	0	000	01110	0	0
srai	0010011	101	1	-	00	0	0	0	1	0	1	0	000	01101	0	0
ori	0010011	110	-	-	00	0	0	0	1	0	1	0	000	00011	0	0
andi	0010011	111	-	-	00	0	0	0	1	0	1	0	000	00010	0	0
auipc	0010111	-	-	-	00	0	0	0	1	1	1	0	100	00000	0	0
addiw	0011011	000	-	-	00	0	0	0	1	0	1	0	000	01100	0	0
slliw	0011011	001	0	-	00	0	0	0	1	0	1	0	000	01010	0	0
srliw	0011011	101	0	-	00	0	0	0	1	0	1	0	000	01001	0	0
sraiw	0011011	101	1	-	00	0	0	0	1	0	1	0	000	01000	0	0
sb	0100011	000	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sh	0100011	001	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sw	0100011	010	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
sd	0100011	011	-	-	00	1	0	0	1	0	0	0	001	00000	0	0
add	0110011	000	0	-	00	0	0	0	0	0	1	0	000	00000	0	0
sub	0110011	000	1	-	00	0	0	0	0	0	1	0	000	00001	0	0
sll	0110011	001	0	-	00	0	0	0	0	0	1	0	000	01111	0	0
slt	0110011	010	0	-	00	0	0	0	0	0	1	0	000	00101	0	0
sltu	0110011	011	0	-	00	0	0	0	0	0	1	0	000	00110	0	0
xor	0110011	100	0	-	00	0	0	0	0	0	1	0	000	00100	0	0
srl	0110011	101	0	-	00	0	0	0	0	0	1	0	000	01110	0	0
sra	0110011	101	1	-	00	0	0	0	0	0	1	0	000	01101	0	0
or	0110011	110	0	-	00	0	0	0	0	0	1	0	000	00011	0	0
and	0110011	111	0	-	00	0	0	0	0	0	1	0	000	00010	0	0

... next page

Ins	op	f3	7	z	S	W	R	P	B	A	r	j	i m	aluSel	b	js
lui	0110111	-	-	-	00	0	0	0	1	0	1	0	100	00000	0	0
addw	0111011	000	0	-	00	0	0	0	0	0	1	0	000	01100	0	0
subw	0111011	000	1	-	00	0	0	0	0	0	1	0	000	01011	0	0
sllw	0111011	001	0	-	00	0	0	0	0	0	1	0	000	01010	0	0
srlw	0111011	101	0	-	00	0	0	0	0	0	1	0	000	01001	0	0
sraw	0111011	101	1	-	00	0	0	0	0	0	1	0	000	01000	0	0
beq	1100011	000	-	0/1	00	0	0	0/1	0	0	0	0	010	10001	1	0
bne	1100011	001	-	0/1	00	0	0	0/1	0	0	0	0	010	10011	1	0
blt	1100011	100	-	0/1	00	0	0	0/1	0	0	0	0	010	10000	1	0
bge	1100011	101	-	0/1	00	0	0	0/1	0	0	0	0	010	10010	1	0
bltu	1100011	110	-	0/1	00	0	0	0/1	0	0	0	0	010	10100	1	0
bgeu	1100011	111	-	0/1	00	0	0	0/1	0	0	0	0	010	10101	1	0
jalr	1100111	000	-	-	10	0	0	1	1	0	1	0	000	00000	0	1
jal	1101111	-	-	-	10	0	0	1	0	0	1	0	011	00000	0	1

EoT

5.2.6 Registers

None.

Chapter 6

Peripherals

6.1 GPIO

6.1.1 System Integration of GPIO0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.1: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.2: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-02-13	Andreas Siggelkow	First draft (AS)
2025-11-19	Andreas Siggelkow	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout.

Functional Configuration of GPIO0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a GPOI peripheral are described in its functional description in Section 6.1.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for GPIO0 in Table 6.3.

Table 6.3: GPIO0 Feature Set

	Generic GPIO Feature Set	Details
yes	Seperate Kernel Clock	125 MHz
no	FIFO Data Buffering	not yet

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailled list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.4: HW Interfaces of Module GPIO0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	gpio_irq_o	gpio_irq_s
DMA	None	-
External Signals	gpio_io cs_o	gpio_io asGpioCs_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.5: Register List of Module GPIO0

Register Name(s) in Chip	Name used in module register description	Comment Comment
GPIO0_ID	id_reg_s	ID of GPIO0 peripheral
GPIO0_DIR	dir_reg_s	Direction of the GPIO: • dir_reg_s[i] = 0 -> output, • dir_reg_s[i] = 1 -> input
GPIO0_DATA	data_reg_s	Data driven to the GPIO or received from the GPIO.
GPIO0_IRQSS	irqss_reg_s	GPIO Interrupt Request Source Status Register
GPIO0_IRQSC	irqsc_reg_s	GPIO Interrupt Request Source Clear Register
GPIO0_IRQSM	irqsm_reg_s	GPIO Interrupt Request Source Mask Register
GPIO0_ISR	isr_reg_s	GPIO Interrupt Set Register
GPIO0_RIS	ris_reg_s	GPIO Raw Interrupt Status Register
GPIO0_IMSC	imsc_reg_s	GPIO Interrupt Mask Control Register
GPIO0_MIS	mis_reg_s	GPIO Masked Interrupt Status Register

Related Sections of this Spec

- GPIO functional description starting with Section 6.1.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”
  - Peripheral ID Registers Section “Architecture Concepts”
  - Peripheral Clocking Concept Section “Architecture Concepts”
  - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.1.2 History of GPIO

Table 6.6: History

History		
Target Spec.	Current version : 1.0, 2025-02-13 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-02-13		
6.1	6.1 / 2025-02-13	First draft (AS)
6.1	6.1 / 2025-11-19	Changed the BPI; removed the registers from the kernel and added them to GPIOTop. Added an interrupt. Removed the address output. changed the GPIO output to inout. (AS)

6.1.3 Functional Overview

The GPIO peripheral allows to drive 8 bit data from the Wishbone data bus to GPIO pins gpio_io (7:0) (input and output). The direction must be programmed

with the **GPIO0_DIR** register. A chip select (cs) is used to activate the GPIO output (dummy citation [4]).

General Features:

- Chip select
- FIFO data buffering (future)
- I/O
- Interrupt

Synchronous Mode:

- 8 data bits

Asynchronous Mode:

- -

6.1.4 Structural Overview

The GPIO is a Wishbone slave (asSlaveBPI) peripheral. This peripheral architecture aims for high bus performance by dividing peripherals into two clock domains. Figure 6.1 shows a simplified block diagram of the GPIO asSlaveBPI peripheral.

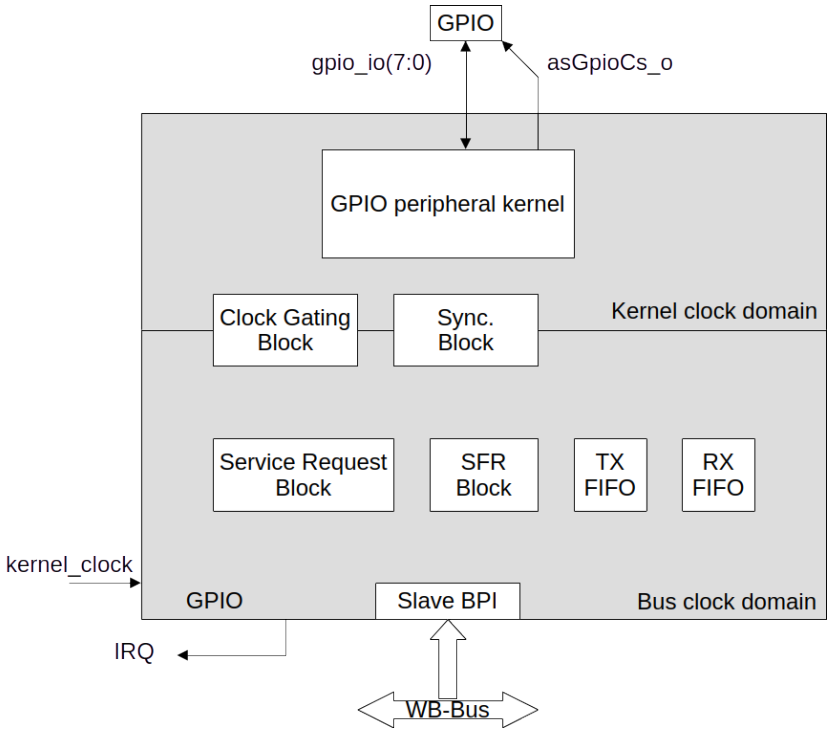

Figure 6.1: GPIO asSlaveBPI Peripheral

GPIO Peripheral Kernel: The GPIO kernel provides the specific functionality of the GPIO interface, that is described in Section 6.1.6. The peripheral kernel has its own kernel clock domain.

Bus Peripheral Interface BPI: The BPI is responsible for the interconnection of the GPIO registers with an on-chip bus. Also the peripheral clock gating is controlled by the BPI.

asSlaveBPI Block: The asSlaveBPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the GPIO Kernel

Table 6.7 shows the input and outputs of the GPIO kernel.

Table 6.7: GPIO Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
direction_i(7:0)	in	8	From GPIO-BPI (**GPIO0_DIR** register). Defines the direction of each GPIO. If the bit is 0, the GPIO is an output, if the bit is 1, the GPIO is an input.
addr_i(3:0)	in	4	From GPIO-BPI. Address for storing cs_s.
data_i(7:0)	in	8	From GPIO-BPI (**GPIO0_DATA** register). GPIO data in output direction. The elements of the register will be forwarded to a tristate buffer which is active when direction is output and 'Z' when direction is input.
data_o(7:0)	out	8	From pad to GPIO-BPI (**GPIO0_DATA** register). GPIO data in input direction. If the direction is input, the signals from the pads will go through a tristate buffer and will be forwarded to the register. Else, a '0' will be driven.
irq_o(7:0)	out	8	From kernel to GPIO-BPI. A change of an input will be detected and send to the IRQ register. Every possible GPIO input will be routed to an edge detector (both, rising and falling). Those detected edges will be forwarded to the IRQ structure in the BPI.
en_i	in	1	From GPIO-BPI. Enables a write to a GPIO block. It stores the chip select cs_o.
gpio_io	bidi	8	To inout pins. GPIO inout.
cs_o	out	1	To output pins. Chip select for outside GPIO block. Stored in an internal register (so, synchronized to the data).

I/O of the GPIO BPI

Table 6.8 shows the input and outputs of the GPIO BPI.

Table 6.8: GPIO BPI I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
addr_o(3:0)	out	4	To GPIO kernel. Address for external GPIO devices. addr_o = addr_i.
dat_from_core_i(7:0)	in	8	From GPIO kernel.
dat_to_core_o(7:0)	out	8	To GPIO kernel. GPIO outputs for one block. dat_to_core_o = dat_i
wr_o	out	1	To GPIO kernel. Enable for GPIO data and address, and chip select for outside GPIO blocks. wr_o = we_i AND stb_i.
addr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
dat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
dat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
we_i	in	1	From WB-Bus. Peripherals write enable.
sel_i(7:0)	in	8	From WB-Bus. Peripherals byte select. Not used here.
stb_i	in	1	From WB-Bus. Valid bus cycle. Combined (AND) with we_i for wr_o.
ack_o	out	1	To WB-Bus. Error free bus transaction. Here, ack_o = stb_i.
cyc_i	in	1	From WB-Bus. High for complete bus cycle. Not used here.

Internals of the GPIO BPI

Table 6.9 shows the internal signals and processes of the GPIO BPI.

Table 6.9: GPIO BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

I/O of the GPIO Top

Table 6.10 shows the input and outputs of the GPIO Top.

Table 6.10: GPIO Top I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
rst_i	in	1	System reset. Active high.
wbdAddr_i(3:0)	in	4	From WB-Bus. Peripherals addresses.
wbdDat_i(63:0)	in	64	From WB-Bus. Peripherals data input.
wbdDat_o(63:0)	out	64	To WB-Bus. Peripherals data output.
wbdWe_i	in	1	From WB-Bus. Peripherals write enable.
wbdSel_i(7:0)	in	8	From WB-Bus. Peripherals byte select.
wbdStb_i	in	1	From WB-Bus. Valid bus cycle.
wbdAck_o	out	1	To WB-Bus. Error free bus transaction.
wbdCyc_i	in	1	From WB-Bus. High for complete bus cycle.
gpio_irq_o	out	1	GPIO interrupt.
gpio_o(7:0)	out	8	To chip pins. GPIO outputs for one block.
cs_o	out	1	To chip pins. Chip select for outside GPIO blocks.

Table 6.11 shows the internals of the GPIO Top.

Table 6.11: GPIO Top Signals

Signal	Width	Explanation
irq_comb_s	1	An ORed signal of all interrupts comming from the GPIO kernel.
irqsc_comb_s	1	An ORed signal of all interrupt clear bits from register IRQSC.
irqss_reg_s	64	GPIO Interrupt Request Source Status Register **GPIO0_IRQSS**. If irq_comb_s = 1, the interrupts from the kernel will be written. Else, if irqsc_comb_s = 1, irqss_reg_s will be cleared and irqsc_reg_s will be cleared.
irqsc_reg_s	64	GPIO Interrupt Request Source Clear Register **GPIO0_IRQSC**. One interrupt request source clear bit is provided for each interrupt request source: 0 - No change, 1 - Clear Interrupt source.
irqsm_reg_s	64	GPIO Interrupt Request Source Mask Register **GPIO0_IRQSM**. One interrupt request source mask bit is provided for each interrupt request source: 0 - Interrupt Request Source Disabled, 1 - Interrupt Request Source Enabled.
irqsm_comb_s	1	An ORed signal of all masked single interrupts comming from the GPIO kernel.
ris_reg_s	64	GPIO Raw Interrupt Status Register **GPIO0_RIS**. One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
isr_reg_s	64	GPIO Interrupt Set Register **GPIO0_ISR**. One interrupt set bit is provided for each service request. The interrupt status bits in the **GPIO0_RIS** and the MIS (if IMSC = 1) registers are set by writing a '1' to the desired positions. 0 - No change, 1 - Set Interrupt. Reading the register returns 0.
imsc_reg_s	64	GPIO Interrupt Mask Control Register **GPIO0_IMSC**. One interrupt mask bit is provided for each service request: 0 - Interrupt Disabled, 1 - Interrupt Enabled.
mis_reg_s	64	GPIO Masked Interrupt Status Register **GPIO0_MIS**. One interrupt status bit is provided for each service request: 0 - No Interrupt, 1 - Interrupt Pending.
id_reg_s	64	GPIO ID Register **GPIO0_ID**.
dir_reg_s	64	GPIO Direction Register **GPIO0_DIR**.
data_reg_s	64	GPIO Data Register **GPIO0_DATA**. If the direction is input, the values from the pad will be written to the register. If the direction is output, the values from the bus will be written to the register.

6.1.5 Service Requests of the GPIO

Table 6.12 shows the service requests of the GPIO.

Table 6.12: GPIO Service Requests

Service Request	IRQ	Explanation
gpio_io (0)	GPIO_INT	The input on gpio_io (0) changed.
gpio_io (1)	GPIO_INT	The input on gpio_io (1) changed.
gpio_io (2)	GPIO_INT	The input on gpio_io (2) changed.
gpio_io (3)	GPIO_INT	The input on gpio_io (3) changed.
gpio_io (4)	GPIO_INT	The input on gpio_io (4) changed.
gpio_io (5)	GPIO_INT	The input on gpio_io (5) changed.
gpio_io (6)	GPIO_INT	The input on gpio_io (6) changed.
gpio_io (7)	GPIO_INT	The input on gpio_io (7) changed.

6.1.6 The GPIO Peripheral Kernel

As common for asSlaveBPI peripherals there are also two main states of the GPIO's kernel.

Before it can actually start operation, it has to be configured. This configuration state is left by setting bit RUN in register RUN_CTRL. Thereby the configuration registers are locked for write accesses, the functional blocks of peripheral kernel and asSlaveBPI block are reset (e.g. the FIFOs are flushed) and working state is entered. The different functional blocks of the GPIO peripheral kernel start operating as described in the following.

Figure 6.2 shows the principal functionality of the GPIO kernel. The shown registers are part of the SFR block of the peripheral. When there is a '1' written to the direction register, the pad acts as an input and the captured value will be written to the data register. When there is a '0' written to the direction register, the pad acts as an output and the value from the data register will be written to the pad. The data register is rw from the bus side.

In case of an input setting, a signal change will be detected and written to the IRQ register inside the SRB.

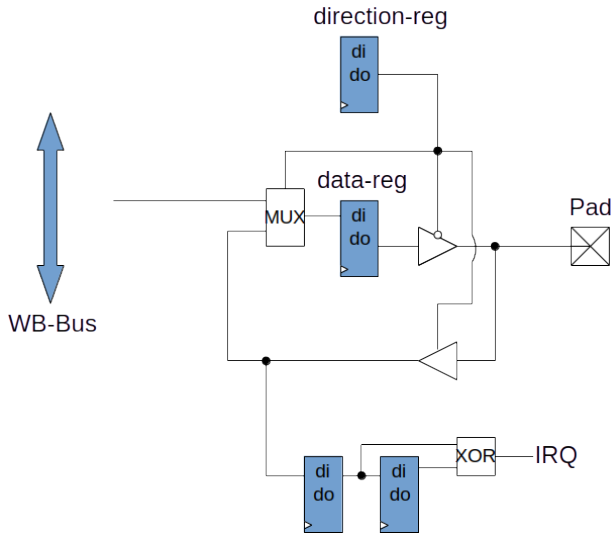

Figure 6.2: GPIO Kernel

6.1.7 Registers

In the following the registers of a GPIO peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that GPIO's instantiation. Such registers and concerned bit fields are denoted by the use of italic style.

GPIO Register Overview

Table 6.13: GPIO Register List

Register Group	Register Name	Register Symbol
System Registers	(Clock Control Register)	(**CLC**)
	Identification Register	**ID**
Special Function Registers	(Run Control Register)	(**RUN_CTRL**)
	Direction Register	**GPIO0_DIR**
	Data Register	**GPIO0_DATA**
Interrupt Registers	Interrupt Mask Control Register	**GPIO0_IMSC**
	Raw Interrupt Status Register	**GPIO0_RIS**
	Masked Interrupt Status Register	**GPIO0_MIS**
	Interrupt Set Register	**GPIO0_ISR**
	Interrupt Source Status Register	**GPIO0_IRQSS**
	Interrupt Source Mask Register	**GPIO0_IRQSM**
	Interrupt Source Clear Register	**GPIO0_IRQSC**

Read-Write Features

- w: writable bit, by SW
- r: readable bit, by SW
- h: bit will be set by HW only

System Registers

GPIO_ID -: Identification register of this peripheral. A write will have no effect, a read will return the fixed ID. The ID will be defined on chip level.

ID register (in BPI) [Reset value: $000100000000C001_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
MOD_NUMBER															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
1	1	0	0	0	0	0	0	REV_NUMBER							
r	r	r	r	r	r	r	r	r							

Field	Bits	Type	Description
MOD_NUMBER	63:48	r	Module Identification Number.
-	47:16	r	Reserved
-	15:8	r	The value C0 indicates the peripheral as 64-bit module.
REV_NUMBER	7:0	r	Module Revision Number: Bits 7-0 are used for module revision numbering.

Special Function Registers

Interrupt Registers

6.2 Instruction Memory (I-Mem)

6.2.1 System Integration of I-Mem0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.14: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	November 2025

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.15: History

Date	Name	Content Changes
Version of this document: V0.1		
2025-11-27	Andreas Siggelkow	First draft (AS)

Functional Configuration of I-Mem0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a **I-Mem** peripheral are described in its functional description in Section 6.2.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for **I-Mem0** in Table 6.16.

Table 6.16: **I-Mem0** Feature Set

	Generic **I-Mem** Feature Set	Details
yes	Seperate Kernel Clock	125 MHz
no	FIFO Data Buffering	no

HW Interfaces

The following table 6.4 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailled list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may be different). A full list is part of the design spec.

Table 6.17: HW Interfaces of Module **I-Mem0**

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	None	-
DMA	None	-
External Signals	addr_i data_o data_i wr_i	iBusAddr2_s iBusDataRd_s iBusDataWr_s iMemWr_s

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.18: Register List of Module **I-Mem0**

Register Name(s) in Chip	Name used in module register description	Comment Comment
None	-	-

Related Sections of this Spec

- **I-Mem** functional description starting with Section 6.2.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”
  - Peripheral ID Registers Section “Architecture Concepts”
  - Peripheral Clocking Concept Section “Architecture Concepts”
  - Clock Control Registers Section “Architecture Concepts”

- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.2.2 History of I-Mem

Table 6.19: History

History		
Target Spec.	Current version : 1.0, 2025-11-27 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2025-11-27		
6.2	6.2 / 2025-11-27	First draft (AS)

6.2.3 Functional Overview

The **I-Mem** peripheral is the internal instruction memory of the **RWU-RV64I** .

General Features:

- 32 kByte ROM (8192 words/instructions)
- Linear addressable
- 32 bit data width
- Wishbone bus slave

Synchronous Mode:

- 32 data bits (write; program download via JTAG)

Asynchronous Mode:

- 32 data bits (read; normal operation)

6.2.4 Structural Overview

The **I-Mem** is a Wishbone slave (asSlaveBPI) peripheral. It has one write port (for JTAG download) and one read port (for normal operation). The address is shared.

I/O of the I-Mem Kernel

Table 6.20 shows the input and outputs of the **I-Mem** kernel.

Table 6.20: **I-Mem** Kernel I/O

Pin	Direction	Width	Explanation
clk_i	in	1	System clock
addr_i	in	15	From program counter (PC). There is no byte access, so the lower two address bits will not be needed. In case of a program download, this is a part of the JTAG scan-chain (outside the block).
data_o	out	32	Instruction. Goes to the instruction decoder.
data_i	in	32	Data input port. From JTAG scan-chain (JTAG data register). This port is used for a program download.
wr_i	in	1	Write enable for data download. From JTAG scan-chain (JTAG data register). This port is used for a program download.

6.2.5 Service Requests of the I-Mem

None.

6.2.6 The I-Mem Peripheral Kernel

The memory is implemented as a synchronous write, asynchronous read memory. The write will be done on the rising edge of TCK and a write enable from a JTAG scan-chain.

6.2.7 Registers

None.

6.3 QSPI (AS)

6.3.1 System Integration of QSPI0

History

Regulations for author history:

- Only visible internally (by conditioning of the table, no other conditions needed)

Table 6.21: Authors

Author	Department	From
Andreas Siggelkow	Fac. E	February 2026

Regulations for history table:

- Last version on top.
- Dont use links in the history table. These may cause unexpected history changes in future versions.
- Name (or short sign) should be visible only internally.
- Create a new version after each (also limited) release of the document. Versioning does not depend on the overall document version.
- Use the form x.y for the version. x for the master versions, y for the project specific changes and updates.

Table 6.22: History

Date	Name	Content Changes
Version of this document: V0.1		
2026-02-16	Andreas Siggelkow	First draft (AS, ChatGPT)
2026-04-13	A. Siggelkow, Claude	Completely revised HW

Functional Configuration of QSPI0

This section should describe the product specific instantiation of this module's feature set (which may be a subset of the full feature set). It is intended also for use in the product overview section of the target spec/summary target spec.

The standard features of a QSPI peripheral are described in its functional description in Section 6.3.3. Generic features, that can be decided for each specific instantiation of the peripheral are listed and completed for QSPI0 in Table 6.23.

Table 6.23: QSPI0 Feature Set

	Generic QSPI Feature Set	Details
yes	Separate Kernel Clock	125 MHz (TBD)
yes	FIFO Data Buffering	RX and TX, each 16 entries $\times$ 64 bit = 128 bytes
yes	Programmable Baud Rate	$f_{SCK} = f_{kernel}/(2 \times (\text{CLKDIV} + 1))$, CLKDIV $\in [0 \ldots 255]$
yes	Single SPI Mode	1-1-1, cmd/adr/data on one wire
yes	Dual SPI Mode	1-1-2, data on two wires (DUAL=1, QUAD=0)
yes	Quad SPI Mode	1-1-4 and 1-4-4, data on four wires (QUAD=1)
yes	Configurable SPI Mode	CPOL and CPHA selectable (Mode 0 and Mode 3)
yes	3-/4-Byte Address	Selectable via ADDR_LEN bit
yes	Execute-In-Place (XIP)	Auto-read on bus access, fixed opcode from CMD register
yes	Double Data Rate (DDR)	DDR bit in CTRL register
yes	Interrupt Generation	7 interrupt sources, maskable via IMSC
yes	FIFO Flush	TX_FLUSH and RX_FLUSH bits in CTRL register
yes	Configurable Timeout	Timeout counter, threshold set via TIMEOUT register

HW Interfaces

The following table 6.24 lists all hardware interfaces which are relevant for the regular operation of the device. It is not a detailed list of all signals and does not include implementation specific informations.

The names used below show the functional connectivity. There is no guarantee that they are electrically compatible (e.g. voltage levels, clock domains etc. may

be different). A full list is part of the design spec.

Table 6.24: HW Interfaces of Module QSPI0

	Module internal name	Chip level name
Supply		
Clock	clk_i	clk_i
Bus	System Bus	Wishbone
Interrupt	qspi_irq_o	qspi_irq_s
DMA	None	-
External Signals	sck_o cs_o data_io[3:0]	qspiSck_o qspiCs_o qspi_data_io [3:0]

Bus Interface

Wishbone Slave BPI.

Registers

Table 6.25: Register List of Module QSPI0

offset	Register Name(s) in Chip	Name used in module register description	Comment
0x00	**QSPI0_ID**	id_reg_s	ID of QSPI0 peripheral
0x08	**QSPI0_CTRL**	ctrl_reg_s	Control register
0x10	**QSPI0_CMD**	cmd_reg_s	SPI command opcode
0x18	**QSPI0_ADDR**	addr_reg_s	Target flash address
0x20	**QSPI0_LEN**	len_reg_s	Transfer length in bytes
0x28	**QSPI0_DUMMY**	dummy_reg_s	Dummy cycles
0x30	**QSPI0_CLKDIV**	clkdiv_reg_s	SCK clock divider
0x38	**QSPI0_TIMEOUT**	timeout_reg_s	Timeout threshold register
0x40	**QSPI0_ISR**	isr_reg_s	Interrupt Set Register
0x48	**QSPI0_RIS**	ris_reg_s	Raw Interrupt Status Register
0x50	**QSPI0_IMSC**	imsc_reg_s	Interrupt Mask Control Register
0x58	**QSPI0_MIS**	mis_reg_s	Masked Interrupt Status Register
0x60	**QSPI0_ICR**	icr_reg_s	Interrupt Clear Register
0x68	**QSPI0_RXDATA**	rxdata_reg_s	RX FIFO read
0x70	**QSPI0_TXDATA**	txdata_reg_s	TX FIFO write
0x78	**QSPI0_FIFOSTAT**	fifostat_reg_s	FIFO fill levels
0x80	**QSPI0_XIPMODE**	xipmode_reg_s	XIP mode
0x88	**QSPI0_STATUS**	status_reg_s	Status flags

Related Sections of this Spec

- QSPI functional description starting with Section 6.3.3.
- Interrupt and DMA Concept Section “Architecture Concepts”
- Peripheral Architecture Concept Section “Architecture Concepts”
  - Slave WB (Wishbone) BPI Section “Architecture Concepts”

- Peripheral ID Registers Section “Architecture Concepts”
- Peripheral Clocking Concept Section “Architecture Concepts”
- Clock Control Registers Section “Architecture Concepts”
- Horizontal Interrupt and DMA Register Structure Section “Architecture Concepts”
- Basic Interrupt and DMA Register Set Section “Architecture Concepts”
- Interrupt Source Combining Register Set Section “Architecture Concepts”
- Synchronization Block Section “Architecture Concepts”
- Standardized Flip-Flop based FIFO Section “Architecture Concepts”
- TX-FIFO Registers Section “Architecture Concepts”
- RX-FIFO Registers Section “Architecture Concepts”

6.3.2 History of QSPI

Table 6.26: History

History		
Target Spec.	Current version : 1.0, 2026-02-17 Previous version : 0.0, 2020-10-12	Author: A. Siggelkow
Paragraph (in previous version)	Paragraph (in current version) / date	Subjects (major changes since last revision)
Current Chapter Version: C0.1; Date: 2026-02-17		
6.3	6.3 / 2026-02-17	First draft (AS)
6.3	6.3 / 2026-04-13	Complete rework (AS, Claude)

6.3.3 Functional Overview

The QSPI controller is a memory-mapped peripheral providing high-speed SPI and Quad-SPI access to external flash devices.

The controller is accessed via a 64-bit Wishbone slave interface (dummy citation [4]).

General Features:

- Single, Dual, and Quad SPI modes
- Configurable dummy cycles
- Programmable baud rate
- TX and RX FIFOs
- Interrupt generation
- Execute-In-Place (XIP)

Synchronous Mode:

- 4 data bits

Asynchronous Mode:

- -

Reference Flash Devices

The QSPI controller is designed to be compatible with widely used NOR-Flash devices. Table 6.27 shows the relevant operating parameters for two representative devices. All register default values in this specification are set to match the Winbond W25Q128JV in standard (non-DDR) mode with CPOL=0, CPHA=0.

Table 6.27: Reference NOR-Flash Devices

Parameter	Winbond W25Q128JV	Micron MT25QL128	Comment
Capacity	128 Mbit (16 MByte)	128 Mbit (16 MByte)	
Supply Voltage	2.7 V…3.6 V	2.7 V…3.6 V	
SPI Modes supported	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	Mode 0 (CPOL=0, CPHA=0) and Mode 3 (CPOL=1, CPHA=1)	
Max. clock (STR)	133 MHz (Quad)	133 MHz (Quad)	
Default address width	3 Byte (24 bit)	3 Byte (24 bit)	4-Byte mode selectable
Quad enable	QE bit in Status Reg 2	via Enhanced Volatile Config Reg (0x61)	Must be set before QUAD=1
Relevant Read Opcodes:			
Read (slow)	0x03 (1-1-1, 0 dummy)	0x03 (1-1-1, 0 dummy)	$\leq$50 MHz
Fast Read	0x0B (1-1-1, 8 dummy)	0x0B (1-1-1, 8 dummy)	up to 133 MHz
Dual Output Read	0x3B (1-1-2, 8 dummy)	0x3B (1-1-2, 8 dummy)	DUAL=1
Quad Output Read	0x6B (1-1-4, 8 dummy)	0x6B (1-1-4, 8 dummy)	QUAD=1
Quad I/O Read	0xEB (1-4-4, 6+2 dummy)	0xEB (1-4-4, 10 dummy)	QUAD=1, fast
Relevant Write / Control Opcodes:			
Write Enable	0x06	0x06	Required before program/erase
Page Program	0x02 (1-1-1)	0x02 (1-1-1)	Up to 256 Byte
Sector Erase (4 KB)	0x20	0x20	
Block Erase (64 KB)	0xD8	0xD8	
Chip Erase	0xC7 / 0x60	0xC7 / 0x60	
JEDEC ID Read	0x9F	0x9F	Returns 3 Byte
JEDEC Manufacturer ID	0xEF (Winbond)	0x20 (Micron)	
Typical Dummy Cycles by Mode and Clock:			
1-1-1 Fast Read	8 cycles	8 cycles	DUMMY = 0x08
1-1-4 Quad Output	8 cycles	8 cycles	DUMMY = 0x08
1-4-4 Quad I/O	6 mode + 2 dummy cycles	10 cycles	DUMMY = 0x08 / 0x0A

Note on Quad Enable: Both reference devices require a Quad Enable (QE) bit to be set in the flash device's non-volatile status register before QUAD mode can be used. This is a one-time (or volatile) configuration step performed by the SW driver via a standard Write Enable (0x06) followed by a Write Status Register command. The QSPI controller itself does not perform this step automatically.

Functional Description OSPI-Kernel

The `as_qspi` kernel implements the SPI/QSPI state machine and clock generator. It is a fully state-driven Moore FSM. No FSM state depends on interrupt signals; all external behavior is driven by register state only.

The FSM traverses the states IDLE $\rightarrow$ CMD $\rightarrow$ ADDR $\rightarrow$ DUM $\rightarrow$ DAT $\rightarrow$ DONE $\rightarrow$ IDLE for a normal transaction. In XIP mode the CMD phase is omitted after the first transaction.

The SCK clock is generated internally from `clk_i` via a symmetric counter. The SCK frequency is:

$$f_{SCK} = \frac{f_{clk}}{2 \times (\text{CLKDIV} + 1)}$$

The kernel drives `data_io[3:0]` as output during CMD, ADDR, and TX-DAT phases and samples `data_io[0]` (single), `data_io[1:0]` (dual), or `data_io[3:0]` (quad) during RX-DAT phases.

Bits per Cycle (bpc):

- QUAD=1: bpc = 4
- DUAL=1, QUAD=0: bpc = 2
- DUAL=0, QUAD=0: bpc = 1 (default)

Start Semantics:

$$start\_s = (start\_i == 1) \ \wedge \ (stat\_busy\_o == 0)$$

The `start_i` signal must be a one-cycle pulse, generated by the BPI/Top from the CTRL.START write strobe.

XIP Mode: When `ctrl_reg_i.xip = 1` the CMD phase is skipped on subsequent transactions. The controller sets `xip_active_o` after the first DONE state. XIP is cleared by writing CTRL.XIP = 0; the current transfer completes before IDLE.

Error Latching:

- TX underflow: `tx_empty_i = 1` while in DAT phase and `tx_flag = 1` (TX direction)
- RX overflow: `rx_full_i = 1` while in DAT phase and `tx_flag = 0` (RX direction)
- Timeout: internal counter reaches zero while kernel is active

Kernel State Machine

Table 6.28 lists the states of the kernel FSM.

Table 6.28: `as_qspi` FSM States

State	`cs_o`	`sck_o`	Condition to leave
IDLE_ST	0	0	`start_i = 1`
CMD_ST	1	running	`cnt_cmd_r` has reached CMD_MAX (7); exit on `sck_drive_s`
ADDR_ST	1	running	`cnt_addr_r` has reached `addr_max_s`; exit on `sck_drive_s`
DUM_ST	1	running	`cnt_dum_r + 1` $\geq$ `dummy_reg_i`; exit on `sck_rise_s`
DAT_ST	1	running	`cnt_dat_r + bpc_s > dat_max_s`; exit on last sample (RX) or last drive (TX)
DONE_ST	0	0	Unconditional, one cycle

Figure 6.3 shows the state machine of the QSPI.

A simple Moore-FSM (Fig. 6.4): Additional to the FSM, a clock generator for sck_gen is required. This clock will be enabled or disabled by the FSM (clock enable cell). And more a few counters will be needed: a counter for the CMD-cycles (cnt_cmd), a counter for the ADDR-cycles (cnt_adr), a counter for the DUMMY-cycles (cnt_dum) and a counter for the DATA-cycles (cnt_dat). The counters end values will be either constants or loaded by a register. When writing to the TX-register a signal tx will be derived, this controls the direction of the data flow in the DATA state of the FSM.

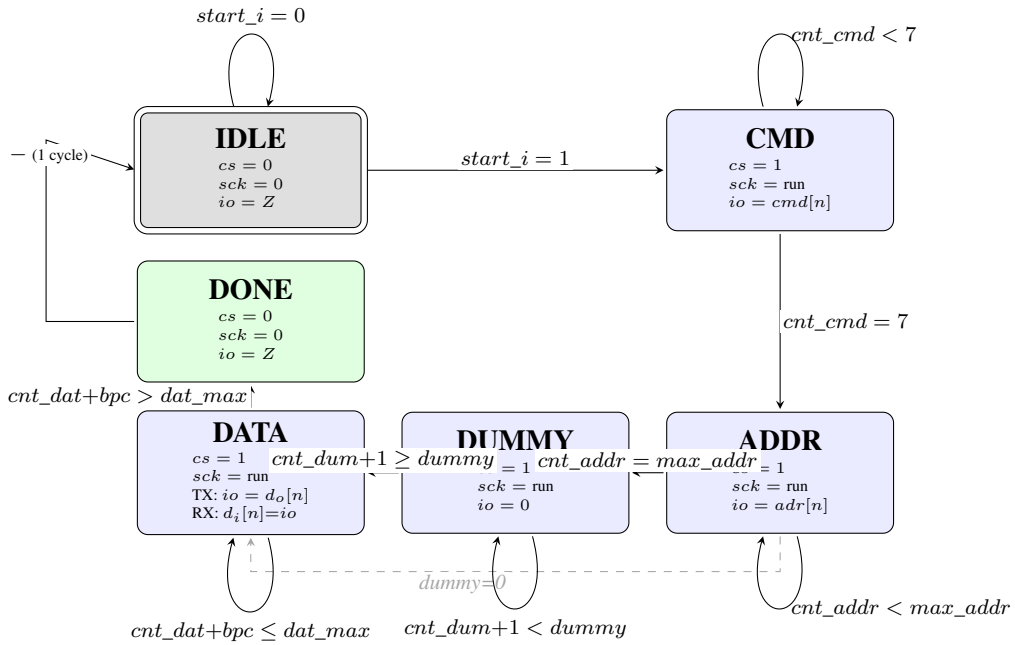

Figure 6.3: QSPI Kernel Moore-FSM. Doppelter Rahmen = Reset-Zustand IDLE (grau). Grüner Rahmen = DONE (1 Zyklus, bedingungslos zu IDLE). Gestrichelter Pfad: ADDR → DATA wenn `DUMMY_CYC` = 0.

Table 6.29 shows the kernel internel counters for the different parts of the SPI waveform.

Table 6.29: `as_qspi` Internal Counters

Counter	Width	Description
`sck_cnt_r`	8	SCK clock divider counter. Resets to `clkdiv_reg_i`.
`cnt_cmd_r`	3	CMD phase bit counter. Counts 0..7 (always 8 bits, single wire).
`cnt_addr_r`	5	ADDR phase bit counter. Max = 23 (3-byte) or 31 (4-byte).
`cnt_dum_r`	6	DUMMY cycle counter. Counts up to `dummy_reg_i`.
`cnt_dat_r`	20	DAT phase bit counter. Counts in steps of `bpc_s`. Max = `len_reg_i*8 - 1`.

Timing Constraints

- All configuration registers (cmd_reg_i, addr_reg_i, len_reg_i, etc.) must be stable before start_i is asserted and must remain stable for the duration of the transfer (BUSY=1).
- clkdiv_reg_i must not change while BUSY=1.
- tx_data_i must be valid when tx_rd_o fires. The BPI preloads the TX shift register one cycle before DAT_ST.
- rx_data_o is valid two clock cycles after rx_wr_o (NBA settling time in simulation; in synthesis: same cycle is valid).

Functional Description QSPI-BPI

The as_slave_bpi module implements a Wishbone Classic Compliant Slave BPI. It provides zero-wait-state single read/write transfers to the attached peripheral (QSPI top level). The BPI is fully combinatorial on the bus side; ACK is asserted in the same clock cycle as STB.

Protocol: A valid Wishbone transfer requires CYC_I = 1 and STB_I = 1 simultaneously. All SEL bits must be asserted (SEL_I = 0xFF) for a transfer to be acknowledged. The ACK is combinatorial:

$$ACK\_O = CYC\_I \ \wedge \ STB\_I \ \wedge \ (\&SEL\_I)$$

Write: When WE_I = 1, WR_O is asserted to the peripheral for the duration of the bus cycle. The address decoder in the Top level routes the write to the appropriate register.

Read: When WE_I = 0, RD_O is asserted. The peripheral must provide read data combinatorially on DAT_FROM_CORE_I; this is passed directly to WB_S_DAT_O.

Functional Description QSPI-Top-Level

as_qspi_top integrates the Wishbone BPI (as_slave_bpi), the QSPI kernel (as_qspi), the TX FIFO, and the RX FIFO into a complete memory-mapped QSPI peripheral. It decodes the register offset address, routes read/write strobes to the appropriate registers, and implements the interrupt logic.

Register Address Decoding: The BPI passes the raw bus address directly as a byte offset. The Top level compares this offset against the constants defined in Table 6.37 using combinatorial equality. There is no base-address stripping; the bus master must provide the byte offset from the peripheral's base address.

Start Pulse Generation: Writing `CTRL.START (bit 3) = 1` causes the Top level to register the write strobe for one clock cycle:

$$start\_r \leftarrow wr\_ctrl \wedge data\_wr\_s[3]$$

$$start\_s = start\_r \quad \text{(one-cycle pulse, registered)}$$

FIFO Flush: `CTRL.TX_FLUSH (bit 1)` and `CTRL.RX_FLUSH (bit 2)` are combinatorial and active only during the write cycle:

$$tx\_flush\_s = wr\_ctrl \wedge data\_wr\_s[1]$$

$$rx\_flush\_s = wr\_ctrl \wedge data\_wr\_s[2]$$

Flushing while BUSY is asserted is implementation-defined; in this implementation the flush takes effect immediately.

RX FIFO Write Pipeline: The kernel outputs `rx_wr_o` and `rx_data_o` (`rx_shift_r`) as NBA assignments in the same clock cycle. To avoid a simulation race condition (and to provide a registered interface for synthesis), a two-cycle pipeline is used:

- Cycle N: `rx_wr_kernel_s = 1`, `rx_shift_r` NBA pending.
- Cycle N+1: `rx_wr_d1 = 1`, `rx_data_kernel_s` captured into `rx_data_snap` (shift register now settled).
- Cycle N+2: `rx_wr_d2 = 1`, `rx_data_snap` written to RX FIFO.

RX FIFO Pop (read-before-pop): Reading `RXDATA` presents `mem[rd_ptr_r]` combinatorially and schedules the pointer advance one cycle later via `rx_pop_r`. This separates data presentation from pointer advancement and is required for correct operation with the asynchronous-read `as_fifo`.

6.3.4 Structural Overview

The QSPI is a Wishbone slave (asSlaveBPI) peripheral. This peripheral architecture aims for high bus performance by dividing peripherals into two clock domains. Figure 6.4 shows a simplified block diagram of the QSPI asSlaveBPI peripheral.

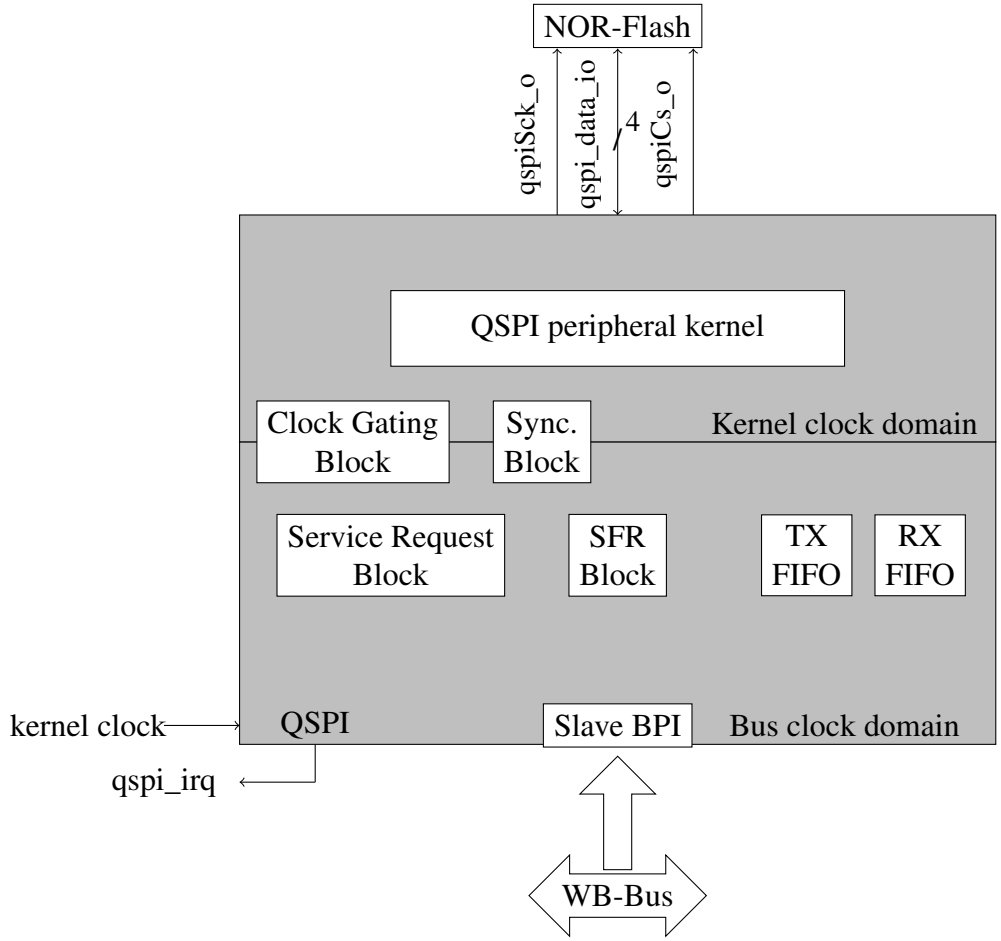

Figure 6.4: QSPI asSlaveBPI Peripheral

QSPI Peripheral Kernel: The QSPI kernel provides the specific functionality of the QSPI interface, that is described in Section 6.3.6. The peripheral kernel has its own kernel clock domain.

Bus Peripheral Interface BPI: The BPI is responsible for the interconnection of the QSPI registers with an on-chip bus. Also the peripheral clock gating is controlled by the BPI.

asSlaveBPI Block: The asSlaveBPI block lies within the bus clock domain and contains the following subblocks:

Special Function Register Block The Special Function Register Block contains all FIFO specific registers and all special function registers (SFR) of the peripheral kernel that are accessible by the SW via the system bus.

Service Request Block SRB The SRB is used to prepare the interrupt and data transfer requests for the Interrupt Control Unit (ICU) and the DMA Controller (DMAC), respectively (no DMA planned!).

TX FIFO The TX FIFO is used to buffer the transmission data from the bus, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the system bus.

RX FIFO The RX FIFO is used to buffer the received data from the kernel, in order to adapt the character processing speed of the peripheral kernel to the transfer rate of the bus system.

Clock Gating Block: The Clock Gating Block is used to derive the necessary clocks for the several blocks within the peripheral from the clocks provided by the system.

Synchronization Block: The Synchronization Block is used to synchronize the signals between the bus clock and kernel clock domains.

I/O of the QSPI Kernel

Table 6.30, 6.31 shows the input and outputs of the QSPI kernel.

Table 6.30: `as_qspi` Ports – System and Configuration

Port	Dir.	Width	Description
`clk_i`	in	1	Kernel clock, active rising edge
`rst_i`	in	1	Synchronous reset, active high
`start_i`	in	1	Start pulse (one cycle wide). Initiates FSM if not busy.
`ctrl_reg_i`	in	8	Packed `qspi_ctrl_t`: {addr_len, cpol, cpha, dual, cs_hold, xip, ddr, quad}
`cmd_reg_i`	in	8	SPI opcode byte
`addr_reg_i`	in	32	Flash target address
`len_reg_i`	in	16	Transfer length in bytes
`dummy_reg_i`	in	6	Number of dummy cycles
`clkdiv_reg_i`	in	8	SCK clock divider
`timeout_reg_i`	in	32	Timeout threshold in clock cycles; 0 = disabled
`xip_mode_bits_i`	in	8	XIP mode byte, driven on IO[3:0] during XIP address phase

Table 6.31: as_qspi Ports – Status, FIFO and SPI

Port	Dir.	Width	Description
stat_busy_o	out	1	FSM is active (not IDLE, not DONE). Level signal.
stat_done_o	out	1	FSM is in DONE state. High for exactly one clock cycle.
stat_error_o	out	1	Error latched. Remains until ICR clear.
stat_timeout_o	out	1	Timeout occurred. Remains until ICR clear.
xip_active_o	out	1	XIP mode is active.
tx_empty_i	in	1	TX FIFO empty indication (from TX FIFO or BPI)
tx_rd_o	out	1	TX FIFO read strobe. One cycle pulse on last TX drive.
tx_data_i	in	64	TX data word from TX FIFO
rx_full_i	in	1	RX FIFO full indication
rx_wr_o	out	1	RX FIFO write strobe. One cycle pulse on last RX sample.
rx_data_o	out	64	RX shift register output. Valid after rx_wr_o.
sck_o	out	1	SPI clock output
cs_o	out	1	Chip select (active high). Deasserted in IDLE and DONE.
data_io	inout	4	SPI data bus (tri-state). Driven in CMD/ADDR/TX-DAT, sampled in RX-DAT.

I/O of the QSPI BPI

Table 6.32 shows the input and outputs of the QSPI BPI.

Table 6.32: as_slave_bpi Ports

Port	Dir.	Width	Description
rst_i	in	1	System reset, active high (unused internally; for consistency)
clk_i	in	1	System clock (unused internally; for consistency)
Peripheral (kernel) side:			
addr_o	out	64	Decoded byte offset address, directly from wb_s_addr_i
dat_from_core_i	in	64	Read data from peripheral, presented combinatorially
dat_to_core_o	out	64	Write data to peripheral, directly from wb_s_dat_i
wr_o	out	1	Write enable to peripheral. Combinatorial.
rd_o	out	1	Read enable to peripheral. Combinatorial.
Wishbone slave side:			
wb_s_addr_i	in	64	Wishbone address (byte offset from peripheral base)
wb_s_dat_i	in	64	Wishbone write data
wb_s_dat_o	out	64	Wishbone read data
wb_s_we_i	in	1	Wishbone write enable
wb_s_sel_i	in	8	Wishbone byte select (all bytes must be selected)
wb_s_stb_i	in	1	Wishbone strobe (valid cycle)
wb_s_ack_o	out	1	Wishbone acknowledge (combinatorial, zero wait state)
wb_s_cyc_i	in	1	Wishbone cycle (high for complete bus transaction)

Internals of the QSPI BPI

Table 6.33 shows the internal signals and processes of the QSPI BPI.

Table 6.33: QSPI BPI Internals

Object	Type	Width	Explanation
any	logic	x	See BPI spec.

BPI Transfer Timing

Table 6.34: Wishbone Classic Zero-Wait-State Timing

Cycle	Description
T0	Master asserts CYC, STB, ADR, DAT (write) or ADR (read), WE, SEL
T0	Slave combinatorially asserts ACK. Read data appears on DAT_O.
T1	Master samples ACK and DAT_O on rising clock edge.
T1	Master deasserts STB (and CYC for single transfers).

Note: The peripheral register file is clocked (`always_ff`), so a write takes effect on the posedge of the clock during which `WR_O` is asserted. Read data must be available combinatorially before the posedge.

I/O of the QSPI Top

Table 6.35 shows the input and outputs of the QSPI Top.

Table 6.35: `as_qspi_top` Ports

Port	Dir.	Width	Description
`rst_i`	in	1	Synchronous reset, active high
`clk_i`	in	1	System and kernel clock
`wbdAddr_i`	in	64	Wishbone address (byte offset from QSPI base)
`wbdDat_i`	in	64	Wishbone write data
`wbdDat_o`	out	64	Wishbone read data
`wbdWe_i`	in	1	Wishbone write enable
`wbdSel_i`	in	8	Wishbone byte select
`wbdStb_i`	in	1	Wishbone strobe
`wbdAck_o`	out	1	Wishbone acknowledge
`wbdCyc_i`	in	1	Wishbone cycle
`sck_o`	out	1	SPI clock to NOR-Flash
`cs_o`	out	1	Chip select to NOR-Flash (active high)
`data_io`	inout	4	QSPI data bus to/from NOR-Flash
`qspi_irq_o`	out	1	Interrupt request to CPU interrupt controller

Table 6.36 shows the internals of the QSPI Top.

Table 6.36: QSPI Top Signals

Signal	Width	Explanation
tbd	1	tbd

6.3.5 Service Requests of the QSPI

Interrupt sources are level-based and derived directly from kernel status or FIFO fill levels.

The details will be shown in section 6.3.7.

6.3.6 The QSPI Peripheral Kernel

Block Architecture -: The block architecture is like:

```
Wishbone
   |
Register File + IRQ Logic (SRB)
   |
QSPI Kernel FSM
   |
TX FIFO / RX FIFO
   |
SPI PHY (SCK / CS / IO[3:0])
```

The kernel is fully state-driven and does not depend on external events such as interrupts or bus timing.

Writing bit 3 of **QSPI0_CTRL .START** initiates a transaction if not busy.

Design Rules -:

- The kernel FSM is fully state-driven.
- No FSM state depends on interrupt signals.
- Interrupts never control hardware behavior.
- All external behavior is driven by register state only.

6.3.7 Registers

In the following the registers of a QSPI peripheral are listed and described. Some registers' layout or availability depends on the generic feature set implemented for that QSPI's instantiation. Such registers and concerned bit fields are denoted by the use of italic style.

QSPI Register Overview

Table 6.37: `as_qspi_top` Register Map

Offset	Symbol	Reset	Access	Description
0x00 (0)	ID	0x00000000_00000010	r	Peripheral identification register
0x08 (8)	CTRL	0x00000000_00000000	rw	Control register (START, FLUSH, mode bits)
0x10 (16)	CMD	0x00000000_00000000	rw	SPI opcode
0x18 (24)	ADDR	0x00000000_00000000	rw	Flash target address
0x20 (32)	LEN	0x00000000_00000000	rw	Transfer length in bytes
0x28 (40)	DUMMY	0x00000000_00000000	rw	Dummy cycles
0x30 (48)	CLKDIV	0x00000000_00000000	rw	SCK clock divider
0x38 (56)	TIMEOUT	0x00000000_00000000	rw	Timeout threshold
0x40 (64)	ISR	0x00000000_00000000	w	Interrupt Set Register (w1 $\rightarrow$ set RIS)
0x48 (72)	RIS	0x00000000_00000000	rh	Raw Interrupt Status Register
0x50 (80)	IMSC	0xFFFFFFFF_FFFFFFFF	rw	Interrupt Mask Control Register
0x58 (88)	MIS	0x00000000_00000000	rh	Masked Interrupt Status (= RIS & IMSC)
0x60 (96)	ICR	0x00000000_00000000	w	Interrupt Clear Register (w1 $\rightarrow$ clear RIS)
0x68 (104)	RXDATA	0x00000000_00000000	r	RX FIFO pop (read-before-pop)
0x70 (112)	TXDATA	0x00000000_00000000	w	TX FIFO push
0x78 (120)	FIFOSTAT	0x00000000_00000000	rh	FIFO fill levels and flags
0x80 (128)	XIPMODE	0x00000000_000000A0	rw	XIP mode byte (Winbond default: 0xA0)
0x88 (136)	STATUS	0x00000000_00000000	rh	Kernel status flags

Read-Write Features

- **r** – read-only; writes have no effect
- **w** – write-only; reads return 0
- **rw** – read/write; value is retained after reset
- **rh** – read-only; value is set by hardware
- **X0** – read always returns 0; write-only access

System Registers

QSPI0_ID -: Identification register of the QSPI peripheral. Read-only; writes have no effect.

ID register [Reset value: 0000000000000010_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								PERIPH_ID							
r								r							

Field	Bits	Type	Description
-	63:8	r	Reserved, reads as 0
PERIPH_ID	7:0	r	Peripheral identification number. Fixed value `0x10`.

Special Function Registers

QSPI0_CTRL -: Main control register. Controls SPI mode, address length, clock polarity/phase, XIP, DDR, CS-hold, FIFO flush, and transfer start. All configuration bits must be set before asserting START.

Control register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	RF	TF	AL	CP	CH	DU	CS	XI	DD	QU	ST	0	0	0
r	r	w	w	rw	rw	rw	rw	rw	rw	rw	rw	w	r	r	r

Field	Bits	Type	Description
-	63:14	r	Reserved
RF	13	w	RX_FLUSH. Write 1 to flush RX FIFO. Self-clearing.
TF	12	w	TX_FLUSH. Write 1 to flush TX FIFO. Self-clearing.
AL	11	rw	ADDR_LEN. 0 = 3-byte address (24 bit); 1 = 4-byte (32 bit).
CP	10	rw	CPOL. Clock polarity. 0 = idle low (Mode 0); 1 = idle high (Mode 3).
CH	9	rw	CPHA. Clock phase. 0 = sample on leading edge; 1 = sample on trailing edge.
DU	8	rw	DUAL. Enable dual-wire data (1-1-2). Ignored when QUAD=1.
CS	7	rw	CS_HOLD. Keep CS asserted between consecutive transactions.
XI	6	rw	XIP. Enable execute-in-place mode.
DD	5	rw	DDR. Enable double data rate.
QU	4	rw	QUAD. Enable quad-wire data (1-1-4 / 1-4-4). Overrides DUAL.
ST	3	w	START. Write 1 to initiate a transfer when BUSY=0. Self-clearing.
-	2:0	r	Reserved.

QSPI0_CMD -: The SPI/QSPI command opcode to be sent in the CMD phase.

Command register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	OPCODE							
r	r	r	r	r	r	r	r	rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
OPCODE	7:0	rw	SPI command opcode. Sent MSB-first in single-wire CMD phase.

QSPI0_ADDR -: Target byte address in the NOR-Flash device.

Address register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
ADDR[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
ADDR[15:0]															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
ADDR	31:0	rw	Flash byte address. Sent MSB-first. Number of address bytes determined by CTRL.AL.

QSPI0_LEN -: Transfer data length in bytes.

Length register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
LEN															
rw															

Field	Bits	Type	Description
-	63:16	r	Reserved
LEN	15:0	rw	Data transfer length in bytes. Must be ≥ 1.

QSPI0_DUMMY -: Number of dummy clock cycles between ADDR and DATA phases.

Dummy register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	DUMMY_CYC					
r	r	r	r	r	r	r	r	r	r	rw					

Field	Bits	Type	Description
-	63:6	r	Reserved
DUMMY_CYC	5:0	rw	Dummy cycle count. 0 = no dummy phase (ADDR directly followed by DATA). Maximum 63.

QSPI0_CLKDIV -: SCK clock divider. Determines the SPI clock frequency.

$$f_{SCK} = \frac{f_{clk}}{2 \times (\text{CLKDIV} + 1)}$$

Must not be changed while BUSY=1.

Clock divider register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								CLKDIV							
r								rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
CLKDIV	7:0	rw	SCK divider. Reset 0x00 = $f_{clk}/2$ (maximum SCK).

QSPI0_TIMEOUT -: Timeout threshold in kernel clock cycles. When the FSM is active and the down-counter reaches zero, a timeout error is raised. Set to 0 to disable.

Timeout register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TIMEOUT_THR[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TIMEOUT_THR[15:0]															
rw															

Field	Bits	Type	Description
-	63:32	r	Reserved
TIMEOUT_THR	31:0	rw	Timeout threshold in clock cycles. 0x00000000 = disabled.

QSPI0_TXDATA -: Writing to this register pushes data into the TX FIFO. If the TX FIFO is full, the write is silently discarded; the Wishbone ACK is still asserted. Reading returns 0.

TX data register (write pushes to TX FIFO) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
TX_DATA[63:48]															
w															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
TX_DATA[47:32]															
w															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
TX_DATA[31:16]															
w															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
TX_DATA[15:0]															
w															

Field	Bits	Type	Description
TX_DATA	63:0	w	Write data word pushed into TX FIFO. MSB transmitted first. Reads return 0.

QSPI0_RXDATA -: Reading this register pops one 64-bit word from the RX FIFO and returns it. The pop (pointer advance) is delayed by one clock cycle (read-before-pop semantics). If the RX FIFO is empty, the read returns 0x0000000000000000 and no pop occurs. Writing has no effect.

RX data register (read pops from RX FIFO) [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
RX_DATA[63:48]															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
RX_DATA[47:32]															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
RX_DATA[31:16]															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
RX_DATA[15:0]															
r															

Field	Bits	Type	Description
RX_DATA	63:0	r	Read data word popped from RX FIFO. MSB received first. Writes have no effect.

QSPI0_FIFOSTAT -: FIFO fill level and status flags. Read-only. Updated combinatorially.

FIFO status register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0	0	0	0	0	0	RF	RH	0	0	0	RX_LEV[4:0]				
r	r	r	r	r	r	rh	rh	r	r	r	rh				
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	TF	TH	0	0	0	TX_LEV[4:0]				
r	r	r	r	r	r	rh	rh	r	r	r	rh				

Field	Bits	Type	Description
-	63:26	r	Reserved
RF	25	rh	RX FIFO full
RH	24	rh	RX FIFO $\geq$ half full (level $\geq$ FIFO_DEPTH/2)
-	23:21	r	Reserved
RX_LEV	20:16	rh	RX FIFO fill level (0…16)
-	15:10	r	Reserved
TF	9	rh	TX FIFO full
TH	8	rh	TX FIFO $<$ half full (level $<$ FIFO_DEPTH/2)
-	7:5	r	Reserved
TX_LEV	4:0	rh	TX FIFO fill level (0…16)

QSPI0_XIPMODE -: XIP mode byte driven on IO[7:4] (upper nibble on IO[3:0]) during the XIP address phase to terminate the mode register entry sequence (e.g. Winbond W25Q128JV: 0xA0).

XIP mode byte register [Reset value: $00000000000000A0_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0								XIP_BYTE							
r								rw							

Field	Bits	Type	Description
-	63:8	r	Reserved
XIP_BYTE	7:0	rw	Mode byte for XIP address phase. Default 0xA0 (Winbond W25Q128JV).

QSPI0_STATUS -: Kernel status register. All bits are read-only and updated combinatorially.

Note on DONE: `STATUS.DONE` reflects the combinatorial `stat_done_o` signal which is high only while the kernel is in DONE_ST (one clock cycle). It is

not suitable for polling. Use `RIS.DO` instead, which latches the DONE event and holds it until cleared via ICR.

Status register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	XI	DO	BU
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT. Timeout occurred; latched until ICR.TIMEOUT is written.
ER	3	rh	ERROR. Error latched; latched until ICR.ERROR is written.
XI	2	rh	XIP_ACTIVE. XIP mode is currently active.
DO	1	rh	DONE. High for exactly one clock cycle when FSM leaves DAT_ST. For polling use RIS.DO.
BU	0	rh	BUSY. FSM is active (not IDLE).

Interrupt Registers

Interrupt sources are level-based and derived directly from kernel status.

Flow of Interrupt Requests -: Figure 6.5 shows the flow of a single interrupt request, which comes from the Peripheral Kernel or from the FIFO Controller.

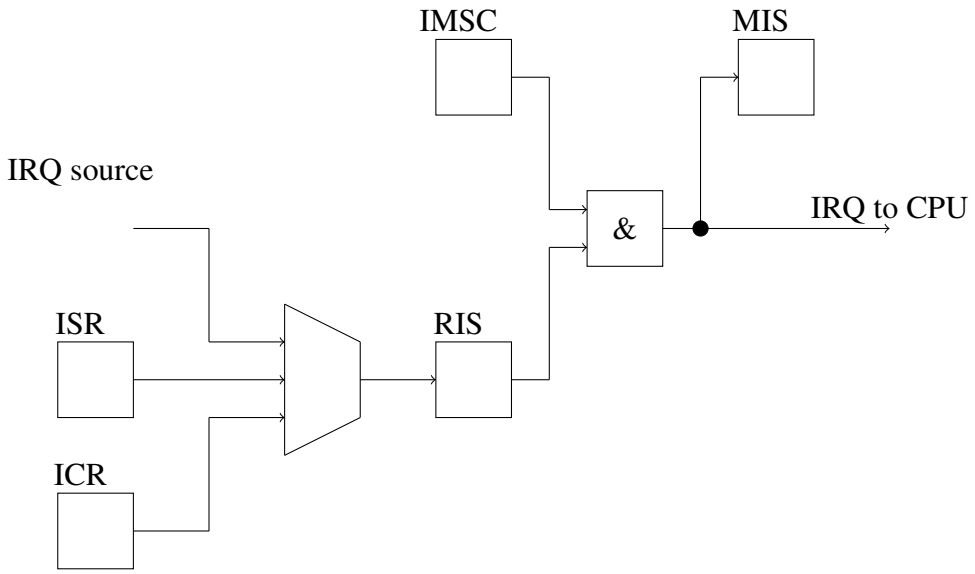

Figure 6.5: Interrupt Request Flow

A pulse on an interrupt request <xyz>_IRQ sets the corresponding interrupt status bit in the Raw Interrupt Status Register RIS. The interrupt status bits can be set with the Interrupt Set Register ISR or can be cleared with the Interrupt Clear Register ICR.

The Interrupt Mask Control Register IMSC enables or disables the level sensitive interrupts to the ICU. The Masked Interrupt Status Register MIS gives the current masked status value of the corresponding interrupt.

If an IMSC bit is disabled while its interrupt is active, then the interrupt for the ICU will be removed, but only until this IMSC bit is enabled again, unless the interrupt has been cleared in the mean time.

If the IMSC bit is not enabled when an interrupt source becomes active, then the interrupt bit will only be set in the RIS register. If the corresponding IMSC bit is later enabled, the interrupt will consequently become active in the MIS register.

Before enabling an interrupt in the MIS bit, it is good practice to always first clear the corresponding interrupt in the RIS via ICR.

In QSPI:

$$MIS = RIS\&IMSC$$

and then the external interrupt output is:

$$IRQ = |MIS$$

Interrupt Sources -:

Table 6.38: QSPI Interrupt Sources

RIS Bit	Name	Source	Latch Semantics
0	DONE	Rising edge of `stat_done_s` (FSM enters DONE_ST)	Set on edge; hold until ICR
1	RXHALF	Rising edge of `rx_half_s` (RX FIFO $\geq$ half full)	Set on edge; hold until ICR
2	TXHALF	Rising edge of `tx_half_s` (TX FIFO $<$ half full)	Set on edge; hold until ICR
3	ERROR	Rising edge of `stat_error_s`	Set on edge; hold until ICR
4	TIMEOUT	Rising edge of `stat_timeout_s`	Set on edge; hold until ICR
63:5	–	–	Reserved

QSPI0_RIS -: Raw Interrupt Status Register. Read-only. Reflects latched interrupt conditions independent of masking. Set by hardware edge events or by ISR write. Cleared by ICR write. Writing has no effect (use ISR/ICR instead).

Raw Interrupt Status Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT. Timeout threshold reached.
ER	3	rh	ERROR. Error latched in kernel.
TH	2	rh	TXHALF. TX FIFO below half-full threshold.
HA	1	rh	RXHALF. RX FIFO at or above half-full threshold.
DO	0	rh	DONE. Transfer completed. Latches on rising edge of `stat_done_s`.

QSPI0_IMSC -: Interrupt Mask Control Register. Each bit enables (1) or disables (0) the corresponding interrupt source. Reset value `0xFFFFFFFFFFFFFFFF` means all interrupts are enabled after reset.

Interrupt Mask Control Register [Reset value: $FFFFFFFFFFFFFFFF_H$]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
IMSC[63:48]															
rw															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
IMSC[47:32]															
rw															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
IMSC[31:16]															
rw															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

Field	Bits	Type	Description
-	63:5	rw	Reserved (kept for forward compatibility; writes ignored by hardware)
TI	4	rw	TIMEOUT mask. 1 = interrupt enabled.
ER	3	rw	ERROR mask.
TH	2	rw	TXHALF mask.
HA	1	rw	RXHALF mask.
DO	0	rw	DONE mask.

QSPI0_MIS -: Masked Interrupt Status Register. Read-only. Returns the effective interrupt status as seen by the CPU. MIS $=$ RIS $\&$ IMSC.

Masked Interrupt Status Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	rh	rh	rh	rh	rh

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	rh	TIMEOUT masked
ER	3	rh	ERROR masked
TH	2	rh	TXHALF masked
HA	1	rh	RXHALF masked
DO	0	rh	DONE masked

QSPI0_ICR -: Interrupt Clear Register. Write-only. Writing 1 to a bit clears the corresponding RIS bit. Writing 0 has no effect. Reading returns 0. The interrupt should be cleared only after the condition has been handled (FIFO drained / refilled) to avoid re-triggering.

Interrupt Clear Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	w	w	w	w	w

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	w	Write 1 to clear RIS.TIMEOUT
ER	3	w	Write 1 to clear RIS.ERROR
TH	2	w	Write 1 to clear RIS.TXHALF
HA	1	w	Write 1 to clear RIS.RXHALF
DO	0	w	Write 1 to clear RIS.DONE

QSPI0_ISR -: Interrupt Set Register. Write-only. Provided for software testing of the interrupt path. Writing 1 forces the corresponding RIS bit to 1. Writing 0 has no effect. Reading returns 0.

Interrupt Set Register [Reset value: 0000000000000000_H]

63	62	61	60	59	58	57	56	55	54	53	52	51	50	49	48
0															
r															
47	46	45	44	43	42	41	41	39	38	37	36	35	34	33	32
0															
r															
31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
0															
r															
15	14	13	12	11	10	09	08	07	06	05	04	03	02	01	00
0	0	0	0	0	0	0	0	0	0	0	TI	ER	TH	HA	DO
r	r	r	r	r	r	r	r	r	r	r	w	w	w	w	w

Field	Bits	Type	Description
-	63:5	r	Reserved
TI	4	w	Write 1 to force-set RIS.TIMEOUT (test only)
ER	3	w	Write 1 to force-set RIS.ERROR (test only)
TH	2	w	Write 1 to force-set RIS.TXHALF (test only)
HA	1	w	Write 1 to force-set RIS.RXHALF (test only)
DO	0	w	Write 1 to force-set RIS.DONE (test only)

6.3.8 Software Programming Sequence

Normal Single Read (e.g. 0x0B Fast Read, 8 B)

1. Write `CLKDIV` to set the desired SCK frequency.
2. Write `CMD` = `0x0B`.
3. Write `ADDR` = target flash address.
4. Write `LEN` = `8`.
5. Write `DUMMY` = `8`.
6. Optionally write `IMSC` to unmask DONE interrupt (bit 0).
7. Write `CTRL` = `0x08` (START = 1, all mode bits = 0 for single mode).
8. Poll `RIS.DO = 1` or wait for `qspi_irq_o`.
9. Write `ICR` = `0x01` to acknowledge DONE.
10. Read `RXDATA` to retrieve received bytes (one 64-bit read per 8 bytes).

Quad Read (e.g. 0x6B Quad Output Fast Read, 8 B)

Same as above, with `CTRL` = `0x18` (QUAD=1, START=1, bit 4 and bit 3 set) and `DUMMY` = `8`.

TX Write (e.g. 0x02 Page Program, 8 B)

1. Write `TXDATA` with up to 8 bytes (pushed to TX FIFO).
2. Program `CMD`, `ADDR`, `LEN`, `DUMMY` as needed.
3. Write `CTRL` = `0x08` (START = 1).
4. Poll `RIS.DO` or wait for IRQ. Clear with ICR.

XIP Enable

1. Configure `CLKDIV`, `DUMMY`, `CMD` (e.g. 0xEB).
2. Write `CTRL` = `0x50` (QUAD=1, XIP=1, START=0).
3. XIP becomes active after the next DONE. `STATUS.XI` = 1.
4. CPU reads to the flash address range are now automatically serviced.
5. To exit XIP: write `CTRL.XIP = 0`.

Chapter 7

Test and Debug

Not yet

Chapter 8

Electrical Characteristics

Not yet

Chapter 9

Memory Map and Registers

Not yet

Bibliography

[1] The RISC-V Instruction Set Manual, Volume I: User-Level ISA, Document Version 2.2, https://riscv.org/wp-content/uploads/2017/05/riscv-spec-v2.2.pdf, Accessed: 2024-02-23, 2017.

[2] RISC-V Guide, https://marks.page/riscv/, Accessed: 2024-02-23, 2024.

[3] Five EmbedDev, https://five-embeddev.com/riscv-isa-manual/latest/instr-table.html, Accessed: 2024-02-23, 2024.

[4] G. Lehmann, B. Wunder, and M. Selz, Schaltungsdesign mit VHDL, Synthese, Simulation und Dokume Poing: Franzis, 1994, ISBN: 978-3772361630. [Online]. Available: https://www.itiv.kit.edu/downloads/Buch_gesamt.pdf.